

Senior Symfony & PHP Engineer — Interview Prep Guide

Prepared for: Smart Working — Senior Symfony & PHP Software Engineer role

Structured edition note: This edition keeps the original interview-prep depth but reorganizes it into Parts, adds architecture diagrams, production caching/queues/deployment, a full Symfony + React API chapter, a package/configuration catalog, and a final Symfony-vs-Laravel framework judgment section.

Current-version note (verified September 2026): PHP **8.5** is the newest actively supported PHP branch. Symfony **8.1** is the current stable release, **7.4** is the current LTS, and **6.4 LTS** is still maintained. Symfony 8.0 is already unmaintained. The JD does not pin a version, so answer primarily from the versions you have genuinely used and use 7.4/8.1 as *current-awareness*, not invented hands-on experience. Official references: [Symfony releases](#), [Symfony docs](#), [PHP supported versions](#).

How to use this guide

This is built specifically against the JD you shared. It's organized to match what they'll actually probe:

JD requirement	Where it's covered
5+ yrs PHP (incl. OOP internals, data structures)	Section 1, incl. 1.7
PHP version changelog: 7.0 → 8.5, what changed and why	Section 1.8
3+ yrs Symfony (Laravel ≠ qualifying)	Sections 2, 7
Routing, CLI (bin/console), Maker commands, package ecosystem	Section 2.4, 2.13, 2.14
Your Symfony 2.8 / 4 / 5 + some 6 hands-on story, plus current-version awareness	Section 2.15
Twig templating, admin panels (EasyAdmin/Sonata)	Section 2.5, 2.16
React / Next.js + Symfony (headless API architecture)	Section 2.17
2+ yrs PHPUnit	Section 4
MongoDB (incl. migrations/data evolution, native depth, NoSQL injection)	Section 3, incl. 3.9, 3.10; Section 6.6
Design Patterns (OOP + enterprise: Repository, Value Object, Data Mapper)	Section 5
Database theory & data structures/complexity	Section 6.8, 6.9
On-call, code review, mentoring, docs	Section 6 (incl. 6.10 code review checklist), Section 8 (behavioral)
System design / open-ended practice	Section 6.11

Your interview positioning — memorize this before anything else

Your strongest story is **real Symfony depth across generations**, not pretending you have spent years on the newest release:

"My hands-on Symfony experience is strongest in **2.8, 4 and 5**, with some exposure to **6**. That means I have worked with both the older explicit style — manual service configuration, older bundle/application structures and annotation-era code — and the newer direction around autowiring, attributes and the modern Security system. I keep current with the framework, so I know **7.4 is the current LTS and 8.1 is current stable**, but I distinguish current knowledge from production experience."

That answer is safer and stronger than saying you "worked through 2.8 to 8.x". A senior interviewer can immediately probe exact APIs, deprecations and migration pain. **Never turn awareness into fake production experience.**

For every technical answer, use this four-part shape:

Definition → Symfony example → trade-off/failure mode → real project example (only if true).

Example: "A Voter centralizes object-level authorization. In Symfony I use `denyAccessUnlessGranted()` and keep the ownership/business rule in a Voter. The benefit is reusable/testable authorization; the trade-off is that too many tiny voters can make policy flow harder to discover. In one project..."

JD-specific risk order

1. **Symfony depth** — mandatory and explicitly used to filter Laravel-only candidates. Be fluent in services, container, routing, HttpKernel lifecycle, Security/Voters, Console, Messenger, Validator and configuration.
2. **MongoDB** — the most distinctive must-have. Know native Mongo concepts, not only ODM syntax: embedding vs references, indexes, aggregation, atomicity, transactions, concurrency, schema evolution and query plans.
3. **PHPUnit** — be able to write a unit test live, mock a dependency, explain unit vs integration vs functional tests, and say what should *not* be mocked.
4. **OOP/design patterns** — don't merely define Strategy/Factory/Decorator/Observer; tie them to Symfony services and explain when *not* to add a pattern.
5. **Senior engineering behavior** — code review, mentoring, decomposition, secure coding, observability, incident handling and communicating trade-offs with Product/QA.

If your real MongoDB or PHPUnit production experience is lighter than the JD asks for, **do not invent years**. Demonstrate depth through the concepts and code in this guide and phrase it accurately: "My strongest production depth is Symfony/PHP; I have worked with/around MongoDB and I can explain the design, indexing and concurrency model clearly." Adjust that sentence to match what you have genuinely done.

Work through it top to bottom, then use **Section 9** to actually run and modify code — muscle memory beats memorized definitions.

If you're short on time — a priority path

This guide is comprehensive by design, but comprehensive isn't the same as cramable. If you don't have time to read all ~1,300 lines before the interview, prioritize in this order:

1. **Section 9's end-to-end walkthrough** first, not last — it's the fastest way to see how every other section connects, and it's the single best rehearsal for "walk me through how you'd build this."
2. **Section 5** (Design Patterns) — explicitly named as a must-have, and the highest-density payoff per minute read.

3. **Section 3.1-3.3 and 3.10** — MongoDB is the JD's most distinctive requirement; this is where a generic "I know Symfony" candidate gets separated from a real fit.
 4. **Section 2.15** — your honest 2.8 / 4 / 5 / some-6 story plus 7.4/8.1 awareness. It directly defuses the JD's Laravel-only concern without overclaiming.
 5. **Section 10** — five minutes, and it's the difference between knowing the material and actually landing it in the room. Everything else rewards a full read if you have the time, but these five are where the marginal value per minute is highest.
-

Part I — PHP Foundations

Section 1 — PHP Fundamentals (and why each one matters in Symfony)

Symfony is written in idiomatic modern PHP OOP. Interviewers for a "Senior" role rarely ask "what is a variable" — they ask PHP questions **through** the lens of "how does this show up in Symfony." So each topic below has a direct "→ Symfony" line.

1.1 OOP Core

- **Class/Object:** blueprint vs instance. Properties (typed since PHP 7.4), methods, visibility (public/protected/private).
- **Constructors & constructor property promotion** (PHP 8.0+):

```
// Old way
class Money {
    private int $amount;
    private string $currency;
    public function __construct(int $amount, string $currency) {
        $this->amount = $amount;
        $this->currency = $currency;
    }
}

// PHP 8 promoted properties — Symfony code is full of this
final class Money {
    public function __construct(
        private readonly int $amount,
        private readonly string $currency = 'USD',
    ) {}
}
```

→ **Symfony:** nearly every service and DTO in modern Symfony code uses promoted, readonly constructor properties.

- **Abstract classes vs Interfaces:** abstract classes share partial implementation + state; interfaces define a contract only. PHP 8.1+ allows interfaces to have public constants. → **Symfony:** the container makes Dependency Inversion practical: type-hint abstractions such as `LoggerInterface` or your own repository interface, then bind an implementation in configuration. The container *facilitates* DIP; good architecture still depends on how you design the boundaries.
- **Traits:** horizontal code reuse when inheritance doesn't fit (PHP has single inheritance). Conflicts resolved with `insteadof` / `as`. → **Symfony:** `KernelTestCase` traits, `LoggerAwareTrait`, and many bundles ship traits for cross-cutting concerns like ID/UUID generation on entities.
- **Static members:** belong to the class, not the instance. Be ready to explain why **overusing statics breaks testability** (can't easily mock/swap) — a very "senior" answer, and directly relevant since DI is the whole point of Symfony.

- **Magic methods:** `__construct`, `__destruct`, `__get`/`__set`, `__call`/`__callStatic`, `__invoke`, `__toString`, `__clone`. → **Symfony:** Doctrine **lazy-loading proxies** override `__get`/`__call`/`__clone` to defer DB hits until a property is actually accessed — this is *the* classic "explain a Doctrine gotcha" interview question (see Section 3).

1.2 Namespaces, Autoloading, Composer

- PSR-4 autoloading maps namespace → directory (App\Service\FooService → src/Service/FooService.php).
- `composer.json` (declared deps + version constraints) vs `composer.lock` (exact resolved versions — **always commit this**).
- Semantic versioning (`^2.3` = compatible up to next major). → **Symfony:** the entire `src/` skeleton, bundle structure, and Symfony Flex recipe system rely on PSR-4 + Composer plugins.

1.3 Type System & Modern PHP (8.0 → 8.4)

Feature	Since	Example
Union types	8.0	<code>function find(int string \$id): ?User</code>
Named arguments	8.0	<code>new Money(amount: 500, currency: 'PKR')</code>
Match expression	8.0	<code>match(\$status) { 200, 201 => 'ok', default => 'fail' }</code>
Nullsafe operator	8.0	<code>\$user?->getAddress()?->getCity()</code>
Attributes	8.0	<code>#[Route('/api/users')]</code>
Enums	8.1	<code>enum Status: string { case Active = 'active'; }</code>
Readonly properties	8.1	<code>public readonly string \$id;</code>
Fibers	8.1	Cooperative-concurrency primitive; know the concept, but don't claim Symfony HttpClient depends on Fibers
Readonly classes	8.2	Whole class immutable by default
Typed class constants	8.3	<code>public const int MAX = 10;</code>

Interviewer favorite: "Why do enums matter for Symfony code you used to write with class constants?"

Answer: Enums are type-safe (can't pass an invalid string), IDE-autocompletable, can implement interfaces, and can carry methods — replacing the old "constants + validation" pattern for things like order status, user roles, etc.

1.4 Error Handling

- Throwable is the root; Error (programming mistakes, e.g. TypeError) and Exception (runtime issues) both implement it.
- Always catch the most specific type first; never blanket-catch \Throwable in business logic without re-throwing or logging. → **Symfony**: the `kernel.exception` event is where Symfony turns any uncaught Throwable into an HTTP response — this is your entry point for custom error pages / API error formats (see Section 7).

1.5 Generators, Closures, SPL

- **Generators (yield)**: lazy iteration without loading everything into memory. Great for processing large DB result sets or CSV imports.

```
function readLargeFile(string $path): \Generator {
    $handle = fopen($path, 'r');
    while (($line = fgets($handle)) !== false) {
        yield trim($line);
    }
    fclose($handle);
}
```

→ **Symfony**: Doctrine's `toIterable()` (formerly `iterate()`) returns a generator so you can batch-process millions of rows without exhausting memory — a very real senior-level talking point.

- **Closures / arrow functions**: use `($var)` captures by value; use `(&$var)` captures by reference. Arrow functions `fn() =>` auto-capture by value. → **Symfony**: event listeners, `array_map/array_filter` in service logic, and Messenger handlers all lean on closures.
- **SPL interfaces**: `ArrayAccess`, `Iterator`, `IteratorAggregate`, `Countable`. Symfony's `ParameterBag`, `HeaderBag`, and `Collections` (`ArrayCollection` from Doctrine Collections) implement these so they behave like arrays.

1.6 SOLID quick recall (you *will* be asked to define these live)

- **Single Responsibility** — one reason to change.
- **Open/Closed** — open for extension, closed for modification (strategy pattern, event listeners).
- **Liskov Substitution** — subclasses must be usable wherever the parent is expected.
- **Interface Segregation** — many small interfaces > one fat interface.
- **Dependency Inversion** — depend on abstractions, not volatile concretions. Symfony's container makes this easy through constructor injection and interface aliases; it does not automatically make poorly designed code SOLID.

1.7 Advanced OOP internals & PHP's built-in data structures

Late static binding — `static::` resolves to the *calling* class at runtime, `self::` always resolves to the class where the code is literally written. This trips people up in abstract base classes/repositories:

```
abstract class Model {
    public static function create(): static // "static" as a return type (PHP
8.0+) = late static binding
    {
        return new static(); // instantiates the CALLING subclass, not Model
    }
}
class Order extends Model {}
Order::create(); // returns an Order, not a Model – because of `static::`, not
`self::`
```

→ **Symfony**: Doctrine's base repository patterns and some abstract test-case helpers rely on this so subclasses "just work" without overriding factory methods.

Object cloning — shallow vs deep: PHP's `clone` does a **shallow copy** by default (nested objects are still shared references). Override `__clone()` to deep-clone nested objects:

```
class Order {
    public function __construct(public array $items, private Customer $customer)
    {}
    public function __clone(): void
    {
        $this->customer = clone $this->customer; // without this line, both
Orders share the same Customer object
    }
}
```

Object comparison: `==` compares property values (loose — two different instances with identical properties are "equal"); `===` checks object *identity* (same instance in memory). A classic gotcha question.

Serialization: `serialize()/unserialize()` for PHP's native format; `JsonSerializable` (implement `jsonSerialize()`) to control how an object becomes JSON (used constantly when an entity/document needs a custom API shape); PHP 7.4+ prefers `__serialize()/__unserialize()` magic methods over the older `Serializable` interface.

Covariance & contravariance (PHP 7.4+): an overriding method may **widen** accepted parameter types (contravariance) and **narrow** its return type (covariance) compared to the parent:

```
class Animal {}
class Dog extends Animal {}

class AnimalShelter { public function adopt(): Animal { return new Animal(); } }
class DogShelter extends AnimalShelter { public function adopt(): Dog { return
new Dog(); } } // covariant return – legal
```

→ **Symfony**: this is *why* a repository method can be typed to return `Order[]` in a subclass while the parent interface says `object[]`, without breaking Liskov Substitution.

PHP arrays are ordered hash maps, not raw arrays — a fact senior PHP devs should be able to explain: `array_key_exists()`/`isset` lookups are $\sim O(1)$ average (hash table), but `in_array()`/`array_search()` by value are $O(n)$ since PHP has to scan. This directly explains why looking up by ID in a large associative array is fast, but "does this array contain this value" on a large list is not — relevant when someone asks you to optimize a hot loop.

SPL data structures (`ext-spl`, built into PHP, worth knowing exist even if you reach for plain arrays 95% of the time):
 | Class | Behavior | Typical use | |---|---|---| | `SplStack` | LIFO (extends doubly linked list) | undo history, DFS traversal, call-stack style logic | | `SplQueue` | FIFO | job/task processing order, BFS traversal | | `SplDoublyLinkedList` | Base for Stack/Queue | $O(1)$ push/pop at both ends | | `SplFixedArray` | Fixed-size, faster/lighter than a normal array | large numeric datasets, performance-critical loops | | `SplObjectStorage` | Map keyed by *object identity*, not scalars | attaching metadata to objects without adding properties to them (e.g. Doctrine's `UnitOfWork` uses this internally) | | `SplPriorityQueue` / `SplMinHeap` / `SplMaxHeap` | Ordered by priority/value, $O(\log n)$ insert | scheduling, "top-N" problems |

→ **Symfony**: Doctrine's internal Unit of Work tracks entity identity using an `SplObjectStorage`-like structure — this is a great "I actually looked under the hood" answer if asked how Doctrine tracks which entities are dirty.

1.8 PHP Version Changelog: 7.0 → 8.5 — what actually changed, release by release

PHP ships one feature release per year (November, since the 8.x line settled into an annual cadence). Below is the full path from PHP 7.0 (the "PHPNG" rewrite — PHP 6 was never released; the project skipped straight to 7) to today's PHP 8.5, organized into three eras. Knowing *why* a feature landed, not just that it exists, is what separates "I've used PHP 8" from a senior-level answer.

Era 1: PHP 7.0 → 7.4 — the performance-and-types era

Version	Released	Most important changes
7.0	Dec 2015	PHPNG performance rewrite — roughly 2x faster than 5.6 with dramatically lower memory use, from a restructured Zend Engine. Scalar type declarations (int, float, string, bool) and return types. Null coalescing operator ?? . Spaceship operator <code><=></code> . Anonymous classes. Throwable interface — engine errors (TypeError, DivisionByZeroError) became catchable instead of silently fatal. <code>yield from</code> (generator delegation). <code>random_bytes()/random_int()</code> (CSPRNG). Removed the old <code>mysql_*</code> extension and PHP 4-style constructors.
7.1	Dec 2016	Nullable types (<code>?int</code>). void return type. Multi-catch : <code>catch (TypeA TypeB \$e)</code> . Symmetric array destructuring with keys (<code>['id' => \$id] = \$row</code>). Class constant visibility modifiers. iterable pseudo-type.
7.2	Nov 2017	object type declaration. Parameter type widening on override. sodium (modern crypto) bundled by default.
7.3	Dec 2018	Flexible heredoc/nowdoc syntax. Trailing commas in function calls. <code>list()</code> reference assignment. <code>JSON_THROW_ON_ERROR</code> . <code>array_key_first()</code> / <code>array_key_last()</code> .

Version	Released	Most important changes
7.4	Nov 2019	Typed properties — the single biggest DX shift of the 7.x line. Arrow functions (fn(\$x) => \$x * 2). Null coalescing assignment ??=. Spread operator in array literals ([...\$a, ...\$b]). Numeric literal separators (1_000_000). Preloading (opcache.preload) for long-lived workers. WeakReference . Deprecated the old curly-brace offset syntax (\$str{0}).

Why this era mattered for a Symfony dev: typed properties (7.4) and scalar/return types (7.0) are exactly what made Symfony's later push toward strict typing and type-hint-driven autowiring practical in the first place — PHP and Symfony's roadmaps have always moved together, not independently.

Era 2: PHP 8.0 → 8.4 — the "attributes and JIT" era (this is what Symfony 6–8 was built on)

Version	Released	Most important changes
8.0	Nov 2020	JIT compiler (opt-in; helps CPU-bound code more than typical I/O-bound web requests). Union types (int string). Named arguments . Attributes ([Route(...)]) — replaced docblock annotations at the language level; <i>this single feature is what reshaped Symfony and Doctrine from version 6.0 onward (Section 2.15, Q18)</i> . Constructor property promotion . Match expression . Nullsafe operator ?->. mixed type, static return type, Stringable, WeakMap. Saner string-to-number comparisons (0 == "foo" is now false). Non-capturing catches, throw as an expression. <code>str_contains()/str_starts_with()/str_ends_with()</code> .

Version	Released	Most important changes
8.1	Nov 2021	Enums (pure and backed) — directly fed Symfony's 5.4→6.3 enum-support push (Section 2.15). Readonly properties . Fibers (cooperative multitasking primitive). First-class callable syntax (strlen(...)). never return type. Pure intersection types. new in initializers. array_is_list().
8.2	Nov 2022	Readonly classes . DNF types ((A&B) null). Standalone null/false/true types. Dynamic properties deprecated (opt back in with #[AllowDynamicProperties]) — a real strictness tightening. #[SensitiveParameter] — auto-redacts arguments like passwords from stack traces. New Random\Randomizer class.
8.3	Nov 2023	Typed class constants . #[Override] attribute — errors at compile time if the parent method you're "overriding" doesn't actually exist, catching refactor typos immediately. Dynamic class constant fetch. json_validate(). Readonly properties can now be modified inside __clone() — closes the exact gap the deep-clone pattern in Section 1.7 depends on.

Version	Released	Most important changes
8.4	Nov 2024	Property hooks — get/set logic defined directly on a property, killing boilerplate getter/setter pairs. Asymmetric visibility (public private(set) string \$id) — publicly readable, only internally writable, natively. New array_find(), array_any(), array_all(), array_find_key(). new Foo()->bar() — chain straight off a new expression without extra parens. Native lazy objects (ReflectionClass::newLazyGhost()/newLazyProxy()) — exactly what Symfony 7.3/8.0 plugged into for lazy services and Doctrine proxies (Section 2.15). New native HTML5-compliant DOM parser (Dom\HTMLDocument) — what Symfony's DomCrawler/HtmlSanitizer switched to in Symfony 8.0. mb_trim(), mb_ucfirst(), mb_lfirst(). Implicit nullable parameter types deprecated (function f(int \$x = null) without an explicit ?int now warns).

Why this era mattered for a Symfony dev — a strong answer to have ready: *"PHP and Symfony's roadmaps are tightly coupled, not coincidental: attributes in PHP 8.0 unlocked Symfony 6's routing/validation rewrite, enums in 8.1 fed straight into Symfony's 5.4-6.3 enum-support push, and PHP 8.4's native lazy objects are precisely what Symfony 7.3/8.0 adopted instead of maintaining their own generated-proxy code. I don't just track Symfony release notes — I track PHP RFCs, because Symfony's next move is usually whatever the language just made possible."*

Era 3: PHP 8.5 (current, released Nov 2025) and what's next

- **Pipe operator** |> — chain calls left-to-right without nesting or temp variables: `$result = $input |> trim(...) |> strtolower(...);` (turns "pyramid of doom" transformations into a readable top-to-bottom pipeline).
- **New URI extension** — built-in, spec-compliant URL parsing (RFC 3986 + WHATWG URL) without a userland library.
- **"Clone with"** — modify properties while cloning in a single expression instead of `clone` + manual reassignment.
- **Native array_first() / array_last()** — complements 7.3's `array_key_first()/array_key_last()`, finally in core.

- **#[\NoDiscard]** attribute — flags a function's return value as something that shouldn't be silently thrown away (e.g. a `Result`-style return), a static-analysis-friendly safety net.
- Closures, casts, and first-class callables now allowed inside **constant expressions** (default parameter values, class constants).
- **Fatal error backtraces** — a real stack trace now prints even on a fatal error, not just on exceptions.

What's next (PHP 8.6, targeted late 2026): still in active RFC discussion at the time of writing — **partial function application** is one of the headline proposals being debated. Mention it as "on my radar" rather than claiming hands-on experience with it yet.

Part II — Symfony Core Architecture

Section 2 — Symfony Architecture & Core Concepts

2.1 The big picture

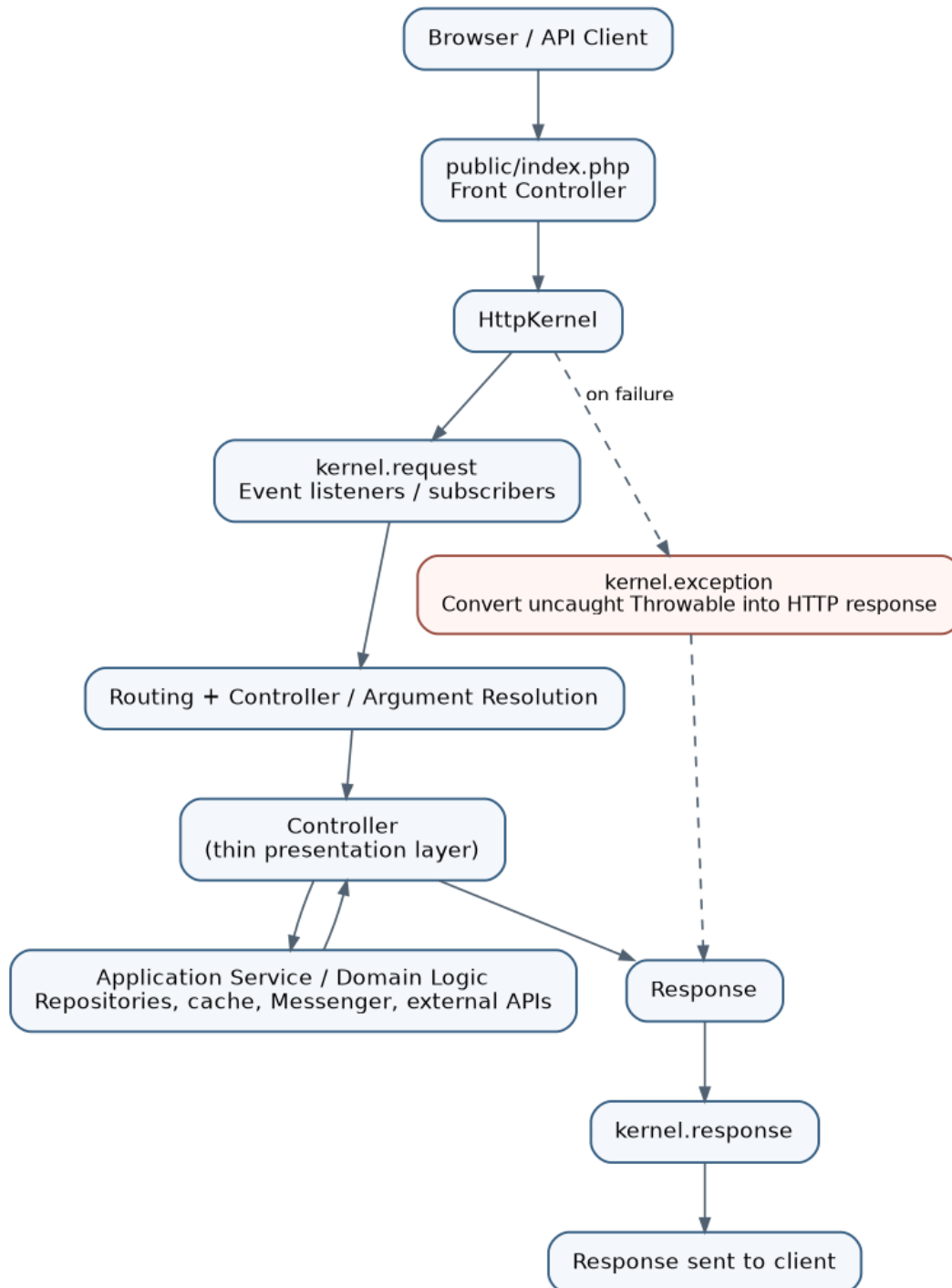
Symfony is two things at once:

1. A set of **decoupled components** (Routing, HttpFoundation, HttpKernel, EventDispatcher, DependencyInjection, Validator, Serializer, Messenger, Security, Console...) usable standalone in any PHP project.
2. A **full-stack framework** that wires those components together via the FrameworkBundle.

Being able to say *"Symfony is a set of components, the framework just glues them together with sensible config"* immediately signals seniority.

2.2 Request lifecycle (the #1 "explain the architecture" question)

Symfony HTTP Request Lifecycle



```

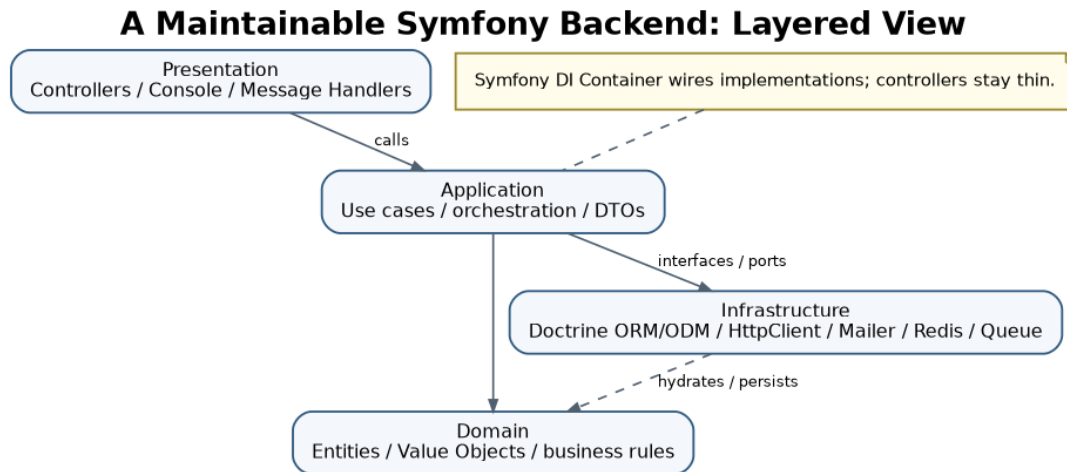
Browser → public/index.php → Kernel::handle(Request)
  → kernel.request      (Request → possibly early Response, e.g. cache)
  → Routing matches URL → Controller resolved
  → kernel.controller    (can swap the controller/args)
  → Controller executes → returns Response OR arbitrary value
  → kernel.view          (if controller didn't return a Response, converts
value → Response, e.g. API Platform)
  → kernel.response      (modify headers, add cookies, etc.)
  → Response sent to browser
  → kernel.terminate     (post-response cleanup: send emails, dispatch async

```

```
messages)
→ kernel.exception      (fired any time an exception escapes the above)
```

Architecturally, describe this as a **Front Controller (public/index.php) feeding the HttpKernel request/response pipeline, with EventDispatcher hooks around controller resolution/execution**. It feels middleware-like, but don't force the Chain of Responsibility label here. A cleaner Symfony example of Chain of Responsibility is **Messenger middleware**, where an envelope flows through an ordered middleware stack.

2.3 Dependency Injection Container (the heart of Symfony)



- **Services** = PHP objects Symfony manages for you. Since Symfony 4+, **autowiring** and **autoconfiguration** are on by default: type-hint an interface/class in a constructor, and the container figures out how to build it.

```
# config/services.yaml
services:
  _defaults:
    autowire: true
    autoconfigure: true
  App\:
    resource: '../src/'
```

- **Service tags**: mark a service so Symfony (or a compiler pass) can find all services of a "kind" — e.g. all App\EventSubscriber or all Messenger handlers use tags under the hood.
- **Public vs private services**: private (default) can only be fetched via injection, not `$container->get()` — encourages proper DI over service-locator anti-pattern.
- **Parameters vs Services**: parameters are scalar/array config values (`%kernel.environment%`), services are objects.

Common trap question: *"What's the difference between autowiring and manual service configuration, and when would you NOT use autowiring?"*

When you have multiple implementations of the same interface (e.g. two PaymentGatewayInterfaces) and need to bind a specific one, or when constructing 3rd-party classes that need scalar config values — you configure the service explicitly and use named arguments or a factory.

2.4 Routing (deep dive — this comes up constantly)

Attribute-based (modern default):

```
#[Route('/api/orders/{id}', name: 'order_show', methods: ['GET'], requirements:
['id' => '\d+'])]
public function show(int $id, OrderRepository $repo): JsonResponse
{
    $order = $repo->find($id) ?? throw new NotFoundHttpException();
    return $this->json($order);
}
```

Also possible via YAML/XML/PHP config — know all three exist, but attributes are current best practice. Routes can also be centralized in config/routes.yaml pointing at a controller directory, or defined per-bundle.

Route parameters & requirements

```
#[Route(
    '/api/orders/{id}/items/{itemId}',
    name: 'order_item_show',
    requirements: ['id' => '\d+', 'itemId' => '\d+'],
    defaults: ['itemId' => null], // optional parameter with a default
    methods: ['GET'],
)]
```

Class-level prefix + method-level route — common pattern for grouping a resource's endpoints:

```
#[Route('/api/orders', name: 'order_')]
final class OrderController extends AbstractController
{
    #[Route('', name: 'list', methods: ['GET'])]
    public function list(): JsonResponse { /* ... */ }

    #[Route('/{id}', name: 'show', methods: ['GET'], requirements: ['id' => '\d+'])]
    public function show(int $id): JsonResponse { /* ... */ }

    #[Route('', name: 'create', methods: ['POST'])]
    public function create(Request $request): JsonResponse { /* ... */ }
}
```

Auto entity/document resolution — Symfony can resolve {id} straight into an object via

`#[MapEntity]` (Doctrine ORM) so you skip the manual `$repo->find()` call:

```
#[Route('/api/orders/{id}', name: 'order_show')]
public function show(Order $order): JsonResponse // Symfony fetches it for
you, 404s automatically if not found
{
    return $this->json($order);
}
```

(For MongoDB documents there isn't a first-party resolver as mature as the Doctrine ORM one — you typically resolve manually via `DocumentManager` in the controller/service, or write a small custom `ValueResolverInterface`.)

Other things worth knowing:

- **Route caching:** Symfony compiles all routes into a matcher class at cache-warmup time — same "compiled for performance" idea as the DI container (see 2.13's `cache:clear`).
- **Sub-domain routing:** `#[Route('/dashboard', host: '{subdomain}.example.com')]` — useful for multi-tenant apps.
- **Localized routes:** `#[Route(['en' => '/about', 'fr' => '/a-propos'])]` for i18n URLs.
- **Redirecting/legacy routes:** `RedirectController` for permanent redirects during URL restructuring.
- **Debugging routes** — see 2.13 below (`debug:router` is one of the most-used commands day to day).

2.5 Twig Templating — deep dive

Even for an API-first Senior role, Twig still shows up (HTML emails via Mailer, admin panels, error pages, maybe a lightly server-rendered marketing site) — and "explain Twig" is a common warm-up question.

Core syntax

```
{# comment #}
{{ user.name }}           {# output — auto-escaped by default (XSS
protection) #}
{% if user.isActive %}...{% endif %}    {# logic/tags #}
```

Template inheritance (the **Template Method pattern** in action):

```
{# base.html.twig #}
<html>
<body>
    {% block content %}{% endblock %}
</body>
</html>

{# order/show.html.twig #}
{% extends 'base.html.twig' %}
{% block content %}
    <h1>Order #{{ order.id }}</h1>
{% endblock %}
```

Includes vs embed: `{% include 'partial.html.twig' %}` renders another template as-is; `{% embed %}` does the same but lets you override the *included* template's own blocks — useful for reusable card/widget components with a couple of customizable slots.

Macros — reusable template "functions", avoid copy-pasted markup:

```
{% macro input(name, value = '') %}
    <input name="{{ name }}" value="{{ value }}">
{% endmacro %}
{{ _self.input('email') }}
```

Filters vs Functions: filters transform a value with pipe syntax (`{{ name|upper }}`, `{{ price|number_format(2) }}`); functions are called directly (`{{ path('order_show', {id: 5}) }}`, `{{ dump(order) }}` in dev). Know the difference when writing a **custom Twig extension**:

```
final class PriceExtension extends AbstractExtension
{
    public function getFilters(): array
    {
        return [new TwigFilter('formatted_price', $this->formatPrice(...))];
    }

    public function formatPrice(float $amount, string $currency = 'USD'): string
    {
        return sprintf('%s %.2f', $currency, $amount);
    }
}
```

Global variables: registered via config/packages/twig.yaml (twig: globals: app_name: '%env(APP_NAME)%') — accessible in every template without passing them explicitly from every controller.

Form theming: Twig also renders Symfony Forms (`{{ form(form) }}` / `{{ form_row(form.email) }}`), and you can override the default form theme globally or per-form to match a design system.

HTML emails: symfony/mailer + Twig integration lets you write emails as templates (`>htmlTemplate('emails/welcome.html.twig')`) with the same inheritance/macro tooling as your web pages.

Performance: Twig templates are compiled to plain PHP classes and cached (`var/cache/{env}/twig`) — same "compile once, run fast" philosophy as the DI container and router. In dev, templates recompile on every change; in prod, always warm the cache before serving traffic.

When it's NOT the right tool: for a fully decoupled API + React/Next.js frontend (Section 2.17), Twig typically only survives for emails and maybe an internal admin panel — the API itself returns JSON via the Serializer, not Twig-rendered HTML.

2.6 Validator Component

```
class RegisterUserDto {
    #[Assert\NotBlank]
    #[Assert\Email]
    public string $email;

    #[Assert\Length(min: 8)]
    public string $password;
}
```

Custom constraints implemented as Constraint + ConstraintValidator pairs — know how to write one, it's a common live-coding ask.

2.7 Security Component

- **Firewalls:** define which authentication applies to which URL patterns (config/packages/security.yaml).

- **Authenticators** (modern system, Symfony 5.3+): replace the old "guard" authenticators; implement `AuthenticatorInterface` (or extend `AbstractAuthenticator`) with `authenticate()`, `onAuthenticationSuccess()`, `onAuthenticationFailure()`.
- **User Providers**: load users (Doctrine entity provider, or custom, e.g. from MongoDB).
- **Voters**: fine-grained, attribute-based authorization (`$this->denyAccessUnlessGranted('EDIT', $post)`) — implement `VoterInterface`/extend `Voter`. This is the **Strategy pattern** in action.
- **Password hashing**: `PasswordHasherInterface`, auto-migrates hashing algorithms.

2.8 Console Component

Custom commands extend `Command`, implement `configure()` and `execute()`. Services can be injected via constructor just like anywhere else — commands are just services tagged `console.command`.

2.9 EventDispatcher

- **Listener**: registered per-event in config, simplest form.
- **Subscriber**: implements `EventSubscriberInterface::getSubscribedEvents()`, self-describing, preferred for anything non-trivial because the event wiring lives with the code.
- Priorities control execution order (higher runs first).

2.10 Messenger Component (async / CQRS)

- **Message** = plain PHP object (a Command or Query/Event).
- **Handler** = tagged service implementing the logic, resolved by message type.
- **Bus** = dispatches the message to its handler(s), possibly through **middleware** (validation, logging, DB transaction wrapping).
- **Transport**: sync (default, same request) or async (`doctrine://`, `amqp://`, `redis://`, `sqs://`) — async is how you offload slow work (emails, exports) outside the request/response cycle, which pairs directly with the JD's on-call/reliability expectations.

2.11 Cache Component

PSR-6 (`CacheItemPoolInterface`) and PSR-16 (`CacheInterface`, simpler `get()/->get($key, callback)`). Supports **cache tags** for group invalidation and adapters for Redis/Memcached/filesystem/APCu.

2.12 Environments & Config

- `.env`, `.env.local`, `.env.$APP_ENV.local` layering.
- `config/packages/{env}/*.yaml` for per-environment overrides (dev/test/prod).
- `%env(resolver:VAR_NAME)%` syntax to pull secrets/config from env vars at runtime — critical for "how do you manage secrets" questions.

2.13 The CLI: symfony binary vs bin/console, MakerBundle, and the commands you'll actually use daily

There are **two different CLI tools** — interviewers sometimes probe whether you know the distinction:

Tool	What it is	Example
symfony (Symfony CLI)	A standalone binary (Go, installed once on your machine) for <i>local dev tooling</i> — not part of your project's code	symfony serve -d, symfony check:requirements, symfony new
bin/console	A PHP script inside your project , your gateway to every registered Console Command (framework + your own)	php bin/console cache:clear

symfony CLI — local development ergonomics

```

symfony new my_project --webapp      # scaffold a new full-stack project
symfony serve -d                     # Local HTTPS dev server (daemonized)
symfony check:requirements           # verify PHP extensions/config match
what Symfony needs
symfony console <any bin/console command> # proxies to bin/console with the
right PHP version/env
symfony var:export                    # print resolved env vars for debugging

```

bin/console — the commands you'll run constantly

```

php bin/console list                  # see every available command,
grouped by namespace
php bin/console about                 # project/env/PHP/Symfony
version summary – great first debugging step

# Debugging (huge for interviews AND real work)
php bin/console debug:router          # list every route, method,
and path
php bin/console debug:router order_show # inspect one route in detail
php bin/console debug:container       # list every service in the
container
php bin/console debug:container --types # see registered
interface→implementation bindings
php bin/console debug:autowiring LoggerInterface # "why isn't autowiring
picking this up" – THE go-to debug command
php bin/console debug:event-dispatcher kernel.request # see every listener on
an event, in priority order
php bin/console debug:config framework # dump the resolved config for
a bundle

# Cache
php bin/console cache:clear           # clear the compiled
container/routes/twig cache
php bin/console cache:warmup          # pre-build caches (common in
a deploy script, before traffic hits)

# Doctrine (ORM side, if the project mixes SQL + MongoDB)

```

```

php bin/console doctrine:database:create
php bin/console doctrine:migrations:diff           # generate a migration from
entity changes
php bin/console doctrine:migrations:migrate
php bin/console doctrine:fixtures:load --append

# Doctrine MongoDB ODM
php bin/console doctrine:mongodb:schema:create     # create indexes defined via
#[ODM\Index]
php bin/console doctrine:mongodb:fixtures:load

# Messenger
php bin/console messenger:consume async           # start a worker consuming
the "async" transport
php bin/console messenger:failed:show            # inspect messages that
landed in the failure transport
php bin/console messenger:failed:retry           # retry failed messages

# Security
php bin/console security:hash-password           # interactively hash a
password with the configured algorithm

# Assets / translations
php bin/console assets:install
php bin/console translation:extract en

```

MakerBundle (symfony/maker-bundle, dev-only dependency) — scaffolding that writes boilerplate for you. Interviewers love asking "how do you generate X" because it reveals whether you actually work in Symfony day-to-day:

```

php bin/console make:controller OrderController
php bin/console make:entity Order                 # interactive: add fields,
types, relations (ORM)
php bin/console make:migration
php bin/console make:form OrderType
php bin/console make:voter OrderVoter
php bin/console make:command app:import-orders
php bin/console make:subscriber OrderSubscriber
php bin/console make:test --unit OrderServiceTest
php bin/console make:security:form-login          # scaffold a form-login flow
(current Maker command)
php bin/console make:security:custom              # scaffold a custom
authenticator
php bin/console make:crud Order                   # full CRUD controller +
templates from an existing entity

```

Note: `make:entity` and the Doctrine `migrations:*` commands are ORM/SQL-oriented. Since this JD is MongoDB-centric, you typically hand-write `#[ODM\Document]` classes (Section 3.2) rather than using `make:entity` — worth mentioning proactively if asked, since it shows you understand *why* the tooling differs by datastore, not just that it does.

A senior-level point to make if asked "how do you approach a new/unfamiliar Symfony codebase":

```

"First php bin/console about and debug:router to get the lay of the land, then
debug:container --types or debug:autowiring when I need to find or swap a specific
service, rather than grepping blindly through services.yaml."

```

2.14 Packages & Bundles You Should Recognize (Composer + Symfony Flex)

Symfony's ecosystem is Composer-driven, and **Symfony Flex** (a Composer plugin installed by default in new projects) is what makes composer require X "just work" — when you require a Flex-aware package, Flex pulls a **recipe** from the public [symfony/recipes](https://github.com/symfony/recipes) repo and auto-drops config files (e.g. config/packages/mongodb.yaml), registers the bundle in config/bundles.php, and adds relevant .env entries — no manual bundle registration needed, unlike older Symfony versions. Know this flow cold; it's a common "explain what happens when you run composer require" question.

Package	Purpose
symfony/framework-bundle	The core framework glue (routing, controllers, DI wiring)
symfony/maker-bundle (dev)	Code scaffolding — Section 2.13
doctrine/orm + doctrine/doctrine-bundle	Relational ORM, if the project also touches SQL
doctrine/mongodb-odm-bundle	MongoDB ODM integration — the JD's core requirement
doctrine/doctrine-migrations-bundle	Versioned schema migrations (ORM side)
symfony/security-bundle	Authentication/authorization, firewalls, voters
lexik/jwt-authentication-bundle	Common JWT auth layer for stateless APIs on top of Security
symfony/validator	Constraint-based validation (Section 2.6)
symfony/serializer	Object ↔ JSON/XML normalization, powers API responses
symfony/messenger	Async messaging / CQRS-style command & event buses
symfony/mailer	Modern replacement for SwiftMailer, transport-agnostic email sending
symfony/monolog-bundle	Structured logging integration (Section 6.7)
symfony/cache	PSR-6/16 caching with Redis/APCu/filesystem adapters
symfony/rate-limiter	Token bucket / sliding window throttling (Section 7, Q14)
symfony/workflow	State machines / workflows (Section 7, Q10)
symfony/uid	UUID/ULID generation with proper typing

Package	Purpose
nelmio/cors-bundle	CORS header handling for browser-based API clients
api-platform/core	Full REST/GraphQL API framework layered on Symfony (auto CRUD, OpenAPI docs) — worth knowing exists even if this role hand-rolls controllers
phpunit/phpunit + symfony/test-pack (dev)	Testing — Section 4
symfony/phpunit-bridge (dev)	Deprecation-tracking helper integrated into PHPUnit runs
friendsofphp/php-cs-fixer / phpstan/phpstan (dev)	Code style + static analysis, standard in CI pipelines (Section 6.4)
symfony/webpack-encore-bundle	Frontend asset bundling, only relevant if the role touches any server-rendered UI

A good answer if asked "what packages have you worked with": don't just list names — pick 2-3 and say *what problem each solved* for you. "I used symfony/messenger with a Redis transport to move CSV export generation off the request cycle" is a far stronger answer than reciting the table above.

2.15 Symfony evolution: your 2.8 / 4 / 5 / some-6 experience → current 7.4/8.1 awareness

Use this section as an **evolution map**, not a claim that you personally shipped every intermediate version. Your stated hands-on experience is **Symfony 2.8, 4, 5 and some 6**. That already gives you a strong migration/legacy-maintenance story: you understand the explicit pre-Flex world and the modern autowired/attribute-based world. Symfony releases minors in May/November, with recurring LTS lines such as 2.8, 3.4, 4.4, 5.4, 6.4 and 7.4. Learn the 7/8 rows as current-awareness so you can discuss modernization intelligently without saying you ran them in production if you did not.

Era 1: Symfony 2.8 → 4.4 (your starting point to the last pre-5.0 LTS)

Version	Released	Most important changes
2.8 (LTS)	Nov 2015	Your starting point. app/AppKernel.php manually listed every bundle in registerBundles(). Routing via @Route docblock annotations (SensioFrameworkExtraBundle, not core Symfony). Services wired mostly by hand in services.yml — no autowiring. The Guard authentication component debuted here — a big simplification over the old provider-based auth system at the time.
3.0	Nov 2015	Released alongside 2.8 with zero new features — purely removed everything 2.8 had marked deprecated. New components landed too: PropertyInfo, LDAP, and the Guard component moved to core.
3.1	May 2016	Simplified anonymous access control config, first (experimental) version of the Workflow component, Lock component introduced.
3.2	Nov 2016	PHP 7.1 support, PSR-16 (SimpleCache) introduced, lazy services in the DI container.
3.3	May 2017	The big one for DX: opt-in autowiring & autoconfiguration — no more manually listing every constructor argument in YAML. Symfony Flex launched as a standalone Composer plugin. PSR-11 ContainerInterface adopted.

Version	Released	Most important changes
3.4 / 4.0	Nov 2017	Released together (same features, different BC-break policy — 4.0 removes 3.4's deprecations). Flex became the default project skeleton: new public/config/src/ layout, no more app/AppKernel.php (replaced by a slim Kernel.php using MicroKernelTrait), .env files, bundles auto-registered via config/bundles.php. Autowiring/autoconfiguration on by default . Services private by default . New debug:autowiring command.
4.1	May 2018	Messenger component introduced (experimental) — message bus pattern for CQRS/async processing.
4.2	Nov 2018	Continued Messenger/Flex maturation, Symfony Contracts groundwork (small decoupled interfaces libraries other packages could depend on without pulling in full components).
4.3	May 2019	Three major new (experimental) components in one release: HttpClient (sync/async HTTP requests), Mailer (SwiftMailer's eventual replacement, Mime-based), Mime (MIME message manipulation). Contracts split into separate per-concern packages (Cache, EventDispatcher, HttpClient, Service, Translation).
4.4 (LTS)	Nov 2019	Messenger/Mailer/HttpClient continue stabilizing toward 5.0. Automatic password rehashing when a stronger algorithm becomes available. New PHPUnit assertions for testing sent emails. Mailer/Mime gain message signing/encryption support.

Safe talking point for this era: "In 2.8 I recognize the older AppKernel/explicit-service style; by Symfony 4 the Flex/autowiring approach made configuration much lighter. That contrast is something I can discuss from hands-on work across those generations."

Era 2: Symfony 4.4 → 5.4 (the 5.x series)

Version	Released	Most important changes
5.0	Nov 2019	Mostly a deprecation-cleanup release paired with 4.4 (same pattern as 3.4/4.0) — no major new features of its own.
5.1	May 2020	The new Authenticator-based Security system shipped experimentally for the first time (enable_authenticator_manager: true). New Uid component (UUIDs and ULIDs).
5.2	Nov 2020	New RateLimiter component, new Semaphore component (concurrent access to a shared resource), login throttling (brute-force login protection) and login links / "magic links" authentication.
5.3	May 2021	The new Authenticator system hit stable and became the recommended approach — and Guard was officially deprecated (removed in 6.0). New standalone PasswordHasher component (extracted out of Security). Auto-generated PHP config builder classes for every installed bundle. Official Tailwind CSS form theme.

Version	Released	Most important changes
5.4 (LTS)	Nov 2021	PHP 8.1 Enum support across Form/Serializer/DependencyInjection/VarExporter. Route aliasing . Cacheable security voters for performance. Console autocompletion . Messenger handlers configurable via PHP attributes . Configurable log level/HTTP status per exception class.

Safe talking point for this era: "Symfony 5 is where the modern authenticator model became the direction of travel and Guard was deprecated. If the interviewer asks whether I personally migrated Guard code, I'll answer from the project I actually worked on rather than claiming a migration I didn't perform."

Era 3: Symfony 5.4 → 6.4 (the 6.x series — attributes take over)

Version	Released	Most important changes
6.0	Nov 2021	Released alongside 5.4. Requires PHP 8.0+. PHP 8 attributes become the idiomatic way to write routes, validation, and Doctrine mapping (annotations still worked via a bridge, but attributes were now "the way"). Old Guard system fully removed .
6.1	May 2022	New HtmlSanitizer component (safely strip/restrict HTML from user-generated content). New LocaleSwitcher service. Routing requirements can use PHP backed enums ; UTF-8 route parameters supported.
6.2	Nov 2022	Enum support expanded : usable in service parameters, YAML config files, and env var processors — building on 5.4's initial Form/Serializer enum support.

Version	Released	Most important changes
6.3	May 2023	New Scheduler component — cron-like recurring Messenger messages defined in PHP instead of crontab entries. Further enum improvements (enum() function in ExpressionLanguage, enum values in Config component nodes, enums as default route-parameter values).
6.4 (LTS)	Nov 2023	Mature modern LTS on PHP 8.1+: attributes are the normal style across routing/validation and much of the ecosystem; legacy annotation-based integrations are increasingly deprecated or removed depending on component/bundle.

Safe talking point for this era: "In Symfony 6-style code I prefer PHP attributes for routes and validation because they are native language metadata and work cleanly with IDE/static-analysis tooling. On an older project I would modernize them incrementally rather than rewriting unrelated code just for syntax."

Era 4: Symfony 6.4 → 7.4 (the 7.x series — leaning into modern PHP)

Version	Released	Most important changes
7.0	Nov 2023	Released alongside 6.4. Requires PHP 8.2+ , which unlocked using readonly classes and DNF (disjunctive normal form) types natively inside Symfony's own code, not just yours.
7.1	May 2024	New #[Lazy] attribute for lazy services — mark a whole service class or a single injected constructor argument as lazily instantiated, without the older lazy-service XML/YAML config ceremony.

Version	Released	Most important changes
7.2	Nov 2024	TypeInfo component graduates out of experimental status (introspecting PHP types, including PHPDoc-derived types, used internally by the Serializer/PropertyInfo). Mostly an iterative-polish release otherwise — Mailer webhook support (Mailtrap), minor String/ExpressionLanguage additions.
7.3	May 2025	Symfony's lazy services start using PHP 8.4's native lazy objects when available (no code changes needed — just upgrade PHP). New PHP attributes for Twig extensions (lazy-loaded by default). New <code>isGrantedForUser()</code> method to check permissions for an arbitrary user, not just the current one. Enum values usable directly in Config node definitions. HTTP/3 and IPv6 support added to HttpClient .
7.4 (LTS)	Nov 2025	New slim helper traits so controllers can use URL generation / Twig rendering / message dispatching <i>without</i> extending the large <code>AbstractController</code> base class. Began deprecating the implicit <code>\$request->get()</code> parameter-bag fallback in favor of explicit <code>\$request->query->get()</code> / <code>->request->get()</code> / <code>->attributes->get()</code> — a deliberate, type-safety-driven cleanup ahead of 8.0.

Current-awareness talking point: "7.4 is the current LTS. I would usually favor the LTS line for a conservative long-lived production system unless the team has a reason to adopt the current feature release."

Era 5: Symfony 7.4 → 8.1 (current)

Version	Released	Most important changes
8.0	Nov 2025	PHP 8.4+ generation; now unmaintained as of Sep 2026. Important architectural direction: Symfony increasingly adopts native modern-PHP capabilities (including native lazy-object support) instead of maintaining equivalent userland machinery.
8.1 (current stable)	May 2026	Current stable Symfony line as of Sep 2026; requires PHP 8.4+. Treat its exact feature list as current-awareness and consult the official changelog when discussing a specific new API.
8.2	Nov 2026 (future at interview time)	In development. Do not present unreleased features as settled behavior; mention only that you track the roadmap if asked.

A model answer for "tell me about your Symfony experience across versions":

"My strongest hands-on experience is Symfony 2.8, 4 and 5, with some Symfony 6. I started in the older world where service wiring and bundle configuration were much more explicit, then worked with the Flex/autowiring era in 4 and the Security/DI improvements in 5, and I have used some of the attribute-based modern style in 6. I also keep up with current Symfony — 7.4 is the LTS and 8.1 is current stable — so I understand where the framework is going, but I don't present those newer versions as production experience I haven't had. That mix is useful because I can maintain older Symfony safely while also moving code toward modern patterns incrementally."

This directly counters the JD's "Laravel-only is not qualifying" concern: your answer names Symfony-specific architectural shifts and clearly separates hands-on experience from current-release awareness.

2.16 Symfony "Admin" — EasyAdminBundle & SonataAdminBundle

Symfony has **no built-in admin generator** (unlike, say, Django) — the ecosystem fills this gap with two well-known bundles. Know both exist and when you'd reach for each:

EasyAdminBundle — lightweight, convention-based, fast to bootstrap:

```
final class OrderCrudController extends AbstractCrudController
{
    public static function getEntityFqcn(): string { return Order::class; }
```

```

public function configureFields(string $pageName): iterable
{
    yield IdField::new('id')->hideOnForm();
    yield TextField::new('customerName');
    yield MoneyField::new('total')->setCurrency('USD');
    yield AssociationField::new('lines');
}

```

Good for: internal tools, back-office CRUD for small-to-medium teams, when you want an admin panel in hours, not days.

SonataAdminBundle — heavier, far more configurable, closer to an "enterprise admin framework": separate Admin classes define `configureListFields()`, `configureFormFields()`, `configureDatagridFields()`, has its own ACL integration, dashboard blocks, and menu system. Good for: complex back-offices with deep relations, per-role field-level permissions, and highly customized workflows.

The MongoDB wrinkle (relevant to this JD): both bundles are built primarily around **Doctrine ORM** — first-class MongoDB ODM support is thin-to-nonexistent in EasyAdmin and requires extra custom "Admin"/data-provider work in Sonata. In a Mongo-first codebase, it's common to either (a) write a slim custom admin controller set by hand, or (b) build the admin UI as a small internal React app hitting the same API the main frontend uses (ties into Section 2.17) rather than fighting an ORM-oriented admin generator. **Bring this up proactively if asked about admin panels** — it shows you understand the tooling's assumptions, not just its existence.

2.17 Symfony as an API backend for React / Next.js

Two real architectures, know both:

A) Fully decoupled/headless (most common for a React/Next.js pairing)

```

Next.js app (Vercel / Node) <--- JSON/REST or GraphQL ---> Symfony API
(stateless)

```

What needs configuring on the Symfony side:

- **CORS:** `nelmio/cors-bundle` — allow the frontend's origin(s), specific methods/headers. Without it, browser-side fetches from React/Next.js get silently blocked by the browser even if the API itself works fine (a very common "why is my API not working from the frontend" debugging question).
- **Stateless auth:** JWT via `lexik/jwt-authentication-bundle` (or OAuth2/OIDC), returned to the client and sent as `Authorization: Bearer <token>` — no server-side session, which is what "stateless API" means in practice. Refresh-token rotation is the usual follow-up question.
- **CSRF is largely a non-issue for pure Bearer-token APIs** (CSRF exploits ambient/cookie-based credentials — a Bearer token in a JS-set header isn't automatically sent by the browser). CSRF *does* matter again the moment you switch to `httpOnly` cookie-based auth for the SPA — then you need real CSRF tokens + `SameSite` cookie settings.

- **Serialization shape:** symfony/serializer (or **API Platform** on top of it) controls the exact JSON contract the frontend team codes against — groups/normalization context let you expose different fields per endpoint (e.g. hide internal fields from a public list endpoint).
- **API Platform** (mentioned in 2.14) is worth naming here specifically: it auto-generates OpenAPI docs a frontend team can use to generate a typed API client for React/Next.js (e.g. via openapi-typescript), removing a whole class of "the frontend and backend drifted out of sync" bugs.
- **Next.js SSR nuance worth mentioning in an interview:** when Next.js does server-side rendering (getServerSideProps / React Server Components), the **data-fetching call happens server-to-server** (often over an internal Docker network hostname, faster and not subject to browser CORS at all) — while client-side navigation/fetches from the *browser* hit the public API domain and go through CORS. Conflating these two is a common junior mistake; explicitly distinguishing them is a strong signal.
- **Real-time updates:** Symfony's **Mercure** component (Server-Sent Events-based pub/sub) is the Symfony-native way to push live updates to a React/Next.js frontend without standing up a separate WebSocket server.

B) Server-rendered Symfony + an embedded React "island" When you don't need a full SPA, just one interactive widget inside an otherwise Twig-rendered page: bundle the React component with **Webpack Encore** (Symfony's asset bundler) and mount it into a `<div id="widget-root">` inside a Twig template. This is different from (and usually simpler than) running a whole separate Next.js app — worth distinguishing if asked "how would you add React to a Symfony app" vs. "how would you connect Symfony to a Next.js app," since they're different problems with different answers.

Symfony UX (Stimulus + Turbo) is worth namedropping as the "native" alternative to reaching for React at all for simple interactivity — shows you know Symfony has its own answer to "modern frontend interactivity" and aren't defaulting to React out of habit.

2.18 Symfony Forms — FormBuilder, FormTypes, data transformers, form events

This JD doesn't name the Form component explicitly, but `make:form` (Section 2.13) and form theming (Section 2.5) are both mentioned only in passing elsewhere in this guide — and "how do you build a form in Symfony" is close to a certainty in a senior interview. This section covers the piece itself.

A FormType class — what `make:form` actually generates

```
final class OrderType extends AbstractType
{
    public function buildForm(FormBuilderInterface $builder, array $options):
    void
    {
        $builder
            ->add('customerEmail', EmailType::class)
            ->add('items', CollectionType::class, [
```

```

        'entry_type' => OrderItemType::class,
        'allow_add' => true,
        'allow_delete' => true,
    ])
    ->add('submit', SubmitType::class);
}

public function configureOptions(OptionsResolver $resolver): void
{
    $resolver->setDefaults([
        'data_class' => Order::class,
        'csrf_protection' => true,
    ]);
}
}

```

`buildForm()` declares the fields and their types; `configureOptions()` binds the form to a class (`data_class`) so submitted data hydrates a real object instead of a bare array. The controller then does the familiar `createForm()` / `handleRequest()` / `isValid()` dance — covered in Section 2.13's `make:form` entry, spelled out here.

Data transformers — when the form's shape and the model's shape differ

- A **ModelTransformer** converts between the underlying model data (e.g. an `Order` object) and a normalized form (e.g. an ID string) — it runs closest to the model.
- A **ViewTransformer** converts between that normalized form and what's actually rendered/submitted in HTML (e.g. a comma-separated tag string the browser submits, turned into an array) — it runs closest to the view.
- Classic use case: a `customerId` field that displays as a searchable text input but should hydrate a real `Customer` object on submit — without a transformer you'd manually convert in the controller, which doesn't compose if the field is reused.

Form events — changing a form's shape based on its data

- `FormEvents::PRE_SET_DATA` fires when the form is initialized with existing data — the standard place to conditionally add/remove fields (e.g. hide a status field unless editing an existing `Order`).
- `FormEvents::PRE_SUBMIT` / `SUBMIT` / `POST_SUBMIT` fire around submission — `PRE_SUBMIT` sees raw submitted data before it's mapped to the object, useful for normalizing input before validation runs.

```

$builder->addEventListener(FormEvents::PRE_SET_DATA, function (FormEvent
$event): void {
    $order = $event->getData();
    $form = $event->getForm();

    // Only an existing (persisted) Order gets a status field to edit.
    if ($order instanceof Order && $order->getId() !== null) {
        $form->add('status', ChoiceType::class, ['choices' => Order::STATUSES]);
    }
});

```

CSRF protection, and how it ties back to the Validator

- CSRF protection is on by default for any form using `FormTypeCsrfExtension` — a hidden token field is rendered automatically and checked on submit; `csrf_protection: false` is the explicit opt-out, and should be a deliberate, reviewed decision (Section 6.10's PR checklist), not a default.
- Submitting a form runs the Validator (Section 2.6) against the bound object's constraints automatically — `$form->isValid()` is really “no violations after mapping submitted data onto the object.” The `validation_groups` option lets a form activate only a subset of an object's constraints (Section 2.19).

MongoDB nuance worth stating proactively: Forms bind to any PHP object or array — they have no idea whether `data_class` is a Doctrine ORM Entity or an ODM Document. Everything above works identically for an `#[ODM\Document]` class; there's no Mongo-specific form integration to learn.

Q1. What is a `FormType`, and what does extending `AbstractType` give you?

A: A `FormType` is a reusable class describing one form's fields, validation binding, and options — extending `AbstractType` gives you `buildForm()` to declare fields and `configureOptions()` to configure defaults like `data_class`, so the same form definition can be reused across controllers/tests instead of being rebuilt inline with `createFormBuilder()` every time.

Q2. What's the difference between building a form directly with `FormBuilderInterface` in a controller vs a dedicated `FormType` class?

A: Building inline with `$this->createFormBuilder()` is fine for a one-off, throwaway form, but it isn't reusable, isn't independently testable, and can't easily be composed into another form. A `FormType` class is a first-class service-like unit — it can be unit tested on its own, reused across multiple controllers, and embedded as a sub-form inside another `FormType` (as the `OrderItemType` embedding above shows).

Q3. What are data transformers, and when do you need one?

A: They convert data between the model's representation and the form's representation when the two don't naturally match — a `ModelTransformer` works close to the underlying object (e.g. an entity/document to an ID), a `ViewTransformer` works close to the rendered HTML (e.g. an array to a comma-separated string). You need one whenever a single form field's displayed/submitted format isn't the same shape as the property it maps to.

Q4. What are form events, and give a concrete use case?

A: Form events (`PRE_SET_DATA`, `PRE_SUBMIT`, `SUBMIT`, `POST_SUBMIT`) let you hook into the form's lifecycle to change its structure or data based on what it's holding. The standard example is a create/edit form where an extra field (like `status`) should only exist when editing an already-persisted object — that decision can only be made once the form has its data, which is exactly what `PRE_SET_DATA` gives you.

Q5. How does CSRF protection work in a Symfony form by default?

A: A hidden token field is automatically added to the rendered form and tied to the form's name/session; on submission, Symfony regenerates the expected token and compares it, rejecting the submission if it doesn't match. It's on by default via `FormTypeCsrfExtension` — turning it off (`csrf_protection: false`) is an explicit, and rare, opt-out.

Q6. How do the Form and Validator components interact?

A: A form doesn't have its own validation rules — when `handleRequest()/isValid()` runs, Symfony maps submitted data onto the bound object (`data_class`) and then runs the Validator against that object's constraints, exactly as if you'd called `$validator->validate($object)` yourself. The `validation_groups` form option controls which subset of the object's constraint groups get checked for that particular form (Section 2.19).

Q7. How do you handle a create-vs-edit form where a field should only appear in one mode?

A: Add the field conditionally inside a `FormEvents::PRE_SET_DATA` listener, checking whether the bound object already has an identity (e.g. a non-null ID means it's being edited, not created) — shown in the code example above. This keeps one `FormType` class serving both modes instead of duplicating near-identical classes for create and edit.

Q8. Can you use Symfony Forms with a MongoDB ODM Document instead of a Doctrine ORM Entity?

A: Yes, without any special integration — the Form component only knows about `data_class` as a plain PHP class; it has no dependency on Doctrine ORM specifically. An `#[ODM\Document]` class works as a form's `data_class` exactly the same way an ORM `#[ORM\Entity]` would.

Q9. What's the difference between a form theme and manually writing the HTML for a field?

A: A form theme is a Twig template (or block) that controls how form types render by default across the whole app — change the theme once, and every `TextType/ChoiceType/etc.` picks up the new markup, which is how Bootstrap/Tailwind form styling gets applied project-wide (Section 2.5). Manually writing HTML with `form_row()/form_widget()` overrides that default for one specific field, which is the right call for a genuinely one-off layout but doesn't scale to theming an entire application.

Q10. How would you test a form-handling controller?

A: A Symfony functional test (`WebTestCase`, Section 4.4) that submits the form through the actual HTTP layer — `$client->request()` to load the form page, then `$client->submitForm()` with field values — exercises routing, the `FormType`, the Validator, and the controller together, which is usually more valuable here than mocking the form in a unit test, since the whole point of a form is wiring those pieces together correctly.

2.19 Validator Component, deeper — groups, cascading, custom constraints

Section 2.6 shows a single flat DTO with `#[Assert\NotBlank]`-style constraints and mentions that writing a custom constraint “is a common live-coding ask” without showing one. This section fills both gaps.

Validation groups — not every constraint should always run

- Every constraint belongs to the `Default` group unless told otherwise; `$validator->validate($object, groups: ['registration'])` only checks constraints tagged with that group, letting one object have different validation rules for different contexts (e.g. a User validated one way on registration, another way on profile update).
- A `GroupSequence` runs groups in order and **stops at the first group with a violation** — useful when a later group's constraints are expensive or meaningless to check if an earlier, cheaper group already failed (e.g. don't run a uniqueness DB query if basic format validation already failed).

Cascading validation into nested objects and collections

```
final class CreateOrderDto
{
    #[Assert\Valid]
    #[Assert\Count(min: 1)]
    /** @var OrderItemDto[] */
    public array $items = [];

    #[Assert\Valid]
    public ?ShippingAddressDto $shippingAddress = null;
}
```

Without `#[Assert\Valid]`, the Validator checks only the top-level object — constraints on `OrderItemDto` or `ShippingAddressDto` are silently skipped, even though they look correctly annotated on their own class. Forgetting `#[Assert\Valid]` on a nested property is one of the most common silent-validation-gap bugs in a Symfony codebase.

Validating data that isn't attached to a form

- Inject `ValidatorInterface` directly and call `$validator->validate($dto)` in a service, a Messenger handler, or a console command — form submission is only one path into the Validator, not a requirement for using it.
- This matters for this JD's Messenger/queue work specifically (Section 13): a message handler processing an async job still needs to validate the message payload's business invariants, and there's no HTTP form anywhere near that code path.

Writing a custom constraint — the live-coding version

```
#[Attribute]
final class NoProfanity extends Constraint
{
    public string $message = 'This text contains disallowed language.';
}
```

```

final class NoProfanityValidator extends ConstraintValidator
{
    public function validate(mixed $value, Constraint $constraint): void
    {
        if (!$constraint instanceof NoProfanity) {
            throw new UnexpectedTypeException($constraint, NoProfanity::class);
        }

        if ($value === null || $value === '') {
            return; // let NotBlank handle emptiness, don't duplicate that
concern
        }

        if (ProfanityList::containsDisallowedWord((string) $value)) {
            $this->context->buildViolation($constraint->message)
                ->addViolation();
        }
    }
}

```

Two classes, always paired: the Constraint is a plain data holder (the attribute you put on a property, carrying configuration like the message), and the ConstraintValidator holds the actual logic, resolved automatically by naming convention (XValidator for X) or by overriding `validatedBy()`. A validator should validate one thing and delegate adjacent concerns (like emptiness) to the constraints already built for them, exactly as `NotBlank` is left alone above.

Q1. What's a validation group, and when would you use one besides Default?

A: A group is a label on a constraint that lets you validate the same object differently depending on context — `$validator->validate($user, groups: ['registration'])` only checks constraints tagged `registration`, skipping others. Common use: a User object needs a password on registration but not on a later profile-edit form, without needing two separate classes.

Q2. What does GroupSequence let you do that a plain group doesn't?

A: It enforces order and short-circuits: groups run one at a time, and validation stops at the first group that has any violation, rather than collecting violations from every group at once. That's valuable when a later group's checks are expensive (e.g. a uniqueness constraint hitting the database) and pointless to run if a cheaper, earlier group (basic format checks) already failed.

Q3. How does cascading validation with `#[Assert\Valid]` work on a nested collection?

A: By default the Validator only inspects the object you hand it directly — constraints declared on a property's own type are invisible unless that property is marked `#[Assert\Valid]`, which tells the Validator to recurse into it (and, for an array/collection, into each element). Leaving it off a nested DTO or collection is a common way validation silently does nothing on data that looks fully annotated.

Q4. How do you validate a plain array or a service-layer DTO that isn't bound to a form?

A: Inject `ValidatorInterface` wherever you need it and call `$validator->validate($data)` directly — forms are one caller of the Validator, not a prerequisite for using it. This is exactly what you need for validating a Messenger message's payload inside a handler, or validating input to a console command, neither of which goes through an HTTP form at all.

Q5. Walk me through writing a custom constraint.

A: Two classes: a `Constraint` subclass that's just configuration (the message, any parameters) and is what you put on a property as an attribute, and a `ConstraintValidator` subclass holding the actual check, whose `validate()` method receives the value and the constraint instance and calls `$this->context->buildViolation(...)->addViolation()` when the value fails. Symfony resolves which validator belongs to which constraint by naming convention (`FooValidator` for `Foo`) unless you override `validatedBy()` to point elsewhere.

Q6. How do you turn a `ConstraintViolationList` into a clean JSON API error response?

A: Iterate the `ConstraintViolationListInterface` returned by `validate()` (or caught from a failed form), and map each `ConstraintViolationInterface` to a small array of its `getPropertyPath()` and `getMessage()`, then return that array as the response body's error list with a 422/400 status — this is exactly the shape Section 15's API error-handling needs, and it's the same violation list whether the constraint failed via a form or a direct `$validator->validate()` call.

2.20 Controller Action Patterns — invokable controllers, sub-requests, custom resolvers

Section 2.4 covers routing itself in depth; this section covers a few controller-side patterns interviewers ask about alongside routing that this guide hasn't named yet.

Invokable, single-action controllers

```
#[Route('/api/orders/{id}/cancel', name: 'order_cancel', methods: ['POST'])]
final class CancelOrderController extends AbstractController
{
    public function __construct(private OrderCanceller $canceller) {}

    public function __invoke(Order $order): JsonResponse
    {
        $this->canceller->cancel($order);
        return $this->json(['status' => 'cancelled']);
    }
}
```

- One class, one responsibility, one route — the class name itself documents what the endpoint does, which scales better than a growing `OrderController` accumulating unrelated actions as a project matures.
- Trade-off: for a genuinely CRUD-shaped resource, a grouped resource controller (Section 2.4's class-level prefix pattern) keeps related endpoints discoverable together; invokable controllers suit actions that don't fit CRUD (`cancel`, `refund`, `approve`) better than they suit `index/show/create`.

Sub-requests: forward() vs redirect()

- `redirect()` sends a new HTTP response telling the browser to make an entirely new request to a different URL — the URL bar changes, and it's a full round trip.
- `$this->forward('SomeController::action', ['id' => $id])` runs another controller **internally**, within the same request, and returns its Response as if it were your own — no second HTTP round trip, no URL change, and the browser never knows a second controller ran at all.
- Typical use: rendering a self-contained widget (e.g. a “related products” block) from inside another page's controller without duplicating its logic — the caching layer's ESI/fragment approach (Section 12) builds on this same sub-request idea for cacheable page fragments.

Custom argument resolvers — generalizing beyond #[MapEntity]

- `#[MapEntity]` (Section 2.4) is one built-in `ValueResolverInterface` implementation; you can write your own to resolve any controller argument automatically — a common one for an API is resolving a validated request-body DTO directly as a typed argument instead of manually reading and decoding `$request->getContent()` in every action.
- `#[CurrentUser]` is another built-in resolver — it injects the currently authenticated `UserInterface` directly as a controller argument, reading it from the security token storage, so you don't manually call `$this->getUser()` and then null-check it in every action.

Q1. When would you use an invokable single-action controller instead of a resource controller with multiple actions?

A: When the endpoint represents a distinct action rather than a CRUD operation on a resource — cancel, approve, refund — where grouping it into a generic `OrderController` alongside `list/show/create` would mix unrelated concerns into one growing class. For straightforward CRUD, a single resource controller with a class-level route prefix (Section 2.4) is usually more discoverable.

Q2. What's the difference between forward() and redirect(), and when would you pick forward()?

A: `redirect()` tells the client to make a brand-new HTTP request to a different URL; `forward()` runs another controller internally within the current request and returns its Response, with no second round trip and no URL change. Pick `forward()` when you need to reuse another controller's rendering logic as a fragment of the current response — for anything the user should be able to bookmark or share as its own URL, `redirect()` is correct instead.

Q3. What is a ValueResolverInterface, and how is it different from #[MapEntity]?

A: `ValueResolverInterface` is the general mechanism Symfony uses to decide what value to inject for a typed controller argument; `#[MapEntity]` is one specific built-in resolver that fetches a Doctrine ORM entity by route parameter. You can implement your own resolver for anything else you want resolved automatically — a validated request DTO, a value object parsed from a header — rather than manually building it inside every action that needs it.

Q4. What does `#[CurrentUser]` resolve, and why is it useful?

A: It injects the currently authenticated user directly as a typed controller argument, pulled from the security token storage — equivalent to calling `$this->getUser()` but without needing the controller to extend `AbstractController` or manually null-check an unauthenticated request. It keeps action signatures declarative about what they need instead of reaching into a helper method for it.

Part III — Data & Persistence

Section 3 — Doctrine & MongoDB (the JD's must-have)

The JD explicitly wants "**Strong experience with MongoDB.**" In a Symfony shop this almost always means **Doctrine MongoDB ODM (Object Document Mapper)** — the Mongo equivalent of Doctrine ORM. Know both, and know the differences.

3.1 ORM vs ODM — say this clearly if asked

	Doctrine ORM	Doctrine MongoDB ODM
Backing store	Relational (MySQL/Postgres)	MongoDB (document store)
Unit	Entity	Document
Manager	EntityManagerInterface	DocumentManager
Relationships	Joins, foreign keys	Embedded docs or references (no real joins)
Schema	Fixed columns/migrations	Schemaless (flexible fields per document)
Query language	DQL	Mongo query builder / aggregation pipeline

3.2 Defining a Document

```
use Doctrine\ODM\MongoDB\Mapping\Attribute as ODM;

#[ODM\Document(collection: 'orders')]
class Order
{
    #[ODM\Id]
    private string $id;

    #[ODM\Field(type: 'string')]
    private string $customerName;

    #[ODM\Field(type: 'float')]
    private float $total;

    #[ODM\EmbedMany(targetDocument: OrderLine::class)]
    private Collection $lines; // embedded sub-documents – no join needed

    #[ODM\ReferenceOne(targetDocument: Customer::class)]
    private ?Customer $customer = null; // reference – separate collection

    #[ODM\Field(type: 'date')]
    private \DateTimeInterface $createdAt;
}
```

3.3 Embedding vs Referencing — classic senior-level MongoDB question

- **Embed** when the sub-data is only ever accessed with the parent, has a bounded/small size, and doesn't need independent querying (e.g. order line items inside an order).
- **Reference** when the related data is large, shared across many parents, updated independently, or queried on its own (e.g. a Customer referenced by thousands of Orders).
- Trade-off: embedding = fewer round-trips but data duplication risk & document size limits (16MB/doc); referencing = normalized but needs extra queries (no native joins — you either do multiple queries or use \$lookup in an aggregation pipeline).

3.4 CRUD with DocumentManager

```
final class OrderService
{
    public function __construct(private DocumentManager $dm) {}

    public function create(Order $order): void
    {
        $this->dm->persist($order);
        $this->dm->flush(); // Unit of Work pattern – batches writes
    }

    public function findRecent(): array
    {
        return $this->dm->getRepository(Order::class)
            ->createQueryBuilder()
            ->field('createdAt')->gte(new \DateTime('-7 days'))
            ->sort('createdAt', 'desc')
            ->getQuery()
            ->execute()
            ->toArray();
    }
}
```

3.5 Aggregation Pipeline

Used for reporting/analytics-style queries that plain find() can't express:

```
$results = $dm->getRepository(Order::class)
    ->createAggregationBuilder()
    ->match()->field('status')->equals('completed')
    ->group()->field('id')->expr()->field('_id')->expression('$customerId')
        ->field('total')->sum('$total')
    ->sort(['total' => -1])
    ->getAggregation()
    ->getIterator()
    ->toArray();
```

Know the pipeline stage names by heart: \$match, \$group, \$project, \$sort, \$lookup, \$unwind, \$limit.

3.6 Indexing

- Define with `#[ODM\Index(keys: ['field' => 'asc'])]` on the Document, or directly in MongoDB.
- Compound indexes for multi-field queries; be ready to explain **index selectivity** and that indexes speed reads but slow writes and cost storage.
- `explain()` to check whether a query is using an index (COLLSCAN = bad, means full scan).

3.7 SQL vs MongoDB — "when would you pick one over the other?"

- **Relational/SQL**: strong consistency, complex multi-table transactions, well-defined schema, strong reporting/joins needs.
- **MongoDB**: flexible/evolving schema, document-shaped data (nested JSON-like structures), high write throughput, horizontal scaling (sharding) needs.
- Many real systems use **both** (polyglot persistence) — SQL for transactional core, Mongo for logs/catalogs/flexible data. Saying this shows maturity, not "Mongo is always better."

3.8 N+1 problem (asked regardless of DB type)

Symptom: looping over a collection and querying inside the loop (once per item) instead of one batched query.

```
// BAD - N+1
foreach ($orders as $order) {
    $customer = $customerRepo->find($order->getCustomerId()); // 1 query per order!
}

// GOOD - batch fetch, or reference/embed pre-fetching
$customerIds = array_map(fn($o) => $o->getCustomerId(), $orders);
$customers = $customerRepo->findBy(['id' => $customerIds]); // 1 query
```

In Doctrine ORM the fix is usually JOIN/fetch: EAGER or explicit `IN()` batching; in MongoDB you batch reference IDs or use `$lookup` in an aggregation.

3.9 Migrations — deep dive (SQL side + the MongoDB reality)

Doctrine Migrations (doctrine/doctrine-migrations-bundle) — the SQL/ORM side:

```
php bin/console make:migration           # diffs current DB schema vs. your
entity mapping metadata, generates a class
php bin/console doctrine:migrations:migrate # applies all pending
migrations, in order
php bin/console doctrine:migrations:migrate prev # roll back the most
recent migration
php bin/console doctrine:migrations:status # what's applied, what's
pending
```

Each migration is a timestamped PHP class:

```
final class Version20260115120000 extends AbstractMigration
{
    public function up(Schema $schema): void
    {
```

```

        $this->addSql('ALTER TABLE orders ADD shipped_at DATETIME DEFAULT
NULL');
    }

    public function down(Schema $schema): void
    {
        $this->addSql('ALTER TABLE orders DROP shipped_at');
    }
}

```

Symfony tracks which migrations have run in a dedicated table, so migrate is idempotent across environments/deployments.

Safe/zero-downtime migration practice — the "expand/contract" pattern (a strong senior-level answer regardless of DB):

1. **Expand:** add the new column/field, keep the old one, deploy code that writes to *both*.
2. **Backfill:** migrate/copy existing data into the new shape in batches (never one giant blocking UPDATE on a huge table).
3. **Switch reads:** deploy code that reads from the new column/field only.
4. **Contract:** once you're confident nothing depends on the old one, drop it in a later, separate deploy. Never combine a destructive schema change with the code change that stops needing it in the same release — that's what makes rollbacks dangerous.

MongoDB's reality — there's no "ALTER TABLE," so "migrations" mean something different:

- MongoDB is schemaless, so adding/removing a field doesn't need a DDL statement — but you still face **data migrations** whenever you change a document's *shape* (renaming a field, flattening an embedded structure, changing a field's type).
- There's no single dominant "doctrine:mongodb:migrations" tool as mature as the SQL migrations bundle. Common real-world strategies:
 1. **One-off migration console commands** (make:command app:migrate-order-shape) that iterate the collection in **batches** via DocumentManager/the raw Mongo driver and bulk-update documents to the new shape — never load a huge collection into memory at once (use cursors/toIterable()-style batching, same idea as Section 1.5's generators).
 2. **schemaVersion field on each document** — the app knows which "version" a document is in and can lazily upgrade it on read, or a background job sweeps and upgrades stragglers.
 3. **Dual-read/dual-write in application code** temporarily (read the new field, fall back to the old one if absent) while a background job backfills — the expand/contract pattern applied without any DDL involved.
- **Index changes** *do* have a migration-like tool: #[ODM\Index] attributes + php bin/console doctrine:mongodb:schema:create (or :update) applies index definitions — but building a large index on a big collection can still be slow/blocking, so plan it like any other production change (plan and monitor index creation carefully, test it against representative data

volume, and use the behavior/options supported by your actual MongoDB server version rather than relying on old background index-build advice).

- **Testing migrations:** run against a fresh seeded DB in CI, and ideally against a realistic data-volume copy before touching production — migration scripts that work on 100 rows/documents can behave very differently at 10 million.

3.10 Native MongoDB Depth — beyond the ODM (this is what a strict Mongo interviewer actually probes)

Doctrine ODM syntax gets you through day-to-day CRUD, but the JD lists MongoDB as a **standalone must-have**, not "familiarity with an ORM that happens to target Mongo." Be ready to talk about MongoDB itself, independent of Doctrine:

Replica sets & consistency knobs

- A replica set is one primary + N secondaries; the primary is elected automatically on failure (no manual failover).
- **Write concern** (`w: 1`, `w: "majority"`, `j: true` for journal) — trades write latency against durability guarantees. `w: "majority"` means the write is acknowledged by most of the replica set before returning — safer, slower.
- **Read concern** (`local`, `available`, `majority`, `linearizable`, `snapshot`) and **read preference** (`primary`, `primaryPreferred`, `secondary`, `secondaryPreferred`, `nearest`) — reading from secondaries trades a small staleness risk for reduced load on the primary.

Change Streams

- A native pub/sub feed of insert/update/delete/replace events on a collection (or the whole DB), consumed via a change-stream cursor.
- Real use case: instead of polling for new orders, open a change stream and dispatch a Symfony Messenger message the moment a document changes — genuinely relevant for a real-time-ish feature without bolting on a separate CDC tool.

Sharding

- Splits a collection horizontally across multiple replica sets ("shards") once it outgrows one machine's capacity.
- **Shard key selection is the whole game:** needs high cardinality and an even write distribution matching your query patterns, or you get "hot shards." A bad shard key (e.g. a monotonically increasing `createdAt`) concentrates all new writes on one shard.
- Senior-level nuance to volunteer: **premature sharding is a common anti-pattern** — a well-indexed single replica set handles far more load than people assume; shard only once you've actually measured you need it.

GridFS

- MongoDB documents cap out at 16MB. GridFS chunks larger files (video, PDFs, exports) across a `fs.files/fs.chunks` collection pair. Know it exists as the native answer to "how

do you store a 200MB file in Mongo" — most teams still prefer S3/object storage for this, which is itself a good thing to say (shows judgment, not just "GridFS exists so I'd use it").

Schema design patterns (beyond embed-vs-reference from 3.3 — naming these specifically signals real Mongo depth): | **Pattern** | What it solves | |---|---| | **Polymorphic** | One collection holds documents of different "shapes," discriminated by a type field (e.g. mixed payment methods) | | **Bucket** | Group time-series/event data into fixed-size time buckets instead of one document per event — fewer, larger documents, much cheaper to scan (e.g. IoT sensor readings, analytics events) | | **Computed** | Precompute an expensive aggregation (e.g. a running total) into a field, refreshed periodically, instead of recalculating on every read | | **Subset** | Embed only the most relevant slice of a large related array (e.g. the 10 most recent reviews) and reference the rest, keeping the main document small | | **Extended Reference** | Duplicate a few frequently-read fields from a referenced document (e.g. customerName copied onto the Order) to avoid a lookup for the common case, accepting eventual staleness |

Security specifics (ties directly to the JD's "secure coding" line — see also the NoSQL injection note in Section 6.6):

- SCRAM-SHA-256 authentication, role-based access control (RBAC) scoped per database/collection.
- Encryption at rest (WiredTiger storage engine) and TLS in transit.
- **Client-side field-level encryption** for genuinely sensitive fields (e.g. SSNs) — encrypted before it ever leaves the application, so even a DB admin can't read it in plaintext.

Performance beyond "add an index"

- The **working set** (actively-used data + indexes) should fit in RAM — once it doesn't, performance falls off a cliff as MongoDB starts hitting disk constantly.
- **Covered queries:** if a query's filter and projection are both satisfied entirely by an index, MongoDB never touches the actual documents — much faster than even an indexed lookup that still fetches the document.

3.11 Doctrine Lifecycle Events, Custom Types & Recent Innovations

Sections 3.1-3.10 cover the MongoDB ODM side in depth; this section covers Doctrine mechanics that apply on both the ORM and ODM side and haven't been named yet — lifecycle hooks, custom types, and what's actually changed recently in how Doctrine works under the hood.

Two ways to hook into an entity/document's lifecycle

```
#[ODM\Document]
#[ODM\HasLifecycleCallbacks]
class Order
{
    #[ODM\PrePersist]
    public function setCreatedAtOnInsert(): void
    {
        $this->createdAt ??= new \DateTimeImmutable();
    }
}
```

```
}
}
```

- **Lifecycle callback attributes** (`#[ODM\PrePersist]`, `#[ODM\PreUpdate]`, `#[ODM\PostLoad]`, and their ORM equivalents) live directly on the class — simplest option, but only reusable within that one class, and the class now knows about persistence timing, which is a small crack in the separation Section 5's Data Mapper discussion praises.
- **EventListener / EventSubscriber** classes registered externally handle the same events (`prePersist`, `preUpdate`, ...) but decoupled from the entity/document itself — reusable across multiple classes, and keeps the persisted class free of infrastructure concerns. This is the Observer pattern from Section 5 applied to persistence timing instead of HTTP events.
- Rule of thumb: a callback tied to one class's own invariant (“this document always has a `createdAt`”) fits a lifecycle callback fine; logic that needs to run identically across many unrelated classes, or that has real business consequences (sending a notification, updating a search index), belongs in a listener — not because callbacks can't do it, but because a `prePersist` hook doing more than trivial bookkeeping is a common code-review flag (Section 6.10).

Custom types — mapping a value object to a scalar storage representation

- A value object like `Money` (Section 5) shouldn't leak its internal representation into every document/entity that uses it; a custom Doctrine type teaches the ORM/ODM how to convert `Money` to and from the scalar the database actually stores (e.g. an integer of cents) in one place, so every property of that type gets the conversion for free instead of manually converting in every getter/setter.
- This is the same idea as a `ValueTransformer` in the Form component (Section 2.18) — both exist because the shape you want to work with in PHP and the shape a boundary (database, HTML form) actually stores don't match, and you want that translation written exactly once.

What's actually changed recently in Doctrine — the “innovations” worth knowing about

- **Attributes are now the primary mapping style**, not just an alternative to annotations — this guide has used `#[ODM\Document]`/`#[Assert\NotBlank]`-style attributes throughout for exactly this reason; annotation mapping (the old `@ORM\Column-in-a-docblock` style) is legacy in current Doctrine and Symfony versions, worth naming explicitly if asked “why attributes and not annotations.”
- **Lazy loading without generated proxy classes**: newer Doctrine ORM versions use PHP 8.1's native lazy objects instead of Doctrine's own code-generated proxy classes for lazy-loaded relations — the practical effect for a candidate to know is that a lazy-loaded object behaves like a normal instance of its real class (no more `instanceof Proxy` surprises or `Doctrine\ORM\Proxy` cache-warming step), even though the mechanism (the Proxy pattern, Section 5) is conceptually the same.
- **Read-only/immutable-friendly mapping**: typed, `readonly` properties (Section 1.3) are now mappable, aligning Doctrine's persistence model with the promoted-constructor-property style this guide's PHP examples already lean on — worth mentioning as a concrete example

of “staying current” without overclaiming production experience on the newest minor version (Section 2.15's honesty framing applies here too).

Interview-safe framing for all of the above: name the direction (“attributes over annotations, native lazy objects over generated proxies”) and why it matters conceptually, rather than citing exact version numbers you haven't personally verified against production — the same discipline this guide applies to Symfony's own version history (Section 2.15).

Q1. What's the difference between a Doctrine lifecycle callback and an EventListener/EventSubscriber for the same event?

A: A lifecycle callback is a method on the entity/document itself, tagged with an attribute like `#[ODM\PrePersist]` — simple, but tied to that one class and coupling it slightly to persistence timing. An EventListener/EventSubscriber is registered externally and can react to the same event across many classes, keeping the persisted class itself free of infrastructure concerns — the Observer pattern (Section 5) applied to Doctrine's own lifecycle instead of Symfony's HTTP events.

Q2. Give an example of when you'd write a custom Doctrine type.

A: Whenever a value object needs to persist as a scalar and you don't want every entity/document using it to hand-roll the conversion — a Money value object stored as an integer number of cents is the canonical case (Section 5). The custom type centralizes `convertToDatabaseValue()/convertToPHPValue()` once, instead of scattering `$money->toCents()` calls across every place Money gets persisted.

Q3. What changed in how Doctrine generates proxy/lazy objects in recent versions, and why does it matter?

A: Newer Doctrine versions use PHP's own native lazy-object support instead of generating a separate Proxy subclass on disk for lazy-loaded relations — practically, a lazy-loaded object now behaves like a genuine instance of its real class rather than a generated stand-in, removing a class of subtle bugs around `instanceof` checks and proxy cache warming, while the underlying idea (defer loading until first access) is still the same Proxy pattern (Section 5).

Q4. Are Doctrine annotations still supported, and what's the current recommended mapping style?

A: Annotation mapping (`@ORM\Column` in a docblock) still works in current versions for backward compatibility, but attributes (`#[ORM\Column]` / `#[ODM\Field]`) are the current recommended and primary style — this guide's own code examples use attributes throughout for exactly that reason. Reaching for annotation syntax unprompted in an interview reads as slightly dated Symfony knowledge.

Q5. What's the risk of putting business logic inside a `prePersist/preUpdate` callback?

A: The callback runs implicitly whenever Doctrine flushes, which makes the business logic easy to miss when reading the code that actually triggers it — a bug or a change in behavior there is invisible from the call site, unlike an explicit service call. It also tends to accumulate unrelated concerns on the entity/document over time; anything beyond trivial, class-local bookkeeping (like a default timestamp) is usually clearer as an explicit service call or an externally registered listener.

Part IV — Testing & Design Patterns

Section 4 — Testing with PHPUnit (2+ yrs required)

4.1 Test pyramid

- **Unit tests:** one class in isolation, all dependencies mocked. Fast, most numerous.
- **Functional/Integration tests:** real Symfony kernel booted, real (test) database, HTTP requests simulated. Fewer, slower, higher confidence.
- **AAA pattern:** Arrange → Act → Assert. Keep each test asserting one behavior.

4.2 Basic unit test — and a currency note on syntax

Modern PHPUnit answer: write new tests with native PHP attributes such as `#[DataProvider]`, `#[Test]`, and `#[Group]`. Current PHPUnit documentation is attribute-first. Older projects may still contain docblock metadata, so don't turn this into a version-trivia argument in the interview — demonstrate the modern syntax and explain that you can maintain legacy suites too:

```
use PHPUnit\Framework\TestCase;
use PHPUnit\Framework\Attributes\DataProvider;
use PHPUnit\Framework\Attributes\Test;

final class DiscountCalculatorTest extends TestCase
{
    #[Test]
    public function appliesTenPercentForVipCustomers(): void
    {
        // Arrange
        $calculator = new DiscountCalculator();

        // Act
        $result = $calculator->calculate(price: 100, isVip: true);

        // Assert
        $this->assertSame(90.0, $result);
    }

    #[DataProvider('priceProvider')]
    public function testNonVipGetsNoDiscount(float $price): void
    {
        $calculator = new DiscountCalculator();
        $this->assertSame($price, $calculator->calculate($price, false));
    }

    public static function priceProvider(): array
    {
        return [[50.0], [199.99], [0.0]];
    }
}
```

(Older PHPUnit versions, or projects still on the docblock style, will still work with `/** @dataProvider */` — just know both exist and lead with attributes as current best practice.)

4.3 Mocking dependencies

```
public function testSendsConfirmationEmailOnOrderCreation(): void
{
    $mailer = $this->createMock(MailerInterface::class);
    $mailer->expects($this->once())
        ->method('send')
        ->with($this->assertInstanceOf(Email::class));

    $service = new OrderService($mailer, /* ... other deps ... */);
    $service->createOrder(/* ... */);
}
```

Mock vs Stub vs Spy — be precise:

- **Stub**: returns canned data, no behavior verification (`->method('find')->willReturn($fakeUser)`).
- **Mock**: verifies *interaction* happened (`->expects($this->once())->method('send')`).
- **Spy**: records calls so you can assert on them *after* the fact.

4.4 Symfony functional tests

```
use Symfony\Bundle\FrameworkBundle\Test\WebTestCase;

final class OrderControllerTest extends WebTestCase
{
    public function testGetOrderReturns404WhenMissing(): void
    {
        $client = static::createClient();
        $client->request('GET', '/api/orders/999999');

        $this->assertResponseStatusCodeSame(404);
    }

    public function testGetOrderReturnsJson(): void
    {
        $client = static::createClient();
        $client->request('GET', '/api/orders/1');

        $this->assertResponseIsSuccessful();
        $this->assertJson($client->getResponse()->getContent());
    }
}
```

`KernelTestCase` boots the Symfony kernel without an HTTP layer (good for testing services against a real container); `WebTestCase` extends it and adds the HTTP test client.

4.5 Fixtures

`DoctrineFixturesBundle` (works for ORM; for MongoDB you write your own fixture loader against `DocumentManager`) — load consistent, known test data before each test run so tests aren't order-dependent or flaky.

4.6 Coverage & CI

- Xdebug or PCOV generate coverage reports (`phpunit --coverage-html`).

- Coverage % is a signal, not a target — 100% coverage with weak assertions is worse than 80% with meaningful ones. **Good senior-level talking point.**
- In CI: run PHPUnit on every PR, fail the pipeline below a coverage threshold or on any failing test, run static analysis (PHPStan/Psalm) alongside.

4.7 Testing best practices to mention

- Test behavior, not implementation details (don't assert on private state).
- Keep tests independent/isolated — no shared mutable state between tests.
- Name tests descriptively (testX_whenY_thenZ style or clear sentence names).
- Prefer constructor injection everywhere specifically *because* it makes mocking trivial — ties back to SOLID/DI.

4.8 Testing MongoDB-backed code — the gap most PHPUnit prep skips

PHPUnit and MongoDB are both must-haves in this JD, but people rarely rehearse the *intersection*. Two levels, matching the test pyramid from 4.1:

Unit level — mock DocumentManager entirely, don't touch a real database:

```
use PHPUnit\Framework\TestCase;
use PHPUnit\Framework\Attributes\Test;
use Doctrine\ODM\MongoDB\DocumentManager;
use Doctrine\ODM\MongoDB\Repository\DocumentRepository;

final class OrderServiceTest extends TestCase
{
    #[Test]
    public function persistsAndFlushesANewOrder(): void
    {
        $dm = $this->createMock(DocumentManager::class);
        $dm->expects($this->once())->method('persist')->with($this->
isInstanceOf(Order::class));
        $dm->expects($this->once())->method('flush');

        $service = new OrderService($dm);
        $service->create(new Order(/* ... */));
    }

    #[Test]
    public function findRecentDelegatesToTheRepository(): void
    {
        $repository = $this->createMock(DocumentRepository::class);
        $dm = $this->createMock(DocumentManager::class);
        $dm->method('getRepository')->with(Order::class)-
>willReturn($repository);
        // further stub the query builder chain, or better: wrap the query in a
        // narrow custom repository method and mock THAT instead of the whole
        // builder chain.

        $service = new OrderService($dm);
        // ... assert on the service's own method, not on Doctrine internals
    }
}
```

Senior-level point to make explicitly: mocking a fluent query-builder chain

(createQueryBuilder()->field()->gte()->sort()->getQuery()->execute()) gets ugly fast — the cleaner fix is to push that chain behind your **own** narrow repository method (findRecentOrders(): array) and mock that one method, not Doctrine's builder. This is the Repository pattern earning its keep for testability (Section 5).

Integration level — hit a real (test) MongoDB instance:

```
use Symfony\Bundle\FrameworkBundle\Test\KernelTestCase;

final class OrderRepositoryIntegrationTest extends KernelTestCase
{
    public function testFindRecentReturnsOnlyLastSevenDays(): void
    {
        self::bootKernel();
        $dm = self::getContainer()->get(DocumentManager::class);

        // seed known documents, then assert against the real repository
        $repository = $dm->getRepository(Order::class);
        $results = $repository->findRecent();

        $this->assertCount(2, $results);
    }
}
```

Run this against a dedicated **test database** (a separate MONGODB_URL in .env.test), and clean/reseed it between tests — there's no in-memory MongoDB equivalent of SQLite, so integration tests here mean a real (disposable, often Dockerized) Mongo instance.

4.9 Test-Driven Development (TDD) — JD must-have

The JD names TDD twice — as a required skill (“Hands-on experience with Test-Driven Development (TDD)”) and as an expected daily practice (“Follow and promote Test-Driven Development (TDD) practices”). That is a different claim from “I write tests.” TDD means the test exists *before* the production code, and it has its own vocabulary and failure modes — this section treats it as its own topic, not a footnote to this Part's PHPUnit material.

The cycle: Red → Green → Refactor

Requirement		
Write a failing test	-> RED	(test exists, code doesn't yet)
Write the minimum code to pass	-> GREEN	(ugly is fine, correctness isn't)
Refactor with tests as a net	-> REFACTOR	(clean up, tests stay green)
Repeat for the next small behavior		

Three rules worth stating explicitly in an interview:

- Write only enough test to fail — one behavior at a time, not a whole test class up front.

- Write only enough code to make it pass — resist solving problems the test doesn't ask for yet (“YAGNI inside the cycle”).
- Refactor only when the suite is green — refactoring on a red bar means you can't tell if you broke behavior or just haven't finished yet.

Classical (Chicago/Detroit) vs London (mockist) TDD

- **Classical/Chicago:** build up from real objects, verify final state; mocks used sparingly, mainly for slow/external boundaries (HTTP, filesystem, real Mongo). Reads close to Section 4.1's test-pyramid bottom layer.
- **London/mockist:** mock every collaborator, verify interactions (was this method called, with what arguments) rather than final state. Drives interface design outside-in, but risks overfitting tests to implementation details if you mock too aggressively — ties to Section 4.7's “test behavior, not implementation” rule.
- Neither is “more correct.” Most Symfony teams land closer to classical for domain/business logic and mockist for I/O-heavy boundaries (repositories, HTTP clients, mail).

A short TDD walkthrough — pricing a discount

```
// Iteration 1 -- RED
public function testNoDiscountBelowThreshold(): void
{
    $calculator = new OrderDiscountCalculator();
    $this->assertSame(0, $calculator->discountFor(5000));
}

// Iteration 1 -- GREEN (the simplest thing that could work)
final class OrderDiscountCalculator
{
    public function discountFor(int $amountInCents): int
    {
        return 0;
    }
}

// Iteration 2 -- RED (triangulate: a second, different example)
public function testTenPercentDiscountAtOrAboveThreshold(): void
{
    $calculator = new OrderDiscountCalculator();
    $this->assertSame(1000, $calculator->discountFor(10000));
}

// Iteration 2 -- GREEN
public function discountFor(int $amountInCents): int
{
    return $amountInCents >= 10000 ? (int) round($amountInCents * 0.10) : 0;
}

// Iteration 3 -- REFACTOR (both tests stay green throughout)
final class OrderDiscountCalculator
{
    private const THRESHOLD_CENTS = 10000;
    private const RATE = 0.10;

    public function discountFor(int $amountInCents): int
```

```

    {
        return $amountInCents >= self::THRESHOLD_CENTS
            ? (int) round($amountInCents * self::RATE)
            : 0;
    }
}

```

Interview sentence to have ready: “I write the smallest failing test for the next behavior, make it pass with the simplest code, then refactor while the suite is green — the tests end up documenting behavior, not implementation, which is what makes them survive refactors.”

Q1. What makes a good TDD test?

A: It tests one behavior, fails for a single clear reason before the code exists, and asserts on the outcome an interface promises rather than on internal steps — so it survives refactors instead of breaking every time you change how the answer is computed. A good TDD test is also fast enough to run on every save; a suite you're afraid to run constantly stops driving design.

Q2. What's the difference between TDD and just writing unit tests?

A: Unit testing describes coverage after the fact — code exists, tests confirm it works. TDD describes an order of operations — the test is written first and used to drive what code gets written, so the design responds to “how would this be easy to test” rather than being retrofitted once the shape is already fixed. The same PHPUnit test can be either, depending on when it was written.

Q3. TDD vs BDD — what's actually different?

A: TDD operates at the unit/class level and is written by and for engineers (assertSame, mocks). BDD (Behat/Gherkin — Section 4.10) operates at the feature/scenario level, written in Given/When/Then language a product owner or QA can read and agree with before implementation starts. They're complementary, not competing: BDD scenarios describe what the system should do; TDD drives how the internals get built to satisfy that.

Q4. When does TDD become counterproductive?

A: When the design is still genuinely unknown — for a rough spike or exploring an unfamiliar API, writing the test first just means rewriting it repeatedly as the shape changes; better to spike untested, throw the spike away, then TDD the real implementation. It's also a poor fit for pure UI/visual work and one-off scripts with no lasting value. Over-mocking to satisfy strict TDD can also lock in an implementation detail as if it were a requirement — the smell covered in Section 4.7.

Q5. Do you TDD controllers?

A: Rarely at the unit level — a controller's job is mostly wiring (call a service, return a Response), which is better covered by a Symfony functional test (WebTestCase, Section 4.4) or left un-mocked and tested through the service layer underneath it. TDD earns its keep on the logic *inside* the services the controller calls, not on the thin controller itself.

Q6. How do you TDD a Symfony service?

A: Same cycle as any class: write a test that constructs the service with mocked collaborators (repository, mailer, clock) via the constructor, assert the behavior you want, watch it fail because the class or method doesn't exist yet, write the minimum implementation, then refactor. Because Symfony favors constructor injection everywhere (Section 4.7), most services are naturally TDD-friendly — you're never fighting a framework base class to substitute a dependency.

Q7. How does dependency injection actually help TDD?

A: DI means a class asks for its collaborators through its constructor instead of instantiating them internally (`new SomeClient()` buried inside a method), so a test can hand it a mock/stub instead of the real thing. Without DI you either can't isolate the class under test at all, or you resort to fragile tricks (static calls, service-locator lookups) that make “write the test first” painful enough that people give up on it.

Q8. London/mockist vs classical TDD — restate the distinction and when you'd use each.

A: Classical TDD builds real objects and asserts on final state, reserving mocks for slow or external boundaries; it tends to produce tests that survive internal refactors. London/mockist TDD mocks every collaborator and asserts on the interactions between objects, useful for driving interface design from the outside in on a new feature, but risks tests that break whenever you reorganize internals without changing behavior. In a Symfony/Doctrine codebase, classical tends to fit domain/business logic, and mockist tends to fit the outer layer that talks to Doctrine, HTTP clients, or the mailer.

Q9. How do you TDD code that interacts with Doctrine?

A: You don't TDD against the real database at the unit level — mock `DocumentManager/EntityManagerInterface` (or better, mock your own narrow repository method) and drive the service's behavior against that, exactly as in Section 4.8's MongoDB testing pattern. The repository method itself gets covered by a slower integration test against a real (disposable) database, run less frequently — the test pyramid (Section 4.1) applied to TDD specifically.

Q10. How do you handle legacy code that wasn't built with TDD?

A: You don't retrofit tests onto the whole codebase before touching it — that's a rewrite in disguise. The practical approach: find a “seam” where you can insert a test (usually by extracting the piece you're about to change behind an interface so it can be mocked), write a characterization test that pins down the *current* behavior (even if it's wrong), then TDD the actual change against that safety net. One changed corner at a time — this also ties back to the ownership lifecycle in Section 6.18.

Q11. Is there a real difference between TDD and “test-after” development in the tests you end up with, or is it just discipline?

A: It's not just discipline — writing the test first forces you to describe the API from the caller's point of view before you know how the implementation will work, which tends to produce simpler, more focused interfaces. Writing tests after the code tends to describe the implementation that already exists, including its accidental complexity, which is why test-after suites more often end up brittle and tightly coupled to internals.

Q12. How do you TDD a bug fix?

A: Reproduce the bug as a failing test first — non-negotiable, because without it you can never prove the bug is actually fixed rather than coincidentally not happening today. Once that test fails for the right reason, write the smallest fix that turns it green, then refactor. The regression test then stays in the suite permanently as protection against the same bug coming back.

Q13. What is “triangulation” in TDD?

A: Writing a second, different example that the current (often over-simplified) implementation can't satisfy, to force generalizing from a hard-coded special case toward the real rule. In the discount-calculator example above, iteration 2's test — a different amount, a different expected discount — is triangulation: it makes return 0; insufficient and forces the actual threshold logic to appear.

Q14. Can you TDD complex/algorithmic logic (like pricing rules) the same way you'd TDD a CRUD endpoint?

A: The cycle is identical, but the granularity of “smallest test” changes — for algorithmic logic you triangulate through several small cases (boundary values, edge cases, rounding behavior) before the general rule falls out, the way the discount example moves from “no discount” to “percentage above a threshold.” A CRUD endpoint has much less inherent logic to discover, so TDD there mostly confirms wiring (validation runs, the right repository method gets called) rather than driving a design.

Q15. Under deadline pressure, how do you keep the “refactor” step from quietly disappearing?

A: Treat refactor as part of the definition of done for the ticket, not a follow-up task — a feature that passes tests but was never cleaned up isn't finished, it's deferred technical debt with no ticket tracking it. Practically: don't move to the next red test until the current green bar has been refactored, and call out explicitly in code review when a PR is “green but not yet refactored” so it isn't merged as if it were.

4.10 Behat & BDD — brief overview (JD names it alongside PHPUnit)

The JD lists testing tools as “PHPUnit, Behat, or similar” — Behat is Symfony's most common BDD framework, and even light familiarity with Gherkin syntax is worth having ready, since it's the tool most likely to come up if the team also does QA-facing acceptance testing.

Gherkin — the language behind Behat

- Feature files are plain-text .feature files written in Given/When/Then steps, designed to be readable by product/QA without PHP knowledge — this is the JD's “collaborate closely with product... QA” bullet made literal.
- Each step maps to a PHP method in a Context class via annotation/attribute; Behat matches the sentence to the method using a placeholder or regex.
- Scenarios describe outcomes, not implementation — “When I submit an order over \$100”, not “When the OrderDiscountCalculator returns 1000”.

```
# features/order_discount.feature
Feature: Order discounts
  In order to reward larger purchases
  As a customer
  I want a discount applied automatically above a spending threshold

  Scenario: Discount applies above the threshold
    Given I have a cart worth "120.00"
    When I proceed to checkout
    Then my order total should be "108.00"

  Scenario: No discount below the threshold
    Given I have a cart worth "50.00"
    When I proceed to checkout
    Then my order total should be "50.00"

final class OrderContext implements Context
{
    private ?Order $order = null;

    #[Given('I have a cart worth :amount')]
    public function iHaveACartWorth(string $amount): void
    {
        $this->order = OrderFactory::withTotal((float) $amount);
    }

    #[When('I proceed to checkout')]
    public function iProceedToCheckout(): void
    {
        $this->order->applyDiscount(new OrderDiscountCalculator());
    }

    #[Then('my order total should be :amount')]
    public function myOrderTotalShouldBe(string $amount): void
    {
        Assert::same($this->order->total(), (float) $amount);
    }
}
```

How it fits alongside PHPUnit

- Behat scenarios sit above the test pyramid's PHPUnit layers (Section 4.1) — fewer of them, each exercising a full user-facing flow, often through a booted Symfony kernel or a real HTTP layer.
- It's not a replacement for unit tests — a Behat scenario failing tells you a user-facing behavior broke, not which class caused it; PHPUnit still does the fine-grained diagnosis.

- Most teams introduce it selectively, for the handful of flows where a shared, readable spec with product/QA is worth the overhead of maintaining feature files — not for every code path.

Q1. What's the practical difference between a PHPUnit test and a Behat scenario?

A: A PHPUnit test is written by and for engineers, exercises a class or a small slice of the system, and is the primary safety net for TDD (Section 4.9). A Behat scenario is written in plain language, exercises a full user-facing flow, and is meant to be readable — and ideally reviewable — by product and QA before the feature is even built. They test at different altitudes and are complementary rather than substitutes.

Q2. What is Gherkin, and who is it meant to be readable by?

A: Gherkin is the structured plain-English syntax (Given/When/Then, Feature, Scenario) that Behat feature files are written in. It's deliberately readable by non-engineers — product owners and QA — so a scenario can double as a shared, agreed specification of behavior, not just a test only developers can parse.

Q3. When would you introduce Behat on a project that already has solid PHPUnit coverage?

A: When there's a real communication gap to close — e.g. product/QA keep discovering that “done” didn't mean what they expected, or a critical user journey (checkout, signup) needs a spec that survives implementation changes and stays readable to non-engineers. It's not worth adding purely for extra test coverage; PHPUnit functional tests already do that job for engineers.

Q4. How does BDD scenario language help the cross-functional collaboration this JD asks for?

A: It gives product, QA, and engineering a single artifact — the feature file — that all three can read, question, and agree on before a line of implementation code is written, instead of requirements living in a ticket description that gets reinterpreted at each handoff. Disagreements (“what should happen if the cart is exactly at the threshold?”) surface as a missing scenario, not as a bug found after release.

Section 5 — Design Patterns (OOP) — explicit must-have

Don't just recite definitions — for each pattern, name **where Symfony itself uses it**. That's what separates “memorized GoF book” from “understands the framework.”

Creational

- **Singleton/shared service idea:** one reused instance within a container lifetime. *Symfony tie-in:* services are **shared by default**, so repeated requests to the container for the same service return the same instance. Under normal PHP-FPM this lifetime is effectively bounded by the request; long-running workers make mutable shared-service state a more important concern.
- **Factory Method:** a method that creates objects, subclasses decide what to instantiate. *Symfony tie-in:* `Symfony\Component\HttpFoundation\Response::create()`-style helpers,

and service **factories** in `services.yaml` (`factory: [App\Factory\ClientFactory, create]`).

- **Builder**: step-by-step construction of a complex object. *Symfony tie-in*: `FormBuilderInterface`, `QueryBuilder` (Doctrine).
- **Abstract Factory / Prototype**: less common in day-to-day Symfony but know the definitions (family of related objects / cloning an existing instance as a template).

Structural

- **Adapter**: wraps an incompatible interface to match what the client expects. *Symfony tie-in*: PSR-adapters (e.g. bridging a 3rd-party HTTP client to `HttpClientInterface`).
- **Decorator**: wraps an object to add behavior without changing its class. *Symfony tie-in*: **service decoration** — `#[AsDecorator(decorates: LoggerInterface::class)]` — this is a real interview favorite, be ready to write it (see demo code).
- **Facade**: simplified interface over a complex subsystem. *Symfony tie-in*: `AbstractController` helper methods (`$this->json()`, `$this->render()`) are facades over lower-level components.
- **Proxy**: stand-in that controls access to the real object. *Symfony tie-in*: **Doctrine lazy-loading proxies** — a generated subclass that only hits the DB when a property is first accessed.
- **Composite**: tree structures treated uniformly. Less central to Symfony core but common in form/menu-building bundles.

Behavioral

- **Strategy**: interchangeable algorithms behind one interface. *Symfony tie-in*: **Security Voters**, multiple `PaymentGatewayInterface` implementations chosen at runtime.
- **Observer**: subject notifies subscribed observers of state change. *Symfony tie-in*: **EventDispatcher** — this is the single most important pattern-to-Symfony mapping to know cold.
- **Command**: encapsulate a request as an object. *Symfony tie-in*: **Console commands**, and **Messenger messages** (a `Message` object literally *is* the Command pattern — an object representing "do this thing later/elsewhere").
- **Chain of Responsibility**: pass a request along a chain until something handles it. *Symfony tie-in*: the **HttpKernel event chain** (Section 2.2) and Messenger **middleware stack**.
- **Template Method**: base class defines the skeleton, subclasses override steps. *Symfony tie-in*: Twig template inheritance, `AbstractController`/`AbstractType` base classes.
- **Mediator**: central object coordinates communication between others rather than direct references. *Symfony tie-in*: the Messenger **Bus** mediates between message dispatchers and handlers so they never know about each other.

Enterprise patterns (not GoF, but the ones that actually run your day-to-day in this stack)

These come from Martin Fowler's *Patterns of Enterprise Application Architecture*, not the GoF book — and they matter *more* than most GoF patterns for a Doctrine/MongoDB-heavy role. Missing these by name is a bigger tell than missing Composite or Bridge.

- **Repository:** abstracts data access behind a collection-like interface (`find()`, `findBy()`, `save()`) so business logic never talks to the database directly. *Symfony tie-in:* every `ServiceEntityRepository/Doctrine ODM repository` IS this pattern. **Senior nuance:** for real testability and to avoid framework lock-in, define your *own* repository interface in your domain/service layer (e.g. `OrderRepositoryInterface`) with a Doctrine-backed implementation behind it, rather than type-hinting `DocumentManager` directly into every service — this is what makes the mocking in Section 4.8 clean instead of painful.
- **Unit of Work:** tracks all objects changed during a request and writes them out in one coordinated batch. *Symfony tie-in:* this is *exactly* what `$entityManager->flush()` / `$documentManager->flush()` does — Doctrine doesn't hit the DB on every `persist()`, it batches everything until `flush()`.
- **Value Object:** an immutable object defined by its *value*, not an identity (two `Money` objects with the same amount+currency are interchangeable) — contrast with an `Entity`, which has identity even if its values change. *Symfony tie-in:* the readonly `Money` class from Section 1.1 is a textbook Value Object; PHP 8.1's readonly properties made these dramatically nicer to write.
- **Data Mapper vs. Active Record — the sharpest technical answer to "how is Symfony different from Laravel":** Doctrine is a **Data Mapper** — your entity/document classes know nothing about persistence; a separate layer (the `EntityManager/DocumentManager`) maps them to storage. Laravel's Eloquent is **Active Record** — the model class *itself* has `save()/find()/delete()` baked in, mixing domain data with persistence logic. This is a real, well-known architectural distinction (not a knock on Laravel) and it's a far more precise answer than "Symfony feels more enterprise-y."

A pattern-recognition drill (practice saying this out loud)

"In Symfony, when a controller needs to check if the current user can edit a Post, it calls `denyAccessUnlessGranted('EDIT', $post)`. Under the hood this is the Strategy pattern — the `AccessDecisionManager` asks every registered Voter (each a different strategy) whether it grants/denies/abstains, and combines results per a configurable strategy (affirmative, unanimous, consensus) — itself another Strategy." This single sentence demonstrates DI, Security, and two design patterns at once — exactly the kind of synthesis a senior interview is fishing for.

Part V — Backend & System Design

Section 6 — General Backend Concepts (breadth signals seniority)

6.1 REST & HTTP

- Verbs: GET (safe, idempotent), POST (create, not idempotent), PUT (replace, idempotent), PATCH (partial update), DELETE (idempotent).
- Status codes: 200/201/204 success family, 400 bad request, 401 unauthenticated, 403 unauthorized, 404 not found, 409 conflict, 422 validation error, 429 rate limited, 500 server error.
- Pagination (?page=2&limit=20 or cursor-based for large/real-time data), filtering, sorting conventions.
- Versioning strategies: URL (/v1/), header-based, or content negotiation.

6.2 Caching layers

1. **HTTP caching:** Cache-Control, ETag, reverse proxy (Symfony's built-in HTTP cache or Varnish).
2. **Application cache:** Redis/Memcached via Symfony's Cache component, PSR-6/16.
3. **Database query cache** / result caching in Doctrine. Know **cache invalidation strategies:** TTL expiry, explicit tag-based invalidation, cache-aside pattern.

6.3 Message queues & async processing

- Why: decouple slow/unreliable work (emails, PDF generation, 3rd-party API calls) from the request/response cycle.
- Symfony Messenger + a broker (RabbitMQ/Redis/SQS): producer publishes a message, consumer(s) process asynchronously.
- Concepts to know: **retry policies with backoff**, **dead-letter queues** for messages that repeatedly fail, **idempotent consumers** (processing the same message twice shouldn't double-charge a customer, etc.) — very relevant given the JD's on-call rotation.

6.4 CI/CD

Typical pipeline: lint (PHP-CS-Fixer/PHPStan) → unit tests → integration tests → build (Docker image) → deploy (staging → prod, often with manual gate). Know the difference between **Continuous Delivery** (always deployable, manual trigger) and **Continuous Deployment** (auto-deploys every passing build).

6.5 Docker & Kubernetes (nice-to-have but often asked)

- **Docker:** image (immutable blueprint) vs container (running instance); Dockerfile builds an image; docker-compose orchestrates multi-container local dev (app + MongoDB + Redis).

- **Kubernetes:** Pod (smallest deployable unit), Deployment (manages replica pods + rollout strategy), Service (stable network endpoint / load balancing), ConfigMap/Secret (config injection) — you don't need deep k8s ops experience for this role, just fluency in the vocabulary.

6.6 Security practices (JD explicitly mentions "secure coding, free of known vulnerabilities")

- **OWASP Top 10** awareness: injection, broken auth, sensitive data exposure, XXE, broken access control, security misconfiguration, XSS, insecure deserialization, vulnerable components, insufficient logging.
- Symfony-specific defenses: Twig auto-escaping (XSS), Doctrine parameterized queries (SQL/NoSQL injection), CSRF tokens on state-changing forms, PasswordHasher for storing credentials, composer audit / Symfony Security Advisories checker for dependency CVEs.
- Principle of least privilege for API tokens/service accounts.
- **NoSQL/operator injection — MongoDB-specific, and easy to miss.** Unlike SQL injection, this doesn't need special characters or escaping bypasses — it's about passing raw, unvalidated arrays straight into a query filter. If a login check builds a filter like `{'username' => $input['username'], 'password' => $input['password']}` and the request body's password field arrives as `{"$ne": null}` instead of a string, the query becomes "password is not null" — **matching any user and bypassing authentication entirely**. Mitigation: always validate/cast incoming input to scalar types before it touches a query (Symfony's Validator + typed DTOs naturally solve this — Section 2.6), never pass a raw `$request->toArray()`-style payload into a Doctrine ODM query builder or native filter array, and avoid `$where` (executes arbitrary JS server-side) for anything influenced by user input.

6.7 Logging & Observability (ties to "monitor systems, on-call")

- Monolog (Symfony's default logger) — channels, handlers, and **log levels** (debug → emergency).
- Structured/JSON logging so logs are queryable in ELK/Datadog/etc.
- Correlate logs with a request ID for tracing a single request across services.
- Alerting on error rate / latency SLOs, not just "server is down" — shows on-call maturity.

6.8 Database Fundamentals (theory questions that apply regardless of SQL or MongoDB)

- **ACID:** Atomicity (all-or-nothing), Consistency (valid state before/after), Isolation (concurrent transactions don't corrupt each other), Durability (committed data survives a crash). MongoDB supports multi-document ACID transactions since v4.0 (replica-set required) but single-document writes were always atomic even before that.
- **Normalization (1NF/2NF/3NF)** in relational design vs. **denormalization** — document stores like MongoDB intentionally lean denormalized (embed related data) to avoid joins, trading storage/duplication for read speed (Section 3.3).

- **Indexing theory:** most relational and MongoDB indexes are **B-tree variants** — shallow, high branching factor, so even huge collections need only a handful of disk reads per lookup. Composite indexes must match query patterns (leftmost-prefix rule); an index speeds reads but costs writes and storage — don't index everything.
- **Isolation levels** (Read Uncommitted → Read Committed → Repeatable Read → Serializable) — know they exist and that stricter isolation trades concurrency for consistency guarantees.
- **CAP theorem** (Consistency, Availability, Partition tolerance — pick two under a network partition): directly relevant to MongoDB replica sets/sharding trade-offs, e.g. choosing read preference (primary vs secondaryPreferred) is literally a CAP trade-off exposed as a config option.
- **Connection handling in PHP-FPM:** each request typically opens/closes its own DB connection (PHP's request lifecycle is short-lived, unlike a long-running Node/Java process) — so connection pooling is usually handled at the infrastructure layer (e.g. PgBouncer, or MongoDB driver's built-in connection pool) rather than in application code.

6.9 Data Structures & Complexity — the practical refresher

Senior backend interviews often slip in a couple of CS-fundamentals questions, usually framed practically rather than "implement quicksort on the whiteboard":

Structure	Big-O (typical)	PHP reality / where it shows up
Array access by index	$O(1)$	Native PHP arrays
Search by value (unindexed)	$O(n)$	<code>in_array()</code> , <code>array_search()</code> — avoid in hot loops on large arrays
Hash map insert/lookup	$O(1)$ average	PHP associative arrays ARE hash maps under the hood — this is why keyed lookups (<code>\$arr[\$id]</code>) are fast even on large arrays
Sorting	$O(n \log n)$ typical	PHP's <code>sort()/usort()</code> family (hybrid algorithms internally)
Binary search	$O(\log n)$	Only valid on sorted data — <code>array_search</code> does NOT do this automatically
Stack push/pop	$O(1)$	<code>SplStack</code> , or just <code>array_push/array_pop</code> on a plain array
Queue enqueue/dequeue	$O(1)$ amortized	<code>SplQueue</code> ; <code>array_shift</code> on a plain array is actually $O(n)$ — a real gotcha, since shifting re-indexes the whole array

Structure	Big-O (typical)	PHP reality / where it shows up
B-tree index lookup	$O(\log n)$	How both SQL and MongoDB indexes achieve fast lookups on huge datasets

Concrete "why does this matter" examples to have ready:

- "Why is `array_shift()` in a tight loop a performance smell?" → because removing from the front of a PHP array re-indexes every remaining element ($O(n)$ per call, $O(n^2)$ over a loop) — use `Sp1Queue` or reverse-iterate instead.
- "How does a DB index turn an $O(n)$ table scan into an $O(\log n)$ lookup?" → the B-tree structure means each lookup narrows the search space by branching, not scanning row by row — same reason `explain()` showing `COLLSCAN` instead of an index scan is a red flag (Section 3.6).
- "When would you use a hash map (associative array) over a plain indexed array?" → whenever you need $O(1)$ lookup by a non-sequential key — e.g. deduplicating a list, or building a lookup table keyed by ID before joining two datasets in PHP to avoid the $N+1$ problem (Section 3.8).

6.10 What a strong reviewer checks for in a Symfony/PHP PR

The JD explicitly lists "participate in code reviews and offer constructive feedback" as a responsibility — worth having an actual checklist ready, not just "I leave nice comments." What to look for, roughly in order of severity:

1. **Correctness & security first:** unvalidated input reaching a query (SQL or NoSQL injection, Section 6.6), missing authorization checks (should this action go through a Voter?), secrets or credentials in the diff.
2. **$N+1$ queries:** a loop with a `find()/query` call inside it (Section 3.8) — the single most common performance bug in a Doctrine/Mongo codebase.
3. **Type safety:** `declare(strict_types=1)`, real type hints instead of `mixed/no types`, `readonly` where the value shouldn't change after construction.
4. **SOLID smells:** a controller with business logic instead of delegating to a service (fat controller anti-pattern), a service directly new-ing up its dependencies instead of accepting them via constructor injection (breaks testability — Section 4.8's mocking pain is usually caused by exactly this).
5. **Test coverage matches the change:** new behavior without a new/updated test is a request-changes, not a nitpick.
6. **No debugging leftovers:** `dd()`, `var_dump()`, commented-out code, `dump()` calls left in.
7. **Consistency with existing patterns:** does this PR introduce a second way of doing something the codebase already does one way (e.g. a new ad-hoc HTTP call instead of using the existing `HttpClient`-based service)?

Keep feedback specific and suggest an alternative, not just "this is wrong" — the JD's mentoring emphasis means how you *phrase* a review comment matters as much as catching the issue.

6.11 System Design / Open-Ended Prompts (practice out loud — no single right answer)

Everything in Sections 7-8 is trivia-format. Senior interviews often add one open-ended design question — these can't be memorized, only practiced. Talk through each for 3-5 minutes, out loud, before the interview:

- **Design an idempotent order-creation API** on Symfony + MongoDB that safely handles a client retrying the same request after a timeout (hint: idempotency key stored on the Order document, checked before processing).
- **Design a notification system** that sends email today but needs SMS and push added later, using Messenger (hint: one NotificationRequested message, multiple handlers or a strategy per channel — ties Section 5's Strategy pattern directly to Messenger).
- **A MongoDB collection has outgrown a single replica set — walk through your decision process**, not just "shard it" (hint: confirm it's actually a capacity problem first, choose a shard key, discuss the migration risk — Section 3.10).
- **Design a safe, zero-downtime way to rename a heavily-queried field across 50 million documents** without an outage (hint: expand/contract from Section 3.9, applied to a schemaless store).
- **A specific endpoint is intermittently slow under load — walk through your triage**, live, out loud (this is Section 8, Q44's on-call triage question, but rehearsed as a spoken walkthrough rather than a memorized bullet list — the delivery is what's being tested here, not new content).

6.12 Concurrency, race conditions, atomicity & idempotency

A senior backend engineer must distinguish these concepts:

- **Atomic operation**: indivisible at the database level. MongoDB writes to a **single document are atomic**; a read-modify-write sequence across requests can still lose updates unless you use conditional updates/versioning.
- **Optimistic locking**: read a version, update only if the version is still the expected value; if not, reject/retry. Great when conflicts are uncommon.
- **Pessimistic locking**: reserve/lock a resource before mutation. Stronger coordination, but more contention and operational complexity. Doctrine MongoDB ODM has locking mechanisms, but know that ODM pessimistic locking is an application/Doctrine mechanism rather than native MongoDB row locking.
- **Transaction**: groups operations into an all-or-nothing unit. Multi-document MongoDB transactions require a replica set or sharded cluster and cost more than a single-document design, so schema design should often keep invariants inside one aggregate/document where practical.

- **Idempotency:** executing the same logical request more than once has the same externally visible effect as executing it once. This is essential for payments, order creation and message consumers.

Idempotent API pattern: client sends Idempotency-Key; persist a unique record/order keyed by that value; retries return the original result instead of creating a second side effect. Enforce uniqueness at the database level — a check-then-insert in PHP alone has a race condition.

Interview sentence: "Retries are a distributed-systems fact, so I design writes and queue handlers for duplicate delivery rather than assuming exactly-once execution."

6.13 API design beyond CRUD

- Correct HTTP semantics: GET safe/idempotent, PUT idempotent replacement/update semantics, PATCH partial update, POST usually non-idempotent unless you add an idempotency contract.
- **Pagination:** offset/limit is simple but can become slow/inconsistent on rapidly changing large datasets; cursor/keyset pagination is better for large ordered feeds.
- **Filtering/sorting:** whitelist allowed fields/operators. Never pipe arbitrary JSON operators from a request into a Mongo query.
- **Versioning:** prefer backward-compatible additive changes; version only when the contract truly breaks. Support old clients during migrations.
- **Errors:** stable machine-readable error codes + human-readable message + correlation/request ID. Do not leak stack traces or secrets in production.
- **Rate limiting:** rate by the correct identity (user/API key/IP depending threat model), and decide whether limits are local or shared across instances.

6.14 Resilience: timeouts, retries, backoff, circuit breakers

Whenever your Symfony service calls another service:

1. Set **explicit connect and total/request timeouts**. "No timeout" means a dependency can consume your worker/request indefinitely.
2. Retry **transient** failures only. Use exponential backoff + jitter to avoid a retry storm.
3. Retry only when the operation is idempotent, or supply a stable idempotency key.
4. Use a **circuit breaker** conceptually when a failing dependency should be short-circuited instead of hammered continuously.
5. Add fallback/degraded behavior only when it preserves correctness; don't return stale/partial data silently for critical financial/business operations.
6. Record latency, error rate, timeout count and dependency name in metrics/logs.

Symfony HttpClient already exposes strong timeout/concurrency primitives; the senior skill is choosing the policy, not merely knowing the API name.

6.15 Messaging semantics, Outbox pattern & long-running workers

At-least-once delivery is the practical mental model for queues: a handler may complete and the worker may crash before acknowledgement, causing redelivery. Therefore:

- Handlers must be **idempotent**.
- Keep a stable business key/event ID when a side effect cannot naturally be repeated.
- Configure retry delays/backoff and a **failure transport / dead-letter workflow**.
- Separate **recoverable** failures (network timeout) from **unrecoverable** ones (invalid permanent payload).
- Monitor queue lag, failure count, retry count and handler duration.

Transactional Outbox: if you must update your database and publish an event reliably, writing the business change and an outbox record in the same database transaction avoids the "DB commit succeeded but queue publish failed" gap. A separate publisher drains the outbox. With MongoDB, whether you need a multi-document transaction depends on where the aggregate/outbox live; preserve the invariant first, optimize second.

Long-running Messenger worker gotcha: unlike short PHP-FPM requests, a worker process lives across many messages. Avoid mutable request/message-specific state in shared services, release large object graphs, use worker limits/restarts where appropriate, and handle deploy signals gracefully.

6.16 Performance & production PHP mental model

- **OPcache** stores compiled PHP bytecode so production requests do not repeatedly parse/compile every PHP file.
- Symfony compiles the service container and routes/cache metadata for production; warm caches during deploys rather than making the first live request pay the cost.
- Measure before optimizing: profiler/APM → slow endpoint → external calls/query plan/serialization/cache → fix the actual bottleneck.
- Avoid N+1, unnecessary hydration, massive `toArray()` calls, unbounded result sets, and large synchronous exports.
- In PHP-FPM, capacity roughly depends on worker count × per-worker memory/CPU/latency. A slow downstream call occupies a worker even when PHP itself is not CPU-bound.
- For huge workloads, stream/generate/batch rather than loading everything into RAM.

Strong answer to "how do you improve a slow Symfony endpoint?": reproduce with representative data, trace wall-clock time, inspect DB query plans and external-call latency, check N+1/serialization/cache behavior, make one measured change, load-test again, and add a regression metric/test where useful.

6.17 Static Analysis & Code Quality Toolchain — JD must-have, deep dive

The JD names static analysis explicitly (“PHPStan, Psalm, or similar”) as a must-have, separate from testing. Sections 4.6 and 6.4 mention PHPStan inside the CI pipeline; this section treats the tool itself, and the surrounding toolchain, as its own interview topic.

Six tools, six different jobs — don't conflate them

PHPUnit	-> Runtime correctness	(does the code behave right when run?)
PHPStan / Psalm	-> Static/type correctness	(would this ever break, without running it?)
PHP-CS-Fixer	-> Formatting/style	(does it look the way the team agreed?)
PHP_CodeSniffer	-> Coding-standard enforcement	(PSR-12 and custom rules)
Rector	-> Automated refactoring	(mechanically rewrite old patterns)
Composer Audit	-> Dependency vulnerabilities	(known CVEs in installed packages)

Where PHPStan earns its place in the pipeline that PHPUnit can't cover:

- Finds bugs on code paths no test happens to exercise — a typo'd property name, a method call on a type that can be null, an argument of the wrong type — without needing a test to trigger that path.
- Runs in milliseconds across the whole codebase, catching an entire class of mistakes before a single test is written or run.
- Understands types through the whole call graph, including PHPDoc generics (`array<int, User>`), which native PHP type hints can't express but PHPStan can still check.

Q1. What does PHPStan catch that PHPUnit doesn't?

A: PHPStan analyzes code without running it, so it catches things a test would only catch if it happened to exercise that exact path: calling a method that doesn't exist on a type, passing the wrong argument type, dereferencing a possibly-null value, unreachable code after a return, or an unused variable. PHPUnit only tells you about behavior you wrote an assertion for; PHPStan checks every line, every time, whether or not a test touches it.

Q2. What are PHPStan levels?

A: PHPStan runs at numbered strictness levels (0 through 9 in current versions) — level 0 catches only obvious errors (unknown classes/functions), and each level up adds stricter checks (undefined variables, missing return types, strict comparison issues, and at the top levels, near-complete type coverage including generics). Teams typically start a new project near the top and ratchet legacy codebases up gradually rather than jumping straight to max strictness.

Q3. What is a PHPStan baseline?

A: A generated file (`phpstan-baseline.neon`) that snapshots every *existing* error at the moment you run `phpstan analyse --generate-baseline`, and tells PHPStan to ignore exactly those errors going forward. It lets you turn on strict analysis for a legacy codebase immediately — CI fails only on *new* errors introduced from that point on, while the baseline is worked down over time instead of blocking day one.

Q4. Why can static analysis find “unreachable code” that a test suite might miss?

A: A test suite only proves that the paths it exercises behave as expected — it says nothing about code nobody's test ever reaches, including code that can genuinely never execute (after an unconditional return/throw, or behind a condition that's always false given the types involved). PHPStan reasons about control flow and types statically, so it can flag that dead code exists at all, which 100% branch coverage doesn't guarantee if a branch is dead rather than merely untested.

Q5. What are PHPDoc generics like `array<User>`, and why do they matter to PHPStan?

A: Native PHP type hints can say a parameter is an array, but not what it contains; `@param array<User> $users` in a docblock tells PHPStan (and Psalm) the array's element type, so it can catch `$users[0]->nonExistentMethod()` or passing an array of the wrong type where native PHP would just accept array and stay silent. It's how you get near-generics type safety in a language that doesn't have real generics.

Q6. When would you suppress a PHPStan warning, and how do you do it responsibly?

A: Only when you've confirmed it's a false positive PHPStan genuinely can't reason about (e.g. a dynamic property Doctrine sets via reflection) — never as a shortcut past a warning you don't understand. Suppress the narrowest possible thing (a line-level ignore with a comment explaining why, not a blanket ignore on a whole block, and never a baseline entry for genuinely new code) so the suppression documents itself and doesn't hide a real bug introduced later on the same line.

Q7. PHPStan vs Psalm — how do you choose?

A: They overlap heavily in what they check and both support levels/baselines and generics via PHPDoc; the practical differences are ecosystem and taste — PHPStan has a larger plugin ecosystem for frameworks (Symfony, Doctrine extensions) and is more common as the default on new projects, while Psalm has historically had strong dedicated support for security-taint analysis. In practice teams pick whichever the project already uses and rarely run both; “which do you use” is really “which does this stack already have,” worth asking about in the interview itself (Section 10).

Q8. Static analysis vs linting — what's the difference?

A: A linter (PHP_CodeSniffer, PHP-CS-Fixer's check mode) verifies style and formatting — indentation, brace placement, naming conventions, PSR-12 compliance — largely without understanding what the code does. Static analysis (PHPStan/Psalm) verifies correctness — types, reachability, nullability — by reasoning about the code's logic. A file can be perfectly linted and still be full of PHPStan-catchable bugs, and vice versa.

Q9. Would PHP-CS-Fixer belong in CI, and what should fail the build vs just auto-fix locally?

A: Yes — but usually in a “check” mode (dry-run with a diff) that fails the build if code doesn't match the configured style, rather than having CI silently rewrite and commit code. Locally, developers run it in fixing mode (often via a pre-commit hook) so style issues never reach CI in the first place; CI's job is to be the backstop that catches anyone who skipped that step, not the primary place style gets fixed.

Q10. What is Rector, and how is it different from PHPStan?

A: Rector automatically rewrites code — upgrading syntax to a newer PHP/Symfony version, applying a coding standard, or removing dead patterns — by understanding and transforming the AST, whereas PHPStan only reports problems without touching code. They're often used together: PHPStan tells you what's wrong; Rector can mechanically fix entire categories of it across a whole codebase (e.g. converting array-based annotations to PHP 8 attributes) far faster than doing it by hand file by file.

Q11. How do you introduce PHPStan onto a large legacy codebase without the build failing on day one?

A: Start at a low level (or let PHPStan pick the highest level the codebase already passes), generate a baseline to freeze existing errors, and wire CI to fail only on new errors from that point forward. Then raise the level and shrink the baseline deliberately over time — as a background task or a light touch whenever you're already editing a file — rather than as a blocking, all-or-nothing migration.

Q12. What's the relationship between `strict_types`, typed properties, and what static analysis can actually verify?

A: `declare(strict_types=1)` and native typed properties/parameters are enforced at runtime by PHP itself — they throw a `TypeError` on mismatch, a real guarantee but only for the exact call that happens to run. Static analysis reasons about *every* call site across the whole codebase without running any of them, so it catches type mismatches PHP's runtime enforcement would only catch if that specific code path executed with those specific values — complementary layers, not substitutes for each other.

6.18 Feature Ownership & Accountability — end-to-end lifecycle

“Own features end-to-end,” “high ownership mindset,” and “accountability” appear repeatedly in this JD — treat this as a distinct interview theme with its own vocabulary, not just an attitude to project. Interviewers probe it with open questions (“what does ownership mean to you?”) precisely because a vague answer is easy to spot.

The ownership lifecycle — one feature, start to finish

Requirement / ticket	
Clarify acceptance criteria	(ask, don't assume -- Section 6.13)
Technical design	(Section 5 patterns, Section 17 synthesis)

Identify risks/dependencies	
Break into tasks	
Implementation	
Tests / TDD	(Section 4.9)
Static analysis	(Section 6.17)
Code review	(Section 6.10)
QA	
Deployment	(Section 14)
Monitoring	(Section 6.7, 14.11)
Production validation	
Own bugs/incidents afterwards	(Section 8's on-call triage story)

What separates ownership from responsibility, in an answer an interviewer will actually credit:

- Responsibility is doing the task you were assigned correctly; ownership is noticing the task was under-specified, missing a dependency, or a bad idea, and saying so before it ships — not just executing what the ticket literally said.
- Ownership doesn't stop at “merged” or “deployed” — it continues through whether the feature actually works in production and stays working, including the incident three weeks later.
- Ownership includes proactively communicating risk/status upward, not waiting to be asked — the JD's “excellent technical communication” bullet showing up inside the ownership bullet.

Q1. What does feature ownership mean to you?

A: Owning a feature means the outcome is on me from the moment the ticket is understood to well after it ships — not just writing the code, but clarifying ambiguous requirements up front, flagging risks or missing edge cases before implementation starts, making sure it's properly tested and reviewed, watching it in production after release, and being the person who responds if it breaks rather than waiting for someone else to notice. It's the difference between “I did what the ticket said” and “I made sure this actually works, and I'll know if it stops.”

Q2. Tell me about a time you owned a feature that had a bug in production weeks after launch — what did you do?

A: Use this structure for your real story, STAR-style: Situation/Task — what the feature was and what broke; Action — how you found out (monitoring vs. a report), how you triaged (Section 8's on-call process: check recent deploys, logs, error spikes), whether you rolled back or hotfixed, and crucially whether you added a regression test/alert so the same bug can't silently recur; Result — what changed afterward (a new test, a new monitor, a process tweak). The strongest answers show you treated it as your problem to close out, not just to patch.

Q3. How do you push back on a deadline when you believe the scope can't be done safely in the time given?

A: Bring the trade-off, not just the objection — “the full version needs X more days for Y and Z reasons; here's a smaller slice we could ship on time that covers the core case, with the rest as a fast follow” is a plan, whereas “we can't make it” is just a complaint. Raise it as early as possible, in writing where it's visible (not just verbally in a hallway), so the decision-maker has time to actually choose rather than finding out at the deadline.

Q4. What's the difference between “responsibility” and “ownership” in a team context?

A: Responsibility is usually assigned to you — a ticket, a component, an on-call rotation. Ownership is taken, not assigned — it shows up as noticing something outside your assigned scope is about to cause a problem and saying something anyway, or continuing to care about a feature's health after the ticket that created it is long closed. A team of people who only do their assigned responsibilities can still ship a broken system if nobody owns the gaps between tickets.

Q5. How do you decide when a task is “done” — ticket closed, or feature actually working in production?

A: “Done” means verified in production, not merged or deployed — for anything customer-facing, that means checking the actual behavior post-deploy (a smoke test, a dashboard, a specific log line), not assuming the deploy succeeding means the feature works. For lower-risk internal changes, watching the relevant metric/error rate for a defined window after release is usually enough; the point is that closing the ticket is a process step, and it shouldn't happen before someone has actually confirmed the outcome the ticket promised.

6.19 Communicating & Defending Technical Decisions

The JD lists this as its own must-have — “excellent technical communication skills and ability to explain and defend technical decisions” — separate from knowing the material. Being right about a technical choice and being able to make a stakeholder understand and trust that choice are different skills, and this one gets tested directly in behavioral rounds.

A structure for defending a decision, instead of just restating it more firmly

- State the context/constraint the decision was made under — deadline, existing architecture, team size — so it reads as reasonable given what was known, not as an isolated opinion.

- Name the alternatives you actually considered, briefly — this shows the decision wasn't the only option you thought of, usually what a skeptical reviewer actually wants to hear.
- State the trade-off plainly, including the downside of the choice you made — a decision with no acknowledged downside sounds unexamined, not confident.
- Say what would change your mind — a concrete condition under which you'd revisit it, which signals the decision was reasoned, not defended out of ego.

Documenting decisions — the JD's “document architecture, decisions, and processes” bullet

- An Architecture Decision Record (ADR) is a short, dated document capturing one decision: the context, the options considered, the choice, and the consequences — written once, referenced repeatedly, instead of the reasoning living only in someone's memory or a buried chat thread.
- Not every decision needs one — reserve ADRs for choices that are expensive to reverse (a datastore, a messaging pattern, a public API shape) rather than every implementation detail.

ADR-014: Use Redis for the product-catalog cache

Status: Accepted

Context:

Catalog reads dominate traffic; the primary datastore is under load during peak hours.

Decision:

Cache catalog reads in Redis with a 10-minute TTL and tag-based invalidation on product updates.

Consequences:

- + Read latency and DB load drop significantly.
- Adds an operational dependency (Redis availability, memory sizing) and a cache-invalidation edge case to reason about on every write path.

Revisit if:

Catalog write frequency increases enough that invalidation traffic approaches read traffic.

Q1. Walk me through a technical decision you made that a non-technical stakeholder disagreed with — how did you explain it?

A: Structure your real example this way: lead with the business consequence of each option in the stakeholder's terms (cost, timeline, risk) rather than the technical mechanism first; name the option they were asking for and why it looked attractive; then state the trade-off you were protecting them from (e.g. “the faster option would work today but breaks the moment we add a second region”). Close with what you'd need to see to reconsider — it turns a disagreement into a shared decision with a clear reopening condition, rather than a standoff.

Q2. How do you write documentation that both engineers and product managers can use?

A: Separate the “what and why” from the “how” — a short summary of the decision and its business consequence at the top (readable by anyone), with implementation detail (code, configuration, sequence diagrams) further down for engineers who need it. An ADR's Context/Decision/Consequences shape does this naturally; a document that opens with a class diagram loses the non-technical reader in the first paragraph.

Q3. A teammate reviews your PR and disagrees with your architectural choice — how do you handle it?

A: Ask for their reasoning and specific concern before defending — a disagreement is often about a trade-off they're weighing differently, not a factual error, and you can't respond to the real objection until you know what it is. If you still believe your approach is right after hearing them out, restate the trade-off explicitly and offer a concrete way to resolve it (a quick sync, a spike, or agreeing to revisit after shipping) rather than a review comment war — ties directly to Section 6.10's code-review checklist and Section 8's “disagreement during a code review” behavioral question.

Q4. How do you decide when a decision needs a documented ADR versus a quick message?

A: Weigh reversal cost, not decision size — a choice that's cheap to change later (a variable name, a small refactor) doesn't need a document even if it took thought; a choice that's expensive to unwind (a datastore choice, a public API contract, a cross-team messaging pattern) deserves an ADR even if the decision itself felt obvious at the time, because the *reasoning* is what future engineers — including you — will need once the context has faded.

Part VI — Advanced Symfony & Interview Practice

Section 7 — Advanced Symfony Concepts: Q&A

These are the questions that separate "used Symfony" from "knows Symfony internals" — exactly what the JD's "Laravel-only is not qualifying" line is testing for.

Q1. What's a Compiler Pass, and when have you needed one?

A: A `CompilerPassInterface` runs after all service definitions are loaded but before the container is dumped/compiled — it lets you programmatically inspect/modify service definitions, e.g. collect all services tagged `app.report_generator` and inject them as an array into a `ReportManager` service. Registered in a `Bundle's build()` method or via `extension.php` in newer Symfony versions.

Q2. Difference between an EventListener and an EventSubscriber?

A: A Listener is configured externally (in YAML/attributes) mapping one event → one method; a Subscriber implements `getSubscribedEvents()` so the class declares its own event map (self-contained, preferred for discoverability). Functionally equivalent at runtime — Subscribers are just a cleaner pattern for anything beyond a single trivial listener.

Q3. What is service decoration, and how do you decorate a service?

A: Wrapping an existing service with another that implements the same interface, delegating to the original ("original service" is autowired into the decorator via a special alias). Classic use: wrap `LoggerInterface` to add request-context to every log line without touching Monolog's code.

```
#[AsDecorator(decorates: MailerInterface::class)]
final class LoggingMailerDecorator implements MailerInterface
{
    public function __construct(
        #[AutowireDecorated] private MailerInterface $inner,
        private LoggerInterface $logger,
    ) {}

    public function send(RawMessage $message, ?Envelope $envelope = null): void
    {
        $this->logger->info('Sending email', ['subject' => $message instanceof
Email ? $message->getSubject() : null]);
        $this->inner->send($message, $envelope);
    }
}
```

Q4. Explain the modern Symfony Security authenticator system.

A: Since Symfony 5.3+, custom auth logic implements `AuthenticatorInterface` (usually extending `AbstractAuthenticator`), overriding `supports()` (should this authenticator handle this request?), `authenticate()` (returns a `Passport` containing the user badge + credentials), `onAuthenticationSuccess()`, and `onAuthenticationFailure()`. This replaced the older, more complex `Guard` system.

Q5. How would you write a custom Voter and why use Voters over role checks?

A: Voters let you express authorization logic that's more granular than a static role (e.g. "can THIS user edit THIS specific post because they authored it"), and they're composable/testable in isolation.

```
final class PostVoter extends Voter
{
    protected function supports(string $attribute, mixed $subject): bool
    {
        return $attribute === 'EDIT' && $subject instanceof Post;
    }

    protected function voteOnAttribute(string $attribute, mixed $subject,
    TokenInterface $token): bool
    {
        $user = $token->getUser();
        return $user instanceof User && $subject->getAuthorId() === $user-
    >getId();
    }
}
```

Q6. What is the N+1 problem and how does Doctrine help you avoid it?

A: (see Section 3.8) — in ORM you fix it with `JOIN/fetch`: `EAGER` mappings or explicit `->leftJoin()->addSelect()` in a custom DQL query so related entities come back in the same query instead of one query per row.

Q7. Sync vs async Messenger transports — how do you decide?

A: sync (default) executes the handler immediately, in-request — fine for fast, must-succeed-before-responding work. Async transports (Doctrine/AMQP/Redis/SQS) queue the message and a separate worker process consumes it — use for slow, non-blocking, or retryable work (emails, webhooks, report generation, image processing). Consider **message priority**, **retry/backoff**, and **dead-letter queues** for messages that fail repeatedly.

Q8. What are Kernel events fired in, and why does order matter?

A: `kernel.request` → `kernel.controller` → `kernel.controller_arguments` → `kernel.view` → `kernel.response` → `kernel.finish_request` → `kernel.terminate` (plus `kernel.exception` on error). Order matters because listeners can short-circuit later stages (e.g. a `kernel.request` listener returning an early `Response` for a cache hit skips the controller entirely) — and listener **priority** controls execution order within the same event.

Q9. How does Symfony's DI container achieve performance despite all the reflection-based autowiring?

A: Autowiring/reflection only happens once, at **container compile time** (bin/console cache:warmup / first request in prod), producing a compiled PHP class (ContainerXxxx.php) with hard-coded service instantiation — so runtime has zero reflection overhead. This is why changing services.yaml requires a cache clear in prod.

Q10. What's the Workflow component and when would you reach for it?

A: Implements finite state machines / workflows declaratively (e.g. an Order moving through draft → submitted → approved → shipped), with guards (conditions before a transition) and events fired on each transition — better than scattering if (\$status === 'x') checks across the codebase.

Q11. Explain Symfony's caching of the DI container vs application-level caching — don't confuse them.

A: Container caching = internal compiled service graph (Section Q9), invisible to your business logic. Application caching (Cache component/Redis) = your own explicit \$cache->get('key', fn() => expensiveQuery()) calls to avoid redoing expensive work — two completely different layers, common trick question.

Q12. How do you handle configuration/secrets across environments (dev/staging/prod)?

A: .env files for defaults (committed), .env.local/.env.\$APP_ENV.local for machine-specific overrides (gitignored), and Symfony's **Secrets Vault** (bin/console secrets:set) for production credentials encrypted at rest, decrypted only with a private key on the server — never commit real secrets to .env.

Q13. Doctrine transactions — how do you ensure multiple writes are atomic?

A: Wrap in \$entityManager->wrapInTransaction(function() { ... }) (or beginTransaction()/commit()/rollback() manually) so either all writes succeed or all roll back. In MongoDB, multi-document ACID transactions are supported (since MongoDB 4.0, requiring a replica set) via DocumentManager's session API — but design toward single-document atomicity where possible since Mongo transactions carry a performance cost.

Q14. What is the Rate Limiter component used for?

A: Built-in token bucket / sliding window limiter for throttling API requests, login attempts, etc., without needing an external service for simple cases.

Q15. You started on Symfony 2.8 — what actually changed most between then and modern Symfony?

A: Three big shifts: (1) manual service wiring in services.yml → autowiring/autoconfiguration (Symfony 3.3+), (2) manual AppKernel::registerBundles() → Symfony Flex auto-registering bundles via recipes (4.0+), and (3) the Guard authentication system (introduced in 2.8 itself) being deprecated in favor of the new Authenticator system (stable since 5.3). Routing/mapping also moved from docblock annotations to native PHP attributes once Symfony required PHP 8 (6.0+). See Section 2.15 for the full timeline.

Q16. Guard authenticators vs. the modern Authenticator system — what's actually different?

A: Functionally similar shape (both implement a "does this request contain credentials, validate them, decide success/failure" flow), but the new system has a single unified `AuthenticatorManagerListener` instead of each auth mechanism registering its own listener, models a request's credentials as a `Passport` object with composable "badges" (e.g. `UserBadge`, `PasswordCredentials`, `CsrfTokenBadge`), and removed the concept of "anonymous users" as a special token type. Migrating meant rewriting `GuardAuthenticatorInterface` implementations to `AuthenticatorInterface/AbstractAuthenticator`.

Q17. How do you safely run a schema-changing migration against a production table with millions of rows?

A: Follow expand/contract (Section 3.9): add the new column without touching the old one, backfill in small batches during low-traffic windows (never one giant `UPDATE`), switch application reads over once backfilled, and only drop the old column in a later, separate deploy. For MySQL specifically, watch out for migrations that take an exclusive table lock — prefer online schema-change tools or Doctrine migrations broken into safe incremental steps.

Q18. Why did Symfony move from annotations to attributes, and is there a real technical difference, not just syntax?

A: Annotations (`@Route(...)` in a docblock) were always just *comments* — Doctrine's Annotations library had to parse comment text with a mini-parser at runtime/cache-warmup, which was slow and fragile (no IDE/static-analysis support, easy to typo silently). PHP 8 **attributes** (`#[Route(...)]`) are actual language syntax, parsed by the PHP engine itself via reflection — faster, type-checked by tooling, and don't require a third-party annotation-parsing library at all. It's a real architectural improvement, not cosmetic.

Q19. How do you expose a Symfony API to a React/Next.js frontend on a different origin — walk through the pieces.

A: `nelmio/cors-bundle` configured with the frontend's allowed origin(s)/methods/headers; a stateless auth mechanism (JWT via `lexik/jwt-authentication-bundle`, Bearer token in the Authorization header) since there's no shared session across origins; the Serializer (or API Platform) controlling exact JSON shape/versioning; and being explicit that Next.js's server-side data fetching hits the API server-to-server (bypassing browser CORS entirely) while client-side fetches go through the full CORS flow. See Section 2.17.

Q20. What's Mercure, and when would you reach for it instead of polling or a custom WebSocket server?

A: Mercure is Symfony's built-in real-time component, using Server-Sent Events (SSE) for pub/sub — the server publishes updates to "topics," subscribed clients (e.g. a React frontend) receive them over a single long-lived HTTP connection. Pick it over polling when you need near-real-time updates without hammering the API with repeated requests, and over rolling your own WebSocket infrastructure when you don't need bidirectional communication (SSE is one-way, server→client, which covers most "live update the UI" use cases with far less operational complexity than WebSockets).

Q21. EasyAdminBundle or SonataAdminBundle — how do you actually decide?

A: EasyAdmin for speed — a small team needing internal CRUD screens live in hours with minimal config. SonataAdmin when you need deep customization: per-role field permissions via ACL, complex nested/related-entity forms, or a highly bespoke back-office UX. In a MongoDB-first codebase (like this JD's stack), I'd also weigh a third option: skipping both and building a thin custom admin against the same API the main frontend uses, since neither bundle has mature ODM support out of the box (Section 2.16).

Q22. How do you evolve a MongoDB document's shape over time without a formal schema migration tool?

A: A few patterns, often combined: batch-processed migration scripts run via a custom console command (never one massive in-memory pass — cursor/batch through the collection); a `schemaVersion` field per document so the app can detect and lazily upgrade old-shape documents on read; or a temporary dual-read/dual-write period in application code while a background job backfills the new shape. See Section 3.9.

Q23. What's late static binding, and where would it bite you in real Symfony code?

A: `static::` resolves to the class actually called at runtime, while `self::` always resolves to the class where the method is defined — so a `Model::create()`-style factory method using `new static()` correctly returns the calling subclass, but the same method using `new self()` would incorrectly always return the base class. Comes up in abstract base repositories, test-case base classes with static factory helpers, and any "fluent base class" pattern. See Section 1.7.

Q24. PHP arrays are described as "ordered hash maps" — why does that matter practically?

A: It means keyed lookups (`$arr[$id]`) are $O(1)$ average, same as a dedicated hash map/dictionary in other languages — but scanning for a *value* (`in_array()`) is still $O(n)$ because there's no secondary value-index. Practically: build a lookup array keyed by ID before joining two datasets in a loop (avoids the $N+1$ -style $O(n^2)$ pattern), and avoid `array_shift()/array_unshift()` in hot loops since removing/inserting at the front re-indexes the whole array (Section 6.9).

Q25. How does NoSQL injection work against MongoDB, and how do you actually prevent it?

A: It doesn't need special characters like SQL injection — it's operator injection through unvalidated input. If user input is merged raw into a query filter, an attacker can submit a Mongo operator (`{"$ne": null}, {"$gt": ""}`) instead of a plain value and change the query's meaning entirely — the classic example is bypassing a login check because the "password" field became "password is not null" instead of "password equals this string." Prevention: validate and cast every field to its expected scalar type before it reaches a query (typed DTOs + the Validator component naturally do this), and never pass a raw request array straight into a query builder. See Section 3.10 and 6.6.

Q26. What's the real architectural difference between Doctrine and an Active Record ORM like Eloquent?

A: Doctrine is a **Data Mapper** — entity/document classes are plain PHP objects with no knowledge of persistence; a separate EntityManager/DocumentManager maps them to storage and tracks changes via a Unit of Work. Active Record (Eloquent) bakes `save()/find()/delete()` directly into the model class itself, coupling domain data to persistence logic. Neither is objectively "better" — Data Mapper scales better for complex domain logic and testability (you can unit-test a Doctrine entity with zero database involved), Active Record is faster to start with for simple CRUD. This is the precise technical answer to "how is Symfony different from Laravel," not a vibes-based one.

Q27. When would you actually need to shard a MongoDB collection, and what's the biggest risk in doing it?

A: Once a single replica set's working set (hot data + indexes) no longer fits comfortably in RAM, or write throughput on one primary becomes the bottleneck — not before. The biggest risk is a poorly chosen **shard key**: it needs high cardinality and a write pattern that spreads evenly across shards, or you end up with a "hot shard" that defeats the whole point. Premature sharding — doing it before you've actually measured a real bottleneck — is a common, avoidable mistake. See Section 3.10.

Section 8 — Rapid-Fire Question Bank

Use these for a final self-test the night before. Try to answer out loud in under 60 seconds each before checking your notes above.

PHP

1. What's the difference between `==` and `===`? (*loose vs strict comparison — type coercion*)
2. What happens when you pass an object to a function — by value or reference? (*the variable holding the object handle is passed by value, but it points to the same object — mutating the object's properties inside the function affects the original*)
3. `abstract class` vs `interface` — when would you pick one over the other?
4. What's a `readonly` property good for?
5. Explain `match` vs `switch`.

6. What's the difference between `include/require` and `include_once/require_once`?
7. What's the difference between `self::` and `static::`? (*late static binding* — Section 1.7)
8. Shallow vs deep clone in PHP — how do you implement the latter?
9. Why are PHP associative arrays effectively hash maps, and what's the practical performance consequence?
10. What's the difference between an Error and an Exception in PHP 8, and why does that split exist?
11. What does `JsonSerializable` let you control that a plain public-property object doesn't?

Symfony 12. What happens when you run `composer require symfony/xxx` — what is Symfony Flex doing? 13. Walk me through what happens between a browser request and a Response being returned. 14. How do you inject a specific implementation when two services implement the same interface? 15. What's the difference between `EntityManagerInterface` and a Repository? 16. How would you add a new custom Twig filter or function, and how is that different from a Twig macro? 17. How do you test a service that sends an email without actually sending one? 18. What's the difference between a Bundle and a Component? 19. `debug:autowiring` shows a service isn't found — what are the likely causes and how do you fix each? 20. What's the difference between annotations and attributes for routing/mapping, and why did Symfony move to the latter? 21. Guard vs the new Authenticator system — what actually changed? 22. Walk me through generating a new entity/document, a migration, and a CRUD controller using the Maker Bundle. 23. How would you safely roll back a bad migration that already ran in production? 24. What's EasyAdminBundle or SonataAdminBundle used for, and what's the catch when your primary datastore is MongoDB? 25. How do you connect a Symfony API to a React or Next.js frontend hosted on a different domain — what has to be configured for it to work at all? 26. What is Mercure, and what problem does it solve that plain polling doesn't?

MongoDB & Databases 27. When would you choose an embedded document over a reference? 28. How do you enforce data validation in a schemaless database? 29. What does `explain()` tell you about a query? 30. What does ACID stand for, and does MongoDB support it? 31. What's the CAP theorem, and how does it show up in MongoDB replica-set configuration? 32. Why do both SQL and MongoDB commonly use B-tree-style indexes specifically, rather than some other structure? 33. How do you migrate/evolve a MongoDB document's shape without a formal "ALTER TABLE"-style tool?

Design Patterns / OOP 34. Give an example of the Observer pattern outside of EventDispatcher. 35. What's the difference between Dependency Injection and the Service Locator (anti-)pattern? 36. Why is "favor composition over inheritance" good advice — give a concrete example. 37. Name a place covariant return types matter in a real Symfony/Doctrine codebase.

Data Structures & Complexity 38. What's the Big-O of looking up a value by key in a PHP array vs. searching for a value with `in_array()`? 39. Why is calling `array_shift()` in a loop over a large array a performance smell, and what's the fix? 40. When would you reach for `SplObjectStorage` instead of a plain array keyed by `spl_object_id`? 41. Stack vs Queue — give one real backend use case for each.

General Backend 42. How do you decide something belongs in a queue (async) instead of the request cycle? 43. What's idempotency and why does it matter for a PUT vs POST endpoint? 44. Your team is on-call and gets paged for a slow endpoint at 2am — what's your triage process? (*check recent deploys → check logs/metrics for error spikes → check DB slow query log → check for cache misses/queue backlog → mitigate, e.g. rollback/feature-flag, then root-cause post-incident*) 45. What's the "expand/contract" pattern and why does it matter for zero-downtime schema changes?

Behavioral / soft-skill questions (use the STAR method — Situation, Task, Action, Result)

- "Tell me about a time you mentored a junior developer."
- "Describe a disagreement you had during a code review — how was it resolved?"
- "Tell me about a production incident you handled while on-call."
- "How do you break down a large project into manageable tasks for a team?"
- "Give an example of when you pushed back on a requirement and why."

Since the JD stresses mentoring, code review, and cross-functional communication heavily, prepare **one concrete story for each** ahead of time — vague answers here read as a red flag for a "Senior" title.

Section 9 — Demo Code Playground (runnable)

The package includes demo-code/ so you can **run, break and modify** the concepts instead of memorizing them:

- 01_php_basics.php — strict types, enums, readonly value object, match, nullsafe access, exceptions and generators.
- 02_oop_solid_di.php — interfaces, constructor injection, repository abstraction, Strategy and Dependency Inversion.
- 03_design_patterns.php — Strategy, Decorator, Observer/Event Dispatcher style, Factory and Command examples with Symfony mapping notes.
- 04_data_structures.php — associative lookup, SplQueue, generators and complexity-oriented examples.
- 05_idempotency_and_concurrency.php — stable idempotency keys and optimistic-version thinking in a small runnable simulation.
- 06_OrderServiceTest.php — PHPUnit-style unit test using modern attributes; requires PHPUnit to execute.
- symfony-order-api/ — current-style Symfony snippets for DTO validation, Route + #[MapRequestPayload], MongoDB ODM attributes, repository/service separation, Voter, Messenger handler and tests.
- README.md — exact commands plus modification exercises.

Run standalone demos with `php demo-code/01_php_basics.php` etc. For the PHPUnit/Symfony snippets, follow the README and install the project dependencies first.

Symfony-specific code (needs a real Symfony app to execute)

The full, current-syntax Symfony examples are inline earlier in this guide — copy these into a real project (`composer create-project symfony/skeleton my-app`) to try them for real:

- Attribute-based routing, prefixes, `#[MapEntity]` — **Section 2.4**
- Custom Twig filter/extension, macros, form theming — **Section 2.5**
- Validator constraints on a DTO — **Section 2.6**
- Security Voter — **Section 2.7** and **Section 7, Q5**
- `bin/console` debugging commands + Maker Bundle scaffolding — **Section 2.13**
- EasyAdminBundle CRUD controller — **Section 2.16**
- CORS + JWT setup for a React/Next.js frontend — **Section 2.17**
- Doctrine MongoDB ODM Document + repository query + aggregation — **Section 3.2, 3.4, 3.5**
- A Doctrine migration class (`up()`/`down()`) and the expand/contract pattern — **Section 3.9**
- Symfony functional test with `WebTestCase` — **Section 4.4**
- Service decoration with `#[AsDecorator]` — **Section 7, Q3**

If you have time before the interview, spin up a small Symfony project (Symfony CLI or Composer), add `doctrine/mongodb-odm-bundle`, and wire up one real Document + Controller + PHPUnit test end-to-end. Nothing beats having touched the real thing in the last 48 hours.

One feature, end-to-end — the synthesis this guide can't give you by reading alone

Everything above is organized by topic, which is great for lookup but doesn't rehearse the question interviewers actually ask most: *"walk me through how you'd build this."* That question tests whether you can **connect** the pieces, not recite them individually. Here's one complete feature, touching almost every section above, that you should be able to narrate end-to-end without notes.

Feature: "Add a comment to an Order, and notify anyone watching it — without blocking the request."

1. Data design decision first (Section 3.3, 3.10): comments are small, always read with their parent Order, and rarely need independent querying — classic case for the **Subset pattern**: embed the most recent N comments directly on the Order document, and if the full history ever matters, reference an archive collection for anything beyond that. Say this trade-off out loud unprompted — it shows you design before you code.

```
#[ODM\Document(collection: 'orders')]
class Order
{
```

```
#[ODM\EmbedMany(targetDocument: OrderComment::class)]
private Collection $recentComments; // capped at, say, the latest 20 in
application logic
}
```

2. Input validation (Section 2.6) — never trust the request body directly into a query or a document (this is also your NoSQL-injection defense from Section 6.6):

```
final class AddCommentDto
{
    #[Assert\NotBlank]
    #[Assert\Length(max: 2000)]
    public string $body;
}
```

3. Routing + controller (Section 2.4) — thin controller, no business logic in it (the exact thing Section 6.10's code-review checklist flags if violated):

```
#[Route('/api/orders/{id}/comments', name: 'order_comment_add', methods:
['POST'])]
public function addComment(Order $order, #[MapRequestPayload] AddCommentDto
$dto, OrderCommentService $service): JsonResponse
{
    $this->denyAccessUnlessGranted('COMMENT', $order);
    $service->addComment($order, $dto->body, $this->getUser());
    return $this->json(['status' => 'created'], 201);
}
```

4. Authorization as a Voter, not an if block (Section 2.7, 5, 7 Q5) — "can this user comment on this order" is exactly the granular, per-object check Voters exist for:

```
final class OrderVoter extends Voter
{
    protected function supports(string $attribute, mixed $subject): bool
    {
        return $attribute === 'COMMENT' && $subject instanceof Order;
    }
    protected function voteOnAttribute(string $attribute, mixed $subject,
TokenInterface $token): bool
    {
        $user = $token->getUser();
        return $user instanceof User && ($subject->getCustomerId() === $user-
>getId() || $subject->hasAssignedAgent($user));
    }
}
```

5. Service layer orchestrates, behind your own Repository interface (Section 5's Enterprise patterns, 4.8) — not DocumentManager injected straight into the controller:

```
final class OrderCommentService
{
    public function __construct(
        private OrderRepositoryInterface $orders, // your own interface, not
DocumentManager directly
        private MessageBusInterface $bus,
        private RateLimiterFactory $commentLimiter, // Section 7 Q14 – stop
comment spam
    ) {}
}
```

```

    public function addComment(Order $order, string $body, User $author): void
    {
        if (!$this->commentLimiter->create($author->getId())->consume(1)-
>isAccepted()) {
            throw new TooManyRequestsHttpException();
        }
        $order->addRecentComment(new OrderComment($body, $author->getId()));
        $this->orders->save($order); // Unit of Work flush happens inside your
Repository implementation

        $this->bus->dispatch(new NotifyOrderWatchers($order->getId())); // async
- Section 2.10, 6.3
    }
}

```

6. Async notification, not inline (Section 2.10, 6.3) — the request returns immediately; a Messenger handler (on an async transport) does the actual notification fan-out, optionally pushing a live update via **Mercure** (Section 2.17) so a React/Next.js frontend updates without polling.

7. Tests at both levels (Section 4.8): a unit test mocking `OrderRepositoryInterface` and `MessageBusInterface` to assert the service dispatches exactly one message and never touches Doctrine directly; a `WebTestCase` functional test hitting the real route and asserting a 403 for a user who isn't the order's customer (proving the Voter actually runs), and a 201 for one who is.

Why this matters more than any single fact in this guide: notice how the data-modeling decision in step 1 justifies the code in every later step — that's the kind of connected reasoning a "tell me about a feature you built" or "how would you design this" question is actually listening for, far more than whether you can define the Observer pattern in isolation.

Section 10 — Interview Day Playbook

- Lead with Symfony specifics, not "it's like Laravel."** The JD explicitly flags Laravel-only experience as disqualifying — never analogize to Laravel unprompted. If you've used both, mention Symfony-specific vocabulary confidently (services, compiler passes, voters, the container) rather than translating from Laravel terms.
- Have one concrete story ready for:** a production incident you handled (on-call), a code review disagreement, mentoring a junior dev, and breaking down a large feature into tasks — all directly called out in the JD's responsibilities. Use STAR (Situation, Task, Action, Result).
- Be ready to whiteboard or live-code:** a Voter, a Strategy-pattern refactor, or a simple PHPUnit test with a mock — these come up constantly for this exact profile.
- Ask questions back** — shows seniority and genuine interest:
 - "What does the current Symfony version and PHP version look like in production?"
 - "Is MongoDB the primary datastore, or used alongside a relational DB for specific use cases?"
 - "How is the on-call rotation structured, and what does the incident/escalation process look like?"

- "What does the code review process look like, and how much autonomy does a senior engineer have in technical decisions?"
- 5. **Logistics reminder:** the role runs fixed PKT shift hours (11:30 AM–9:00 PM in summer) with no weekend work — worth confirming during the call if time zone/schedule fit matters to you.
- 6. **Final gut-check:** for every bullet in "Must-Have Skills," you should be able to say a specific *thing you built* — not just "I've used it." Interviewers probe for depth by asking "tell me more about that" on whatever you claim.

Section 11 — ADVANCED LEVEL: senior internals, architecture & final Q&A

This is deliberately the **last technical section**. Read it after you can already explain PHP/OOP, the request lifecycle, DI, MongoDB and PHPUnit. These questions are useful because a senior interviewer often keeps drilling until the candidate stops reciting framework APIs and starts explaining mechanisms and trade-offs.

11.1 What actually happens to the service container in production?

Symfony loads service definitions, resolves aliases/bindings/tags, runs compiler passes, removes/inlines services where possible, and **dumps a compiled PHP container** into cache. Autowiring is primarily a build/compile-time convenience; Symfony's official docs emphasize that the compiled container means there is no runtime autowiring penalty.

Q: Why use a Compiler Pass?

A: When registration cannot be expressed cleanly with normal autoconfiguration/tags alone. Example: discover every service tagged `app.exporter`, validate metadata, sort it and wire the collection into an `ExporterRegistry`. Don't use a compiler pass merely because it feels "advanced" — normal DI configuration is easier to understand and test.

11.2 Tagged iterators, service locators and Strategy at scale

Instead of injecting 20 implementations manually:

```
final class ExporterRegistry
{
    /** @param iterable<ExporterInterface> $exporters */
    public function __construct(private iterable $exporters) {}
}
```

Autoconfiguration/custom tags can collect implementations. If instantiating every strategy would be expensive, a **service locator** can lazily resolve only the requested implementation. Trade-off: locators hide dependencies more than explicit constructor arguments, so use them for a deliberate dynamic plugin/registry problem, not as a global container shortcut.

11.3 Service decoration vs inheritance

Question: "I need to add tracing around an existing third-party service without modifying it. What Symfony mechanism fits?"

Answer: Service decoration. The decorator implements the same interface, receives the inner service, adds behavior before/after delegation, and preserves callers. It is the Decorator pattern and is usually safer than subclassing a vendor class.

11.4 Custom argument/value resolution

Modern Symfony can map request data to controller arguments/DTOs. For domain-specific resolution — e.g. resolve {tenantId} plus the authenticated user to a TenantContext — a custom **ValueResolver** can keep repetitive lookup logic out of controllers. Use it when resolution is infrastructure/controller plumbing, not when it hides substantial business behavior.

11.5 Serializer depth

Know these terms:

- **Normalizer:** object ↔ array/scalar representation.
- **Encoder:** array/scalar ↔ format such as JSON/XML.
- **Serialization groups/context:** control which fields are exposed for a use case.
- **Custom normalizer:** for domain-specific representation that built-in object normalization cannot express cleanly.

Senior warning: do not serialize persistence objects blindly. API contracts should not accidentally expose internal fields or trigger lazy-loading/N+1 behavior. DTO/resource models are often clearer at public boundaries.

11.6 Messenger middleware — the clean Chain of Responsibility example

An Envelope travels through ordered middleware. Middleware can add tracing, validation, transaction handling, dispatch routing or custom behavior before/after the next middleware.

Q: Can a Messenger message be delivered twice?

A: Yes. Normal failure/redelivery scenarios can produce duplicate delivery. Design handlers to be naturally idempotent, or use a stable business idempotency key for non-idempotent operations. Configure retry strategy and failure transport rather than swallowing exceptions.

Q: Would you send an email inside the request after creating an order?

A: If the email is not required to make the order transaction successful, usually dispatch an async message after the core write. The API latency and availability should not depend on the email provider. Then make the handler retryable/idempotent and observable.

11.7 Domain events vs integration events

- **Domain event:** expresses something meaningful that happened inside your domain model/bounded context, e.g. `OrderConfirmed`.
- **Integration event:** contract published to other services/systems; should be stable/versioned and avoid leaking internal object shapes.

Don't equate "EventDispatcher event" with "distributed event". In-process event dispatch and a durable message broker have different failure and delivery semantics.

11.8 Workflow/State Machine

For states such as `draft` → `submitted` → `approved/rejected` → `paid`, Symfony Workflow can centralize allowed transitions and guards instead of spreading `if ($status === ...)` checks across controllers/services.

Q: When is Workflow overkill?

A: When there are only one or two obvious transitions and no cross-cutting guards/events. A simple domain method plus enum may be clearer. Use Workflow when transition rules, hooks, auditability or multiple paths justify it.

11.9 Cache stampede / dogpile

If an expensive cached value expires and 500 requests miss simultaneously, all 500 may recompute it. Mitigations include locking/single-flight computation, probabilistic early refresh, stale-while-revalidate patterns, and TTL jitter. Symfony Cache's callback-based APIs help structure cache-aside behavior, but you still need to think about invalidation and distributed coordination.

11.10 Security questions a senior candidate should answer cleanly

Authentication vs authorization? Authentication establishes identity; authorization decides whether that identity may perform a specific action. Firewalls/authenticators/providers handle the former; roles/access-control/Voters commonly handle the latter.

Why Voters instead of `if ($user->getId() === ...)` in controllers? Reuse, testability, consistent policy, object-level decisions and separation of concerns.

How do you prevent NoSQL injection? Validate DTOs, whitelist supported filters/operators, bind/build queries through APIs rather than accepting raw Mongo operators from user JSON, enforce authorization at the data/action boundary, and use least-privilege DB credentials.

JWT solves security? No. JWT is a token format. You still need secure key handling, expiry, issuer/audience validation, revocation/rotation strategy, transport over TLS, XSS/CSRF-aware client storage choices and authorization checks.

11.11 Zero-downtime upgrades from an older Symfony codebase

For your background, this answer can be especially strong:

1. Establish a green test/CI baseline first.
2. Upgrade PHP/framework within the supported compatibility path rather than attempting one giant jump.
3. Move to the newest patch of the current major/minor and remove deprecations before the next major.
4. Use Symfony's deprecation tooling/PHPUnit Bridge where applicable to reveal deprecated usage.
5. Upgrade bundles/libraries and replace abandoned dependencies.
6. Modernize incrementally: service wiring → autowiring where it helps; annotations → attributes; Guard-era auth → modern authenticator system; controller/business-logic separation; stronger types.
7. Deploy with telemetry and a rollback plan; separate high-risk data migrations from framework upgrade mechanics.

Your interview phrasing: "Because I've worked with 2.8, 4 and 5, I don't treat modernization as a rewrite. I first stabilize behavior with tests, remove deprecations, upgrade in supported increments, then refactor toward modern idioms. That reduces the number of variables changing at once."

11.12 Twenty final senior Q&As

1. Autowiring vs autoconfiguration?

Autowiring resolves **constructor/method dependencies** from type hints. Autoconfiguration adds **tags/configuration based on implemented interfaces/base classes/attributes** so Symfony knows a service is a command, event subscriber, message handler, etc.

2. Why are services private by default?

It discourages container-as-service-locator code. Dependencies should normally be explicit through injection, which improves design and testability.

3. What if two services implement the same interface?

Autowiring becomes ambiguous unless there is an alias/binding/targeted injection. Configure the desired implementation explicitly, use named aliases/`#[Target]`, or inject a tagged collection/registry if multiple are genuinely needed.

4. Event listener vs subscriber?

Both react to events. A subscriber declares its event map in the class and is self-describing/reusable; a listener can be wired externally and is flexible for a small number of handlers. Choose consistency/discoverability, not dogma.

5. `kernel.request` vs `kernel.controller` vs `kernel.response`?

`kernel.request` runs early and can even create an early response; `kernel.controller` runs after a controller is resolved and can inspect/change it; `kernel.response` runs after a Response exists and is suited to headers/cookies/response decoration.

6. Why keep controllers thin?

Controllers are transport adapters. Business behavior in services/domain objects is reusable from HTTP, CLI and Messenger and can be unit-tested without booting the framework.

7. Repository pattern with Doctrine — redundant?

Doctrine already has repositories, but your own domain-facing repository interface can be valuable when you want persistence-independent use cases, explicit domain queries and easier unit tests. Don't wrap Doctrine with meaningless one-line CRUD methods everywhere.

8. `persist()` vs `flush()`?

`persist()` tells Doctrine to manage/schedule a new object; `flush()` synchronizes the Unit of Work's pending changes to the database. Calling `flush()` inside every tiny loop is usually inefficient — choose transaction/unit-of-work boundaries deliberately.

9. Mongo embed vs reference?

Embed when child data is bounded and belongs/read together with the parent aggregate. Reference when it is shared, unbounded, independently queried/updated, or has a separate lifecycle. Start from access patterns and consistency boundaries, not relational habits.

10. Mongo transaction vs document redesign?

Use transactions when a real invariant spans documents. But Mongo's strongest/simple guarantee is single-document atomicity, so if a natural aggregate can safely contain related state in one document, that may be simpler and faster than coordinating many documents.

11. How do you diagnose a slow Mongo query?

Capture the actual filter/sort/projection, run `explain`, look for collection scans and keys/docs examined, verify compound-index prefix/order/selectivity, check whether sort is covered, then test with production-like cardinality.

12. What should a unit test mock?

External collaborators at the class boundary (repository/gateway/bus) when you are testing orchestration. Don't mock the class under test; don't mock simple value objects; use integration tests when the framework mapping/query/config itself is what you need confidence in.

13. What makes a test brittle?

Asserting implementation details, excessive mocks, depending on ordering that isn't part of the contract, shared mutable fixtures, real clock/network randomness and huge all-purpose tests.

14. 100% code coverage = good suite?

No. Coverage only says lines/branches executed. Prioritize critical behavior, failure paths, security rules and meaningful assertions. Mutation testing or strong behavioral tests reveal more than chasing a percentage alone.

15. How do you handle a production incident?

Protect users first: assess impact, stop/rollback/feature-flag when appropriate, inspect metrics/logs/traces using correlation IDs, mitigate, communicate status, then root-cause and add a durable prevention/action item. Avoid making five speculative changes at once.

16. How do you review code as a senior?

Correctness/security first, then data/query behavior, tests, architecture/maintainability, observability and readability. Explain the *reason* for blocking feedback and distinguish blocking issues from suggestions/nits.

17. When would you reject a design pattern?

When it adds indirection without a changing axis/problem. Patterns are vocabulary for recurring design forces, not a scorecard. Three explicit lines of code can be better than a Strategy/Factory hierarchy that will never vary.

18. What makes an API scalable?

Stateless horizontal scaling where practical, bounded work per request, indexed data access, pagination, cache where correctness allows, async offloading of non-critical slow work, connection/resource limits, backpressure, observability and idempotent failure handling.

19. Docker vs Kubernetes in one sentence?

Docker packages/runs the application in containers; Kubernetes orchestrates container deployment, scheduling, health, scaling, networking and rollout across a cluster. Don't claim deep Kubernetes experience if you only consumed manifests/pipelines.

20. What should you ask the interviewer about this exact role?

"Which Symfony/PHP versions are in production, how is MongoDB modeled and scaled, what does the test pyramid look like, what is the biggest reliability/performance pain today, how is on-call run, and what technical ownership do you expect from a senior in the first 90 days?"

11.13 Final 30-minute revision checklist

Be able to explain without notes:

- PHP: interfaces/abstract classes/traits, type system, exceptions, generators, readonly, enums, Composer/PSR-4, SOLID.
- Symfony: lifecycle, container/autowiring/autoconfiguration, routes/controllers, Validator, Security/Voters, EventDispatcher, Messenger, Cache, Console/config.
- MongoDB: embed/reference, indexes/explain, aggregation, single-document atomicity, transactions, optimistic concurrency, schema evolution.
- Testing: one PHPUnit test from memory, mocks vs integration, KernelTestCase/WebTestCase, failure-path tests.
- Patterns: Strategy, Factory, Decorator, Observer, Command, Repository, Unit of Work — with one Symfony example and one overuse warning.
- Backend: HTTP/idempotency, queues/retries, cache, security, CI/CD, Docker/K8s basics, logs/metrics/traces, incident handling.
- Your story: **2.8 / 4 / 5 strongest, some 6; current 7.4/8.1 awareness — no overclaiming.**

Official resources to keep open while preparing

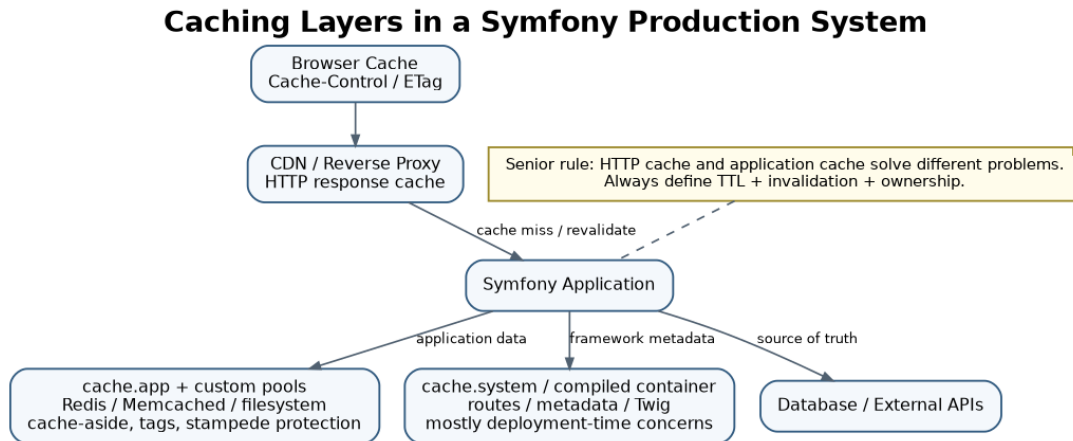
- Symfony current documentation: <https://symfony.com/doc/current/index.html>

- Symfony release/support status: <https://symfony.com/releases>
- Service container/autowiring: https://symfony.com/doc/current/service_container.html
- HttpKernel lifecycle: https://symfony.com/doc/current/components/http_kernel.html
- Routing/controllers: <https://symfony.com/doc/current/routing.html> and <https://symfony.com/doc/current/controller.html>
- Security/custom authenticators/Voters: <https://symfony.com/doc/current/security.html>
- Messenger: <https://symfony.com/doc/current/messenger.html>
- Testing: <https://symfony.com/doc/current/testing.html>
- MakerBundle: <https://symfony.com/bundles/SymfonyMakerBundle/current/index.html>
- Doctrine MongoDB ODM: <https://www.doctrine-project.org/projects/doctrine-mongodb-odm/en/stable/>
- PHP manual / supported versions: <https://www.php.net/manual/en/> and <https://www.php.net/supported-versions.php>
- PHPUnit manual: <https://docs.phpunit.de/>

Part VII — Production Symfony Engineering

Section 12 — Caching in Symfony: framework cache, application cache and HTTP cache

Caching is not one thing. In a senior interview, separate these layers before discussing implementation.



12.1 The three cache categories you should distinguish

Layer	What it caches	Typical backend	Invalidated when	Interview point
Symfony/system cache	container metadata, validation/serializer metadata, routes and other derivable framework state	files/APCu via cache.adapter.system	deployment / code change	Treat as deployment-time/rebuildable state, not business data.
Application data cache	expensive query results, API lookups, computed business data	Redis, Valkey, Memcached, filesystem	TTL, explicit key deletion, tag invalidation	This is where cache.app and custom pools are used.
HTTP/gateway cache	complete HTTP responses	browser, CDN, Symfony HttpCache, Varnish/Nginx/CDN	freshness/validation headers or purge	Can serve a response without executing the controller at all.

Symfony always provides `cache.system` and `cache.app`. A good production design normally leaves the system cache on Symfony's optimized system adapter and moves application data to a shared backend such as Redis when the application runs on more than one instance.

12.2 Redis-backed application cache configuration

```
# config/packages/cache.yaml
framework:
    cache:
        # Keep cache namespace stable across release directories.
        prefix_seed: 'smart-working-symfony-api'

        # Shared application cache.
        app: cache.adapter.redis
        default_redis_provider: '%env(REDIS_URL)%'

    pools:
        cache.catalog:
            adapter: cache.adapter.redis_tag_aware
            default_lifetime: 600
            tags: true

        cache.external_api:
            adapter: cache.adapter.redis
            default_lifetime: 60

# .env.local in development; real environment variables/secrets in production
REDIS_URL=redis://localhost:6379
```

12.3 Cache-aside with Symfony Cache Contracts

The Cache Contracts callback API is usually the cleanest application-level approach because it is concise and includes stampede protection.

```
<?php

namespace App\Service;

use Symfony\Component\DependencyInjection\Attribute\Autowire;
use Symfony\Contracts\Cache\CacheInterface;
use Symfony\Contracts\Cache\ItemInterface;

final class ProductCatalogService
{
    public function __construct(
        #[Autowire(service: 'cache.catalog')]
        private CacheInterface $catalogCache,
        private ProductRepository $products,
    ) {}

    public function getProduct(string $id): array
    {
        return $this->catalogCache->get(
            'product_'. $id,
            function (ItemInterface $item) use ($id): array {
                $item->expiresAfter(600);
                $item->tag(['product', 'product_'. $id]);

                return $this->products->getApiView($id);
            }
        );
    }
}
```

12.4 Tag invalidation

Tag invalidation is useful when many keys belong to the same business concept.

```
use Symfony\Contracts\Cache\TagAwareCacheInterface;

final class ProductCacheInvalidator
{
    public function __construct(
        private TagAwareCacheInterface $cache,
    ) {}

    public function productChanged(string $productId): void
    {
        $this->cache->invalidateTags([
            'product',
            'product_'. $productId,
        ]);
    }
}
```

Senior warning: invalidation must be part of the write path or event flow. A cache without an invalidation strategy is a correctness bug waiting to happen.

12.5 HTTP caching

For public/read-heavy resources, use HTTP semantics instead of re-inventing them in Redis.

```
#[Route('/api/catalog/{id}', methods: ['GET'], format: 'json')]
public function show(string $id, ProductCatalogService $catalog): JsonResponse
{
    $data = $catalog->getProduct($id);

    $response = $this->json(['data' => $data]);
    $response->setPublic();
    $response->setMaxAge(60);
    $response->setSharedMaxAge(300);
    $response->setEtag(hash('xxh128', json_encode($data, JSON_THROW_ON_ERROR)));

    return $response;
}
```

Use Cache-Control, ETag and Last-Modified correctly. A reverse proxy/CDN can then serve cached responses before PHP/Symfony runs.

12.6 Cache stampede / dogpile problem

If a popular key expires and 500 requests arrive simultaneously, all 500 must not recompute the same expensive value. Symfony's Cache Contracts are designed to protect against this pattern. Also consider soft-expiry/beta recomputation for particularly hot data.

12.7 Cache interview Q&A

Q: cache.system vs cache.app?

cache.system holds rebuildable framework metadata tied mainly to source/deployment.

cache.app is runtime application data and can survive deployments. On horizontally scaled systems, cache.app should normally use shared storage.

Q: Redis vs HTTP cache?

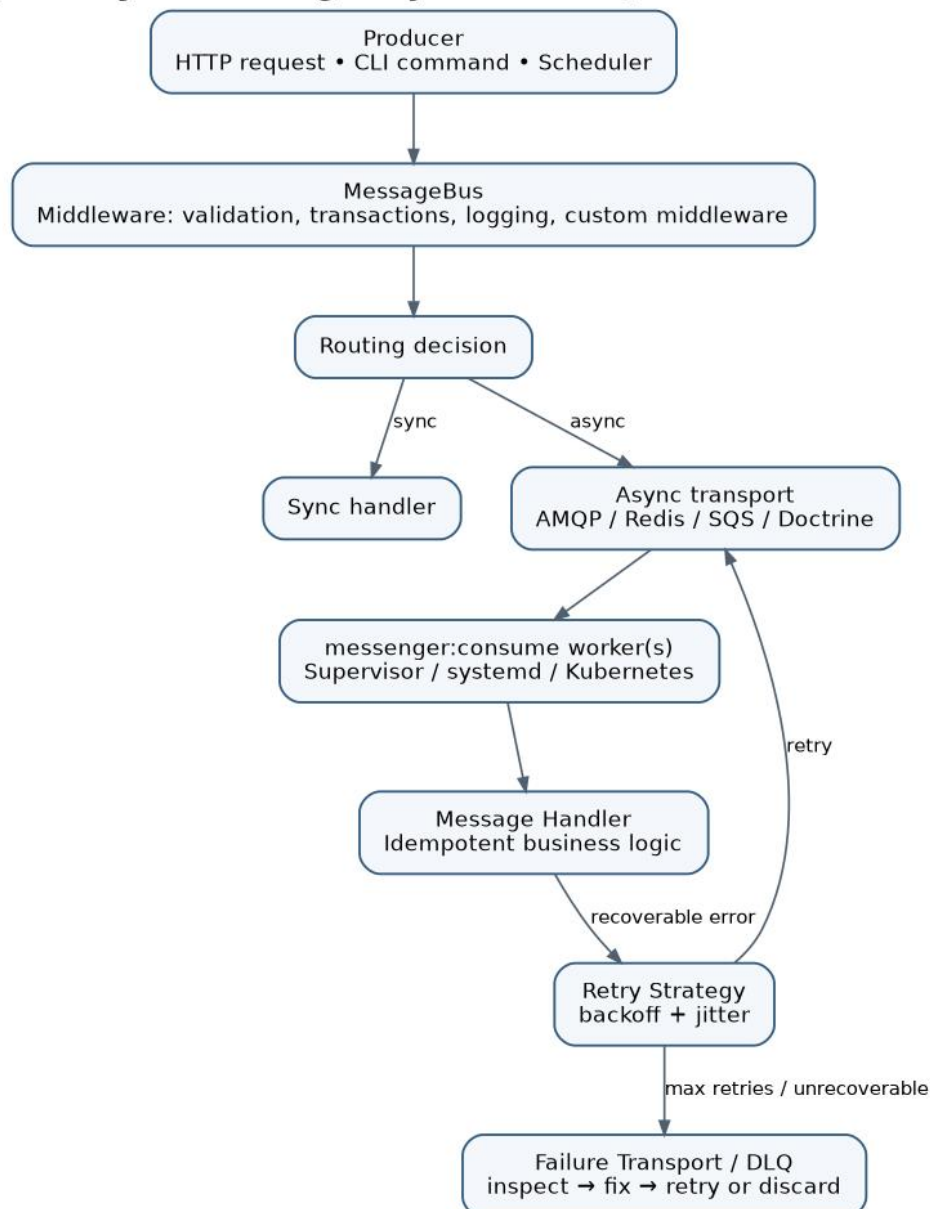
Redis application cache still executes Symfony and returns data to your code. HTTP cache can answer the complete request at a browser/CDN/reverse-proxy layer without executing the application.

Q: What makes caching dangerous?

Stale data, cache-key collisions, accidental caching of user-specific/private data, stampedes, deployment namespace changes, and invalidation bugs.

Section 13 — Jobs, Queues, Messenger and Scheduler

Symfony Messenger: Jobs and Queue Processing



13.1 What Symfony Messenger solves

Messenger decouples the code that **requests work** from the code that **performs work**. A message can run synchronously or be routed to a transport and processed by one or more workers.

Good async candidates:

- emails and notifications;
- PDF/report/export generation;
- image/video processing;
- webhook delivery;
- long third-party API operations;
- search indexing;
- large imports;
- audit/event integrations;
- background synchronization.

Do **not** put a job on a queue just because queues sound scalable. If the caller must know the result before the HTTP response can be correct, the operation may need to remain synchronous.

13.2 Message and handler

```
<?php
namespace App\Message;

final readonly class GenerateInvoicePdf
{
    public function __construct(
        public string $invoiceId,
        public string $requestedBy,
    ) {}
}

<?php
namespace App\MessageHandler;

use App\Message\GenerateInvoicePdf;
use Symfony\Component\Messenger\Attribute\AsMessageHandler;

#[AsMessageHandler]
final class GenerateInvoicePdfHandler
{
    public function __construct(
        private InvoiceRepository $invoices,
        private PdfGenerator $pdf,
        private ObjectStorage $storage,
    ) {}

    public function __invoke(GenerateInvoicePdf $message): void
    {
        // Must be safe if the same message is delivered more than once.
        if ($this->storage->invoicePdfExists($message->invoiceId)) {
```

```

        return;
    }

    $invoice = $this->invoices->get($message->invoiceId);
    $binary = $this->pdf->generate($invoice);
    $this->storage->storeInvoicePdf($message->invoiceId, $binary);
}
}

```

13.3 Production Messenger configuration

```

# config/packages/messenger.yaml
framework:
    messenger:
        failure_transport: failed

        transports:
            async:
                dsn: '%env(MESSENGER_TRANSPORT_DSN)%'
                retry_strategy:
                    max_retries: 5
                    delay: 1000
                    multiplier: 2
                    max_delay: 60000
                    jitter: 0.1

            failed:
                dsn: 'doctrine://default?queue_name=failed'

        routing:
            'App\Message\GenerateInvoicePdf': async
            'App\Message\SendCustomerEmail': async

```

Common transport choices:

Transport	When it fits	Trade-offs
Doctrine transport	simple system, low-to-moderate volume, already has SQL	easiest operationally; DB becomes queue load.
Redis	fast existing infrastructure, simple workloads	understand Redis durability and consumer semantics.
RabbitMQ/AMQP	complex routing, mature messaging, high-volume event/job systems	extra infrastructure and operational expertise.
Amazon SQS	AWS-native managed queue	managed reliability; cloud-specific semantics/cost.

13.4 Worker operations

```

# Local/debug
php bin/console messenger:consume async -vv

# bounded production worker: process manager will restart it
php bin/console messenger:consume async \
    --time-limit=3600 \

```

```
--memory-limit=256M \
--failure-limit=10

# inspect queue counts if the transport supports it
php bin/console messenger:stats

# failed jobs
php bin/console messenger:failed:show
php bin/console messenger:failed:retry

# graceful code-deploy restart
php bin/console messenger:stop-workers
```

Why bounded workers? Symfony/PHP services, Doctrine managers, SDKs and application code can accumulate state or memory in a long-running process. Recycling workers is normal production hygiene.

13.5 Supervisor example

```
; /etc/supervisor/conf.d/symfony-messenger.conf
[program:symfony-messenger]
directory=/srv/app/current
command=php bin/console messenger:consume async --time-limit=3600 --memory-
limit=256M
user=www-data
numprocs=4
autostart=true
autorestart=true
startsecs=0
startretries=10
stopwaitsecs=30
process_name=%(program_name)s_%(process_num)02d
```

In Kubernetes, web and worker processes should normally be separate Deployments so they can scale independently.

13.6 Failure classification

- **Transient:** remote service timeout, temporary network problem, rate-limit response → retry with backoff.
- **Permanent:** invalid business state or permanently invalid payload → do not burn retry capacity forever.
- **Poison message:** repeatedly crashes the handler → move to failure transport and investigate.

Use `RecoverableMessageHandlingException` or `UnrecoverableMessageHandlingException` when you can classify failure explicitly.

13.7 Idempotency: assume duplicate delivery

Queues are generally designed around at-least-once delivery. A worker might perform the side effect and crash before the broker receives acknowledgement. Design handlers so retrying does not double-charge, double-send or corrupt state.

Good strategies:

- unique business operation key;
- database unique index;
- processed-event/inbox table/document;
- natural idempotency (upsert, set desired state instead of increment blindly);
- transactional outbox for reliable DB-write + event publication.

13.8 Priorities and queue isolation

Do not mix every message type into one queue. Payment confirmations, user emails and 30-minute catalog imports have different latency requirements. Separate transports/worker pools prevent a slow workload from starving a critical one.

13.9 Scheduler: recurring jobs, not a replacement for Messenger

Symfony Scheduler defines recurring work in application code rather than scattered server crontabs. Scheduler uses Messenger under the hood.

```
<?php
namespace App\Scheduler;

use Symfony\Component\Scheduler\Attribute\AsCronTask;

#[AsCronTask('0 2 * * *', transports: 'async')]
final class RebuildDailyReport
{
    public function __invoke(): void
    {
        // Keep this small or redispatch actual heavy work to Messenger.
    }
}

php bin/console debug:scheduler
php bin/console messenger:consume scheduler_default
```

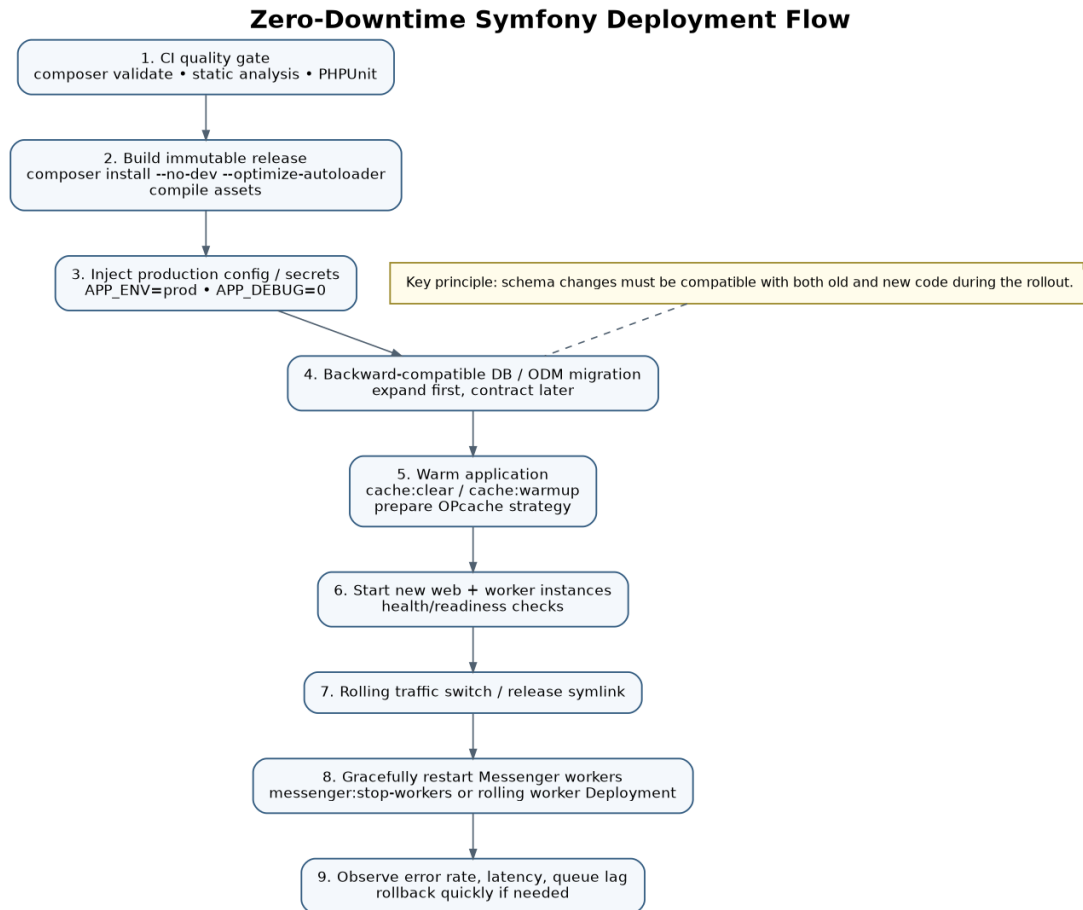
For multiple scheduler workers, use a lock to avoid duplicate schedule generation. For high throughput, have the scheduler publish to an async transport and scale normal Messenger workers.

13.10 Scheduler vs cron

Symfony Scheduler	OS/Kubernetes cron
schedule lives with application code and DI	infrastructure owns the schedule
easy to unit test services	good for very simple one-command triggers
uses Messenger worker/retry ecosystem	independent from application worker runtime
supports locks/stateful schedules	relies on external orchestration behavior

A senior answer is not “Scheduler is always better.” Choose based on ownership and operational architecture.

Section 14 — Symfony-Specific Production Deployment and Operations



14.1 What a production deploy must accomplish

A Symfony deployment is more than copying PHP files. A reliable release should make **code, dependencies, configuration, schema, caches, long-running workers and traffic** move together safely.

14.2 Recommended production sequence

```

# CI / build stage
composer validate --strict
vendor/bin/phpstan analyse
vendor/bin/phpunit

# create immutable release
composer install --no-dev --prefer-dist --optimize-autoloader --classmap-authoritative

# production environment
export APP_ENV=prod

```

```
export APP_DEBUG=0

# optional optimized dotenv representation when .env files are used
composer dump-env prod

# backward-compatible schema/data changes
php bin/console doctrine:migrations:migrate --no-interaction

# build/warm framework state BEFORE receiving traffic
php bin/console cache:clear --env=prod --no-debug
php bin/console cache:warmup --env=prod --no-debug

# assets if applicable
# npm ci && npm run build
# php bin/console asset-map:compile

# gracefully make workers exit and restart on current code
php bin/console messenger:stop-workers
```

14.3 Zero-downtime rule: database changes must work with old and new code

Use **expand/contract** migrations.

Example: rename customerName to customerDisplayName:

1. add new field/column, keep old one;
2. deploy code that can read both and writes both;
3. backfill old records;
4. deploy code reading only the new field;
5. later remove the old field.

Never make a destructive schema change first while old application instances are still receiving requests.

14.4 Release-directory / symlink deployment

A classic VM approach:

```
/srv/app/
  releases/
    2026-09-08-150000/
    2026-09-07-221500/
  shared/
    uploads/
  current -> releases/2026-09-08-150000/
```

Build and warm the new release offline, then atomically switch current. Keep several old releases for rollback. When application cache should survive these directories, configure a stable `framework.cache.prefix_seed` and use a shared backend for runtime cache.

14.5 PHP-FPM and OPcache

- PHP-FPM web workers are short-request oriented; Messenger workers are long-running processes.
- Enable OPcache in production.
- Decide how deployment refreshes OPcache: process reload, container replacement or platform-managed strategy.
- Do not rely on production workers noticing code-file changes magically.

14.6 Reverse proxy / load balancer

When Symfony sits behind Nginx, an AWS load balancer, Cloudflare, Varnish or Kubernetes ingress, configure trusted proxies/headers correctly or Symfony can misread the client IP, scheme and host.

```
# config/packages/framework.yaml
framework:
    trusted_proxies: '%env(TRUSTED_PROXIES)%'
    trusted_headers:
        - 'x-forwarded-for'
        - 'x-forwarded-host'
        - 'x-forwarded-proto'
        - 'x-forwarded-port'
        - 'x-forwarded-prefix'
```

Restrict trusted proxies to infrastructure you actually control. Blindly trusting X-Forwarded-* from the public internet is a security mistake.

14.7 Health and readiness endpoints

A Kubernetes/load-balancer health endpoint should be cheap and intentional.

- **Liveness:** is the process healthy enough to remain running?
- **Readiness:** should this instance receive traffic now?
- Do not turn every liveness check into an expensive full dependency audit.
- If readiness verifies DB/cache, use strict timeouts.

14.8 Docker image principles

```
# simplified example
FROM php:8.4-fpm-alpine

WORKDIR /app
COPY --from=composer:2 /usr/bin/composer /usr/bin/composer

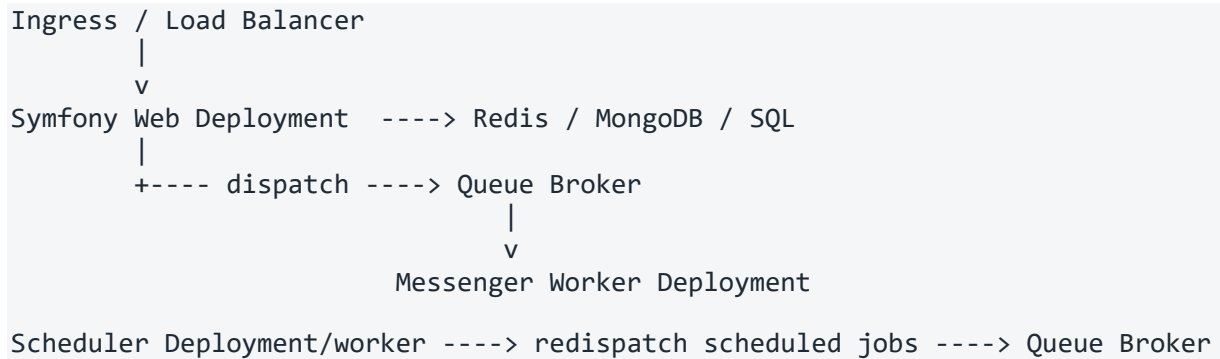
COPY composer.json composer.lock ./
RUN composer install --no-dev --prefer-dist --no-scripts --optimize-autoloader

COPY . .
RUN composer dump-autoload --classmap-authoritative \
    && APP_ENV=prod APP_DEBUG=0 php bin/console cache:warmup

CMD ["php-fpm"]
```

For a real system use a multi-stage image, required PHP extensions, non-root execution where practical, a proper web server/FrankenPHP/Nginx topology, health checks and image vulnerability scanning.

14.9 Kubernetes topology



Scale web pods by HTTP pressure and worker pods by queue lag/processing time rather than forcing them to share one replica count.

14.10 Deployment rollback

A rollback plan is part of deployment design, not an afterthought.

- immutable previous application release/image;
- backward-compatible database migration;
- feature flag for dangerous changes;
- monitoring after rollout;
- known command/procedure to restore traffic;
- avoid data migrations that cannot be safely reversed without a plan.

14.11 Production monitoring signals

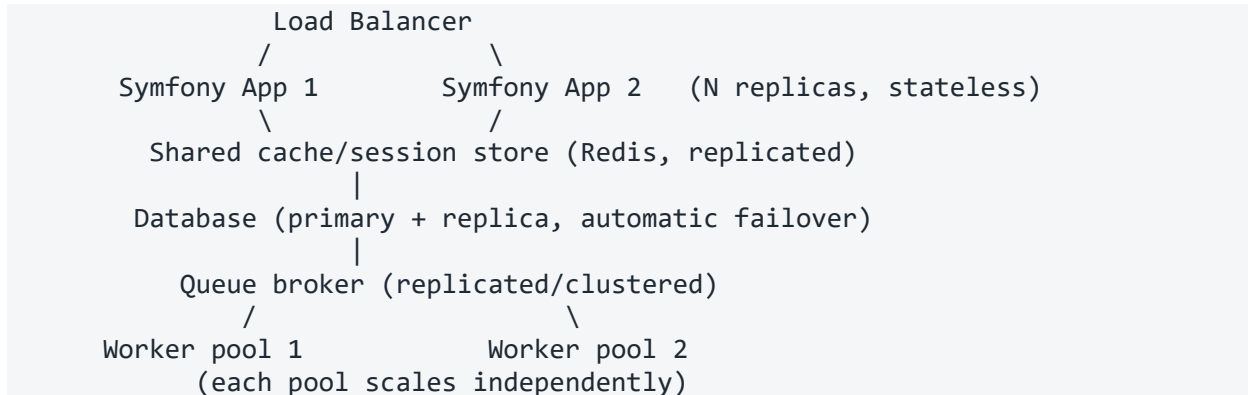
For this JD's on-call requirement, know what you would watch:

- HTTP p50/p95/p99 latency;
- 4xx/5xx rate;
- PHP-FPM saturation;
- CPU and memory;
- database latency/slow queries;
- cache hit ratio;
- queue depth/lag, retry and failed message count;
- external dependency latency/errors;
- deployment version/commit in logs/traces;
- correlation/request IDs.

14.12 Designing for High Availability & Mission-Critical Reliability

The JD's language is unusually strong for an “About the Role” section — “mission-critical backend systems,” reliability and accountability described as “non-negotiable” — and lists high-availability experience as a nice-to-have. Sections 6.14 (resilience) and 14.6-14.11 (deployment/monitoring) already cover the individual pieces; this section connects them into one architecture answer.

A stateless, no-single-point-of-failure Symfony topology



Each element, and why it removes a single point of failure:

- **Stateless app nodes:** no session or in-process state lives on a single server, so the load balancer can send a request to any instance and kill/replace any instance without losing user data (ties to 14.9's Kubernetes replica/Deployment model).
- **Shared session/cache in Redis** rather than local memory: without this, losing one app server logs users out or drops their cart — exactly the class of failure the JD's “mission-critical” language is about avoiding.
- **Database replication with automatic failover:** a replica set/cluster survives losing the primary node; the trade-off is replication lag, which matters for read-after-write consistency (Section 3.1, 6.8).
- **Redundant, clustered queue infrastructure:** a single-broker queue is a hidden single point of failure for anything async (Section 6.3, 13); workers should also run in independently scalable pools so a slow workload doesn't starve a critical one (13.8's queue isolation).
- **Health/readiness probes (14.7) and autoscaling:** the load balancer and orchestrator need to know an instance is actually healthy before routing to it, not just that the process is running.
- **Timeouts, retries with backoff, and circuit breakers (6.14):** prevent one slow downstream dependency from exhausting every PHP-FPM worker across every app node at once.
- **Graceful degradation:** a non-critical dependency failing (e.g. a recommendations service) should degrade that one feature, not fail the whole request.
- **Zero-downtime deploys (14.3-14.4) and rollback (14.10):** availability during normal operations is only half the story — releases are a routine, frequent risk to reliability, and should be boring by design.

- **Monitoring/SLOs (14.11) and a defined disaster-recovery target:** knowing you're unhealthy before customers report it, and having an agreed RPO/RTO for how much data loss and downtime is acceptable in the worst case.

Q1. What would you do to make a Symfony application highly available?

A: Start from removing state from the app tier — no local sessions or in-process caches, everything shared in Redis — so any number of identical app instances can sit behind a load balancer with none of them special. Then remove single points of failure one layer down: a database with a replica and automatic failover, a clustered queue broker instead of one box, and worker pools that scale independently of the web tier. Wrap external calls in timeouts/retries/circuit breakers so one slow dependency can't cascade, deploy with the zero-downtime/rollback pattern from Section 14, and monitor against defined SLOs so degradation is caught before it becomes an outage.

Q2. What's the difference between RPO and RTO, and how do they shape your backup strategy?

A: RPO (Recovery Point Objective) is how much data you can afford to lose, measured in time — an RPO of 5 minutes means backups/replication must happen at least that often. RTO (Recovery Time Objective) is how long you can afford to be down while recovering. A tight RPO drives continuous replication rather than nightly backups; a tight RTO drives having a warm/hot standby ready to take traffic rather than a cold backup you'd need hours to restore and reprovision.

Q3. How do you avoid a single point of failure in the session layer once you have more than one app server?

A: Don't store sessions in local files or in-process memory on the app server at all — store them in a shared, replicated store (Redis is the common Symfony choice) that every app instance reads and writes to, so a user's session survives their next request landing on a different instance, and losing one app server doesn't log anyone out. The Redis instance itself then needs its own redundancy (replica/sentinel or a managed cluster) or it just becomes the new single point of failure one layer down.

Q4. What's a circuit breaker, and how is it different from a retry-with-backoff?

A: A retry-with-backoff keeps trying a failing call, with increasing delay, hoping it was a transient blip — useful for brief hiccups, but if the dependency is actually down, retries just pile up load on an already-struggling service and tie up your own workers waiting. A circuit breaker tracks failure rate and, once it crosses a threshold, “opens” and fails fast without even attempting the call for a cooldown period — protecting your own system's resources and giving the failing dependency room to recover, at the cost of temporarily refusing calls that might have succeeded.

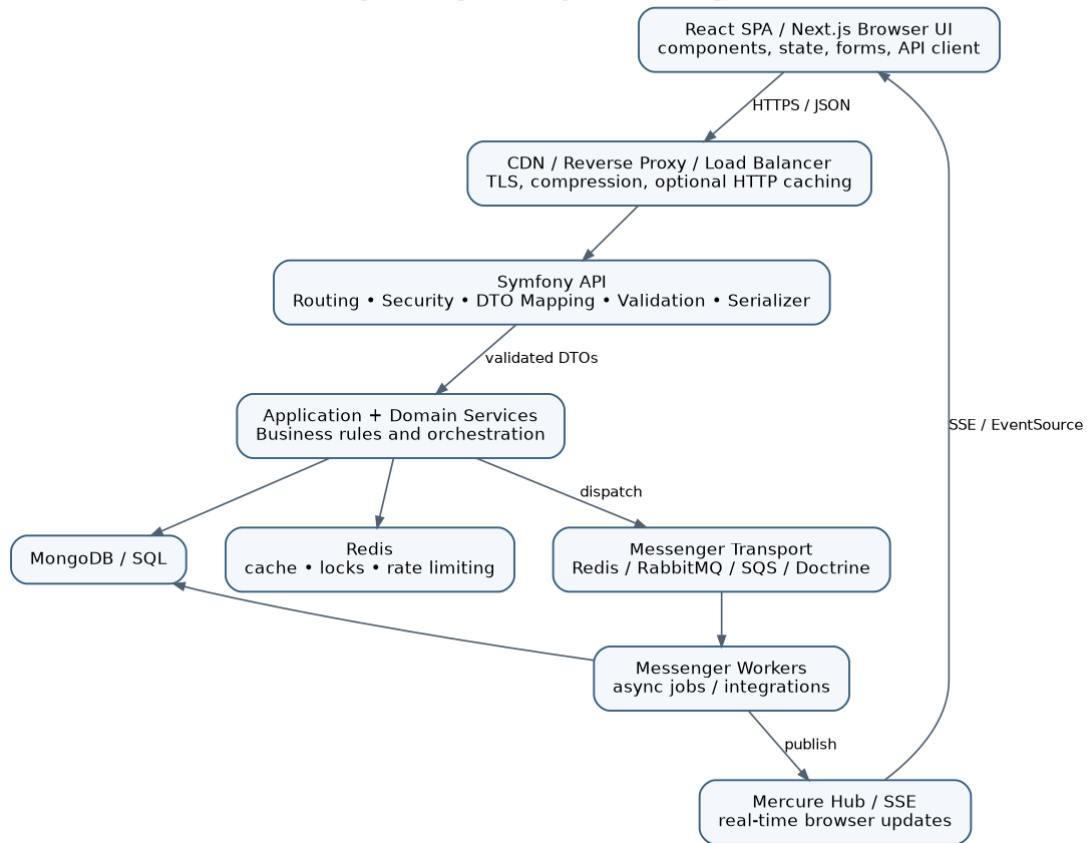
Q5. How would you design graceful degradation for a feature that depends on a flaky third-party API?

A: Wrap the call with a timeout and circuit breaker so a slow/failing dependency can't stall or exhaust the request path, and define an explicit fallback for when it's unavailable — cached/stale data, a default value, or simply hiding that one section of the response — rather than letting the whole request fail because one non-critical dependency did. The key design question to answer up front is which features are “must-have” (failure should fail the request) versus “nice-to-have” (failure should degrade silently), and that classification should be a deliberate decision, not an accident of how the code happened to be written.

Part VIII — Symfony with React / Next.js

Section 15 — Using Symfony as an API-Only Backend for React

React / Next.js + Symfony API-Only Architecture



15.1 Recommended separation of responsibilities

React/Next.js owns: UI, components, routing in the SPA, browser state, forms, frontend validation and calling the API.

Symfony owns: authentication/authorization, authoritative validation, business rules, persistence, transactions, messaging, caching, integrations and API response contracts.

The frontend can validate for UX; the Symfony backend must validate again because clients are untrusted.

15.2 API request flow

```

React form
-> POST /api/orders JSON
-> CORS / reverse proxy
-> Symfony route
-> authentication
-> #[MapRequestPayload] DTO
-> Validator
-> controller
-> application service
  
```

```
-> repository / MongoDB
-> Messenger event/job
-> Serializer / JsonResponse
-> React updates UI
```

15.3 DTO + validation

```
<?php

namespace App\Api\Dto;

use Symfony\Component\Validator\Constraints as Assert;

final readonly class CreateOrderRequest
{
    public function __construct(
        #[Assert\NotBlank]
        public string $productId,

        #[Assert\Positive]
        public int $quantity,

        #[Assert\Email]
        public string $customerEmail,
    ) {}
}
```

15.4 API controller using modern payload mapping

```
<?php

namespace App\Controller\Api;

use App\Api\Dto\CreateOrderRequest;
use App\Service\OrderApplicationService;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\JsonResponse;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\HttpKernel\Attribute\MapRequestPayload;
use Symfony\Component\Routing\Attribute\Route;

final class OrderController extends AbstractController
{
    #[Route('/api/orders', methods: ['POST'], format: 'json')]
    public function create(
        #[MapRequestPayload] CreateOrderRequest $request,
        OrderApplicationService $orders,
    ): JsonResponse {
        $order = $orders->create($request);

        return $this->json(
            [
                'data' => [
                    'id' => $order->id(),
                    'status' => $order->status(),
                ],
            ],
            Response::HTTP_CREATED,
        );
    }
}
```

`#[MapRequestPayload]` deserializes request data into the DTO and integrates with validation. For a JSON API, declaring the route format as JSON helps Symfony return API-appropriate errors rather than HTML pages.

15.5 React call

```
export async function createOrder(payload: {
  productId: string;
  quantity: number;
  customerEmail: string;
}) {
  const response = await fetch(`${import.meta.env.VITE_API_URL}/api/orders`, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      Authorization: `Bearer ${getAccessToken()}`,
    },
    body: JSON.stringify(payload),
  });

  if (!response.ok) {
    const error = await response.json();
    throw new Error(error.detail ?? 'Order creation failed');
  }

  return response.json();
}
```

15.6 CORS configuration

For a separately hosted browser frontend, CORS must allow only the origins and methods you need.

```
composer require nelmio/cors-bundle
# config/packages/nelmio_cors.yaml
nelmio_cors:
  defaults:
    origin_regex: true
    allow_origin: ['%env(CORS_ALLOW_ORIGIN)%']
    allow_methods: ['GET', 'OPTIONS', 'POST', 'PUT', 'PATCH', 'DELETE']
    allow_headers: ['Content-Type', 'Authorization']
    expose_headers: ['Link']
    max_age: 3600
  paths:
    '^/api/': ~

CORS_ALLOW_ORIGIN='^https://(www\.)?frontend\.example\.com$'
```

Do not use `*` with credentials. CORS is a browser policy; it is not authentication or authorization.

15.7 Authentication options for React

Option A — Bearer JWT

A common stateless API approach is `lexik/jwt-authentication-bundle`.

```
composer require lexik/jwt-authentication-bundle
php bin/console lexik:jwt:generate-keypair
```

```
# config/packages/lexik_jwt_authentication.yaml
lexik_jwt_authentication:
    secret_key: '%env(resolve:JWT_SECRET_KEY)%'
    public_key: '%env(resolve:JWT_PUBLIC_KEY)%'
    pass_phrase: '%env(JWT_PASSPHRASE)%'
    token_ttl: 3600

# config/packages/security.yaml - simplified
security:
    password_hashers:
        Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface:
            'auto'

    providers:
        app_user_provider:
            entity:
                class: App\Entity\User
                property: email

    firewalls:
        login:
            pattern: ^/api/login
            stateless: true
            json_login:
                check_path: /api/login
                username_path: email
                password_path: password
                success_handler:
                    lexik_jwt_authentication.handler.authentication_success
                failure_handler:
                    lexik_jwt_authentication.handler.authentication_failure

    api:
        pattern: ^/api
        stateless: true
        jwt: ~
```

Security nuance: storing long-lived tokens in browser local storage increases XSS impact. For higher-security systems, many teams prefer secure/httpOnly cookie designs, BFF patterns or an external OIDC provider. Cookie-based authentication brings CSRF considerations back into scope.

Option B — OAuth2/OIDC provider

For enterprise products, it may be preferable for Symfony to validate access tokens from Auth0, Okta, Azure AD/Entra ID, Keycloak or another identity provider rather than owning credentials itself.

15.8 Authorization stays in Symfony

React may hide an Edit button, but Symfony must still enforce access.

```
#[Route('/api/orders/{id}', methods: ['PATCH'], format: 'json')]
public function update(Order $order): JsonResponse
{
    $this->denyAccessUnlessGranted('EDIT', $order);

    // ...
}
```

Use Voters when authorization depends on the current user + a domain object.

15.9 Serialization and stable API contracts

Avoid returning persistence objects blindly. Use DTOs, serialization groups or explicit response models so your database structure does not accidentally become your public API.

```
use Symfony\Component\Serializer\Annotation\Groups;

final class OrderView
{
    public function __construct(
        #[Groups(['order:read'])]
        public readonly string $id,

        #[Groups(['order:read'])]
        public readonly string $status,
    ) {}
}
```

15.10 API Platform — when it helps

API Platform is a Symfony-based API framework that can give you resource metadata, REST/GraphQL endpoints, OpenAPI docs, filters, pagination, validation integration and client generation. It is excellent when your domain maps well to resource-driven APIs; hand-written controllers/services may be clearer for unusual workflows and highly customized use cases.

A senior answer should evaluate it rather than say “API Platform is always better.”

15.11 OpenAPI / Swagger

Two common routes:

- API Platform: OpenAPI generated from API resources;
- nelmio/api-doc-bundle: document a custom-controller API and expose Swagger/OpenAPI docs.

Use OpenAPI to generate typed TypeScript clients and reduce frontend/backend contract drift.

15.12 Real-time React updates with Mercure

Mercure uses Server-Sent Events (SSE) and is a Symfony-native option for real-time updates.

```
composer require mercure

MERCURE_URL=http://mercure/.well-known/mercure
MERCURE_PUBLIC_URL=https://api.example.com/.well-known/mercure
MERCURE_JWT_SECRET=change-me
```

React/browser:

```
useEffect(() => {
    const source = new EventSource(mercureUrl);

    source.onmessage = (event) => {
        const update = JSON.parse(event.data);
```

```
// update React state / query cache
};

return () => source.close();
}, [mercureUrl]);
```

Use Mercure for live dashboards, notification badges, long-job completion and collaborative updates. For a very small internal use case, native Symfony SSE may be enough.

15.13 File uploads

Use multipart/form-data; validate size, MIME/type and business constraints server-side; store large binaries in object storage rather than MongoDB unless GridFS is a deliberate requirement.

15.14 React/Next.js SSR nuance

With Next.js, distinguish:

- **browser → Symfony API:** CORS applies;
- **Next.js server → Symfony API:** server-to-server call, browser CORS does not apply.

This distinction is useful when designing internal service URLs, auth propagation and caching.

15.15 Alternative: Symfony + React islands

A full SPA is not mandatory. You can render most HTML with Twig and mount React only where interactivity is complex.

```
{# templates/dashboard.html.twig #}
<div
  id="analytics-widget"
  data-props='{ { widgetProps|json_encode|e('html_attr') } }'
></div>

import { createRoot } from 'react-dom/client';
import { AnalyticsWidget } from './AnalyticsWidget';

const element = document.getElementById('analytics-widget');
if (element) {
  createRoot(element).render(
    <AnalyticsWidget {...JSON.parse(element.dataset.props ?? '{}')} />,
  );
}
```

For simple interactivity, also know Symfony UX/Stimulus/Turbo as a lower-complexity alternative.

Part IX — Package Ecosystem and Configuration

Section 16 — Widely Used Symfony Packages and What They Are For

Symfony is intentionally modular. A senior engineer should know what belongs to core Symfony, what comes from Doctrine, and what is a common ecosystem bundle.

16.1 Core framework and HTTP

Package	Purpose	Typical configuration / note
symfony/framework-bundle	connects Symfony components into the full framework	framework: in config/packages/framework.yaml
symfony/http-foundation	object-oriented Request/Response abstraction	usually indirect via FrameworkBundle
symfony/http-kernel	request handling/kernel lifecycle	listeners/subscribers, kernel events
symfony/routing	route matching and URL generation	attributes or config/routes.yaml
symfony/dependency-injection	service container, autowiring, compiler passes	config/services.yaml
symfony/config / symfony/yaml	configuration loading/processing	widely used by bundles and custom configuration

16.2 API, validation and serialization

Package	Purpose	Important config/concept
symfony/serializer-pack	object ↔ JSON/XML/CSV serialization ecosystem	groups, contexts, normalizers
symfony/validator	declarative input/domain validation	PHP attributes, custom Constraints
api-platform/core / composer require api	full API framework on Symfony	#[ApiResource], OpenAPI, filters, pagination
nelmio/api-doc-bundle	OpenAPI/Swagger documentation for custom APIs	route/documentation config
nelmio/cors-bundle	CORS response handling	allowed origins/methods/headers per path

16.3 Authentication and authorization

Package	Purpose	Important config/concept
symfony/security-bundle	authentication, firewalls, access control, voters	security.yaml, providers, authenticators
symfony/password-hasher	secure password hashing	password_hashers config
symfony/rate-limiter	login/API/business throttling	fixed/sliding/token bucket policies
lexik/jwt-authentication-bundle	common JWT implementation for stateless APIs	key pair, TTL, JWT firewall

16.4 Persistence

Package	Purpose	Important config/concept
doctrine/orm	relational ORM	EntityManager, mappings, DQL
doctrine/doctrine-bundle	Doctrine ORM integration with Symfony	DBAL/ORM connections in doctrine.yaml
doctrine/doctrine-migrations-bundle	versioned relational schema migrations	doctrine:migrations:migrate
doctrine/mongodb-odm-bundle	MongoDB ODM integration	DocumentManager, document mappings, indexes

Example MongoDB ODM configuration:

```
# config/packages/doctrine_mongodb.yaml
doctrine_mongodb:
  connections:
    default:
      server: '%env(resolve:MONGODB_URL)%'
  default_database: '%env(resolve:MONGODB_DB)%'
  document_managers:
    default:
      auto_mapping: true
```

16.5 Async, scheduled and concurrent work

Package	Purpose	Important config/concept
symfony/messenger	sync/async message bus and workers	transports, routing, retry, failure transport
symfony/scheduler	recurring tasks built on Messenger	#[AsCronTask], #[AsPeriodicTask], schedule workers
symfony/lock	distributed/process locks	Redis/DB/filesystem stores; prevent overlap

Package	Purpose	Important config/concept
symfony/rate-limiter	control job/API execution rate	can rate-limit Messenger transports too

16.6 Cache and performance

Package	Purpose	Important config/concept
symfony/cache	PSR-6 + Cache Contracts, Redis/Memcached/etc.	pools, tags, stampede protection
symfony/stopwatch	profiling timing helper	useful in dev/custom instrumentation
symfony/runtime	runtime abstraction/bootstrap	modern application runtime integration

16.7 External communication

Package	Purpose	Important config/concept
symfony/http-client	HTTP API client	timeouts, concurrency, retry policy around calls
symfony/mailer	email delivery	MAILER_DSN, Messenger integration possible
symfony/notifier	SMS/chat/push/email notification abstraction	provider DSNs, channel policy
mercure / MercureBundle	real-time SSE updates	hub publish/public URLs and JWT

Mailer example:

```
# config/packages/mailer.yaml
framework:
  mailer:
    dsn: '%env(MAILER_DSN)%'

MAILER_DSN=smtp://user:pass@smtp.example.com:587
```

16.8 UI and admin

Package	Purpose	When to use
twig/twig + symfony/twig-bundle	server-rendered templates	HTML, email templates, admin/internal pages
symfony/ux-*	Stimulus/Turbo/live components and frontend bridges	avoid full SPA complexity for moderate interactivity
easycorp/easyadmin-bundle	fast CRUD/admin panels	ORM-centric internal back office

Package	Purpose	When to use
sonata-project/admin-bundle	highly configurable enterprise admin	complex legacy/enterprise admin requirements
symfony/webpack-encore-bundle	Webpack integration	existing React/Twig asset pipelines
AssetMapper (symfony/asset-mapper)	modern no-bundler/static asset mapping	simpler Symfony-owned frontend assets

16.9 Logging, debug and observability

Package	Purpose	Important config/concept
symfony/monolog-bundle	structured application logging	channels, handlers, prod log levels
symfony/var-dumper	dump() and debugging	dev only; remove debugging leftovers
symfony/web-profiler-bundle	Symfony debug toolbar/profiler	development/test, never expose publicly in production
symfony/debug-bundle	debug integrations	dev/test

Example Monolog production configuration:

```
# config/packages/prod/monolog.yaml
monolog:
  handlers:
    main:
      type: stream
      path: php://stderr
      level: info
      formatter: monolog.formatter.json
```

Container environments usually prefer structured stdout/stderr logs and let the platform ship them to an aggregation system.

16.10 Testing and code quality

Package/tool	Purpose
symfony/test-pack	convenient Symfony test dependencies
phpunit/phpunit	unit/integration test runner
symfony/phpunit-bridge	Symfony PHPUnit integration/deprecation helpers
phpstan/phpstan	static analysis
friendsofphp/php-cs-fixer	automated code-style enforcement

Package/tool	Purpose
rector/rector	automated refactoring/version-upgrade assistance
infection/infection	mutation testing for test quality

16.11 Composer + Flex mental model

```
composer require symfony/messenger
```

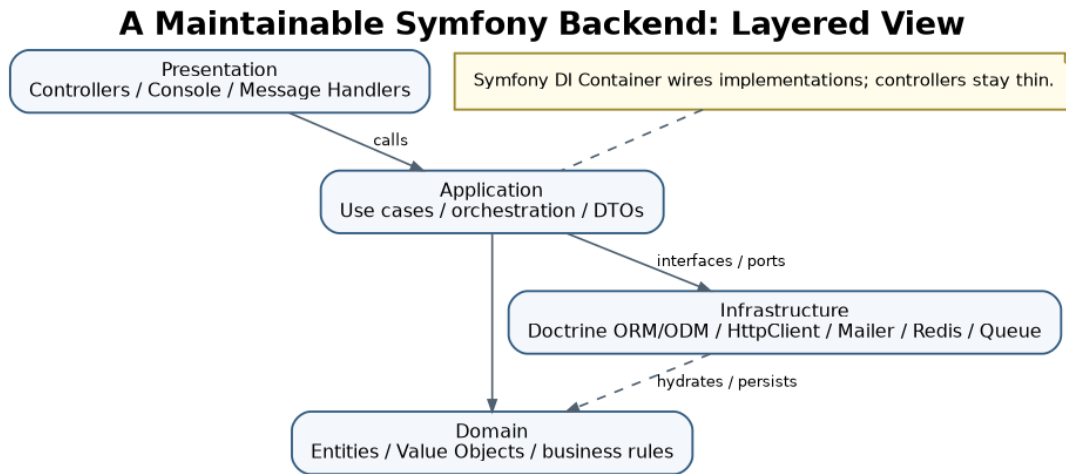
Composer installs packages. Symfony Flex can then apply a recipe that creates default config files, updates environment templates or registers a bundle. Always review the recipe-generated changes rather than treating Flex as magic.

Useful commands after package/config work:

```
php bin/console about
php bin/console debug:config framework
php bin/console debug:container --types
php bin/console debug:autowiring
php bin/console debug:router
php bin/console debug:event-dispatcher
composer recipes
composer audit
```

Part X — Architecture Synthesis

Section 17 — How the Pieces Fit Together in a Senior Symfony Backend



17.1 Example: create order request end-to-end

1. React posts JSON to `/api/orders`.
2. CORS/reverse proxy passes the request.
3. Security authenticates the token.
4. Router resolves the controller.
5. `#[MapRequestPayload]` maps JSON into `CreateOrderRequest`.
6. Validator rejects malformed data before business logic.
7. Controller calls `OrderApplicationService`.
8. Domain code enforces business invariants.
9. Repository persists through Doctrine ODM/ORM.
10. Service dispatches `OrderCreated` / `SendConfirmation` through Messenger.
11. Response DTO is serialized to stable JSON.
12. Worker handles slow side effects.
13. Cache entries/tags are invalidated.
14. Mercure may push a real-time update to React.
15. Logs/traces carry the request/order correlation ID.

17.2 What belongs where

Controller

GOOD: HTTP concerns, DTO input, response status, authorization invocation
 BAD: 150 lines of pricing, persistence and notification logic

Application service

GOOD: use-case orchestration and transaction boundaries
 BAD: HTTP Request/Response objects leaking into business logic

Domain

GOOD: business invariants, value objects, state transitions

BAD: framework service locator calls

Infrastructure

GOOD: Doctrine repositories, Redis, HTTP clients, queue adapters

BAD: deciding business rules that belong in the domain/application layer

Part XI — Final Comparison and Framework Judgment

Section 18 — Symfony vs Laravel: differences, problem fit and real-world usage

The goal in this interview is not to criticize Laravel. The JD explicitly requires Symfony because the employer needs someone who understands Symfony's architecture and ecosystem directly.

18.1 High-level philosophy

Topic	Symfony	Laravel
Core philosophy	component-oriented, explicit, highly configurable framework	highly integrated framework optimized for developer productivity and conventions
Learning curve	more concepts/configuration are visible; rewards architectural understanding	fast onboarding and cohesive “happy path”
DI	first-class compiled container, autowiring/autoconfiguration, explicit service patterns	powerful service container plus service providers; facades are common
Persistence	typically Doctrine Data Mapper, but framework does not require one persistence model	Eloquent Active Record is the default and deeply integrated
Routing	PHP attributes, YAML, XML or PHP config	route files / fluent route definitions
Templates	Twig	Blade
CLI	bin/console / Console component	artisan
Async	Messenger buses/transport/middleware	queues/jobs/events with driver abstraction
Auth	Security component: firewalls, authenticators, voters, access decision manager	guards/providers, gates/policies plus Sanctum/Passport ecosystem
Caching	Cache Contracts/PSR-6, explicit pools/adapters, HTTP cache ecosystem	unified Cache facade/driver model

Topic	Symfony	Laravel
Modularity	Symfony components are intentionally usable standalone	framework is cohesive, though individual Illuminate packages exist
Enterprise/long-lived systems	strong fit where explicit architecture, modularity and upgrade discipline matter	also supports large systems; often chosen for delivery speed and cohesive developer experience

18.2 Doctrine vs Eloquent is one of the biggest practical differences

Doctrine Data Mapper: entities are domain objects and persistence is handled separately by EntityManager/UnitOfWork/repositories.

```
$order = new Order($customerId);
$entityManager->persist($order);
$entityManager->flush();
```

Eloquent Active Record: models expose persistence/query behavior directly.

```
$order = Order::create([
    'customer_id' => $customerId,
]);
```

Neither model is automatically “better.” Data Mapper can produce stronger separation in complex domains; Active Record can be very productive for CRUD-heavy applications.

18.3 What problems Symfony is particularly good at

Symfony is a strong choice for:

- large, long-lived business platforms;
- enterprise APIs and integration-heavy systems;
- products that need strict module boundaries and explicit dependency management;
- multi-team backends where conventions need to remain stable for years;
- complex authorization/workflows;
- async/message-driven systems;
- headless backends for React/mobile clients;
- e-commerce/CMS platforms and extensible product ecosystems;
- systems that want to use only selected Symfony components rather than a full framework.

Symfony is not restricted to “enterprise.” It can build small apps too. The important question is whether its explicit architecture and flexibility justify the complexity for the project/team.

18.4 When Laravel may be the better choice

Laravel is often attractive when:

- rapid product delivery and a strong conventional stack are priorities;

- the application is CRUD-heavy and fits Eloquent naturally;
- the team already has Laravel expertise;
- first-party ecosystem tools solve most requirements with little custom architecture;
- developer ergonomics and quick onboarding outweigh the need for low-level configurability.

18.5 Real projects that use Symfony or Symfony components

Symfony's component model has broad reach. The official Symfony project directory lists projects such as **Drupal, Joomla, Sulu and Grav** in CMS; **Magento, PrestaShop, Sylius and Shopware** in e-commerce; and **Laravel itself** among frameworks that use Symfony packages. Shopware is a particularly relevant modern example of a major commerce platform built on a wide set of Symfony components.

This is a strong interview point: **Symfony is both a full-stack framework and an ecosystem of reusable infrastructure components used by other PHP systems.**

18.6 A safe interview answer: “Why Symfony instead of Laravel?”

“I don't frame it as one framework being universally better. Laravel gives a very productive integrated developer experience, and it's excellent when the application's shape fits its conventions. Symfony gives me more explicit control over service wiring, boundaries, security, messaging and infrastructure, and its component model is useful for complex, long-lived systems. In this role, the important thing is that I have genuine Symfony experience—services, the container, Security, Messenger, Doctrine/ODM and the request lifecycle—not Laravel knowledge translated into Symfony terminology.”

18.7 What not to say

Avoid:

- “Symfony is just a more complicated Laravel.”
- “Laravel is for juniors.”
- “Symfony is always more scalable.”
- “Eloquent is bad and Doctrine is good.”
- “I've used Symfony 8 extensively” when your production experience is primarily 2.8/4/5 and some 6.

18.8 Final framework judgment question

Q: If you were starting a new React + PHP product tomorrow, would you choose Symfony?

A senior response should ask about the domain, expected lifetime, team expertise, integration complexity, traffic profile, data model, auth requirements, release constraints and operational environment. Then make a trade-off-based recommendation rather than answering from framework loyalty.

Current-version awareness checkpoint — September 2026

- Symfony 8.1 is the current stable line and requires PHP 8.4+.
- Symfony 7.4 is the current LTS and requires PHP 8.2+.
- Symfony 6.4 remains a maintained branch.
- For your interview, present Symfony 2.8/4/5 as your strongest hands-on experience, some Symfony 6 hands-on experience, and 7/8 as current architectural/upgrade knowledge unless you genuinely used them in production.

Official references used for the production/API expansion

- [Symfony Documentation — Cache](#)
- [Symfony Documentation — HTTP Cache](#)
- [Symfony Documentation — Messenger](#)
- [Symfony Documentation — Scheduler](#)
- [Symfony Documentation — Deployment](#)
- [Symfony Documentation — Reverse Proxies](#)
- [Symfony Documentation — Serializer / Controller request payload mapping](#)
- [Symfony Documentation — Mercure](#)
- [Symfony Documentation — Rate Limiter](#)
- [Symfony Releases — current stable/LTS branches](#)
- [Doctrine MongoDB ODM Documentation](#)
- [NelmioCorsBundle Symfony recipe](#)
- [LexikJWTAuthenticationBundle documentation](#)
- [Symfony Projects — projects using Symfony packages](#)